



Advancements and Future Directions in Loss Function Designs for Physics-Informed Neural Networks: A Comprehensive Review

Mohsen Esmaeilbeigi¹ · Daniela Annunziata² · Salvatore Cuomo² · Francesco Piccialli² 

Received: 13 August 2025 / Revised: 13 June 2026 / Accepted: 17 August 2026 /
Published online: 29 August 2026
© The Author(s) 2026

Abstract

This review synthesizes recent advances in loss function designs for Physics-Informed Neural Networks (PINNs), a transformative approach to solving partial differential equations (PDEs) by embedding physical laws into deep learning frameworks. We begin by exploring the foundational role of loss functions in deep neural networks and their adaptation in PINNs to enforce PDE residuals, boundary conditions, and data constraints. The article systematically categorizes core loss components and early optimization strategies, including collocation point selection, which underpin effective PINN training. A comprehensive survey examines advanced loss function designs, including adaptive and self-adaptive weighting strategies, alternative formulations such as weak forms, energy-based, and gradient-enhanced methods, as well as uncertainty quantification and training enhancements like meta-learning, transfer learning, loss landscape engineering, regularization techniques, and optimization algorithms. We evaluated their impacts on convergence speed, predictive accuracy, and computational efficiency in applications such as fluid dynamics, solid mechanics, wave propagation, and geotechnical modeling. Challenges such as gradient imbalances, multi-scale dynamics, and high-dimensional scaling are critically analyzed, with emerging solutions like automated loss balancing, physics-aware regularization, and innovative architectures proposed to enhance robustness. A comparative analysis highlights performance trade-offs, while future directions emphasize dynamic optimization, hierarchical multi-fidelity losses, and scalable uncertainty quantification. This structured synthesis, drawing on developments

✉ Francesco Piccialli
francesco.piccialli@unina.it

Mohsen Esmaeilbeigi
m.esmaeilbeigi@malayeru.ac.ir

Daniela Annunziata
daniela.annunziata@unina.it

Salvatore Cuomo
salvatore.cuomo@unina.it

¹ Department of Artificial Intelligence and Data Science, Malayer University, Malayer, Iran

² Department of Mathematics and Applications “R Caccioppoli”, University of Naples Federico II, Naples, Italy

up to 2025, equips researchers with insights to refine PINN methodologies, bridging data-driven and physics-based paradigms for transformative scientific computing.

Keywords Deep neural networks · Physics-informed neural networks · Loss function designs · Scientific machine learning · Partial Differential equations

Abbreviations

AM-PINN	Adaptive movement PINN
AtPINN	Adaptive transfer learning PINN
BC-PINN	Boundary condition PINN
gPINN	Gradient-enhanced PINN
iPINN	Incremental PINN
lbPINN	Loss-balanced PINN
PDE	Partial differential equation
RAD	Residual-based adaptive distribution
SA-PINN	Self-adaptive PINN
VPINN	Variational PINN
VW-PINN	Volume-weighted PINN
AW-vsPINN	Adaptive weighted variable-separated PINN
B-PINN	Bayesian PINN
FO-PINN	First-order PINN
hp-VPINN	Hp-variational PINN
LA-PINN	Loss-Attentional PINN
MRF-PINN	Multi-receptive-field PINN
PINN	Physics-informed neural network
RAR-D	Residual-based adaptive refinement with distribution
VI-PINN	Variance-involved PINN
VS-PINN	Variable-scaling PINN
wbPINN	Weight-balanced PINN

1 Introduction

The rise of deep learning has transformed our ability to model complex systems, harnessing the power of neural networks to uncover patterns in vast datasets across domains such as computer vision [38], natural language processing [61] and beyond. At the heart of this revolution lies the loss function, a mathematical construct that measures the disparity between the predictions of a model and the truth of the ground, steering the optimization process toward solutions of remarkable accuracy [17, 18, 24, 57]. Traditionally, loss functions in deep learning are engineered to minimize empirical errors, relying solely on data to guide learning. However, in scientific domains where physical laws govern behavior, such as fluid dynamics, quantum mechanics, or heat transfer, data-driven models alone often fail to capture the underlying principles that dictate reality [8, 16, 19, 34]. Enter Physics-Informed Neural Networks (PINNs), a paradigm that marries the adaptability of neural networks with the certainty of physical laws, redefining how we approach problems rooted in partial differential equations (PDEs) [54].

PINNs offer a novel framework for solving PDEs by embedding their governing equations, along with initial and boundary conditions, directly into the loss function. This ingenious

design enables the neural network to learn solutions that respect both sparse data and the physical constraints imposed by the PDEs, achieving a synergy that traditional numerical methods and purely data-driven approaches struggle to replicate. Since their seminal introduction by Raissi et al. [54], PINNs have gained prominence, demonstrating their versatility in addressing forward simulations, inverse problems, and parameter estimation across a spectrum of applications, from modeling ocean currents [31] to designing advanced materials [45]. The loss function, as the linchpin of this framework, balances competing factors effectively between data fidelity, boundary enforcement, and PDE residuals, making its design pivotal to the success and scalability of PINNs [63].

This review article embarks on a journey through the evolution of loss function designs in PINNs, synthesizing foundational principles with state-of-the-art advancements documented up to 2025. We begin by laying the groundwork with an exploration of loss functions in deep neural networks, illuminating their role as the compass of machine learning optimization [24]. Next, we dive into the mechanics of PINNs, detailing how physical knowledge is seamlessly woven into the loss function to solve complex PDEs [54]. The heart of this review is a comprehensive literature survey that charts the progression of loss function designs, from early formulations to sophisticated strategies such as adaptive weighting [49], weak-form methods [35], and energy-based approaches [9] tailored to specific scientific challenges. Looking ahead, we face unresolved hurdles—such as optimizing multiterm loss functions, tackling high-dimensional systems, and integrating intricate physics [34]—and propose innovative directions, including automated loss balancing and physics-aware regularization, to propel PINNs into new frontiers.

To maintain a disciplined focus amidst the rapid expansion of physics-informed machine learning literature up to 2025, specific inclusion and exclusion criteria guided our selection of surveyed methodologies. This review strictly prioritizes works where the primary innovation lies in the mathematical formulation, regularization terms, or optimization dynamics of the loss function itself—such as adaptive weighting strategies, weak-form variations, and gradient enhancements within standard PINN architectures. Consequently, frameworks that primarily focus on infinite-dimensional operator mappings (e.g., DeepONets or Fourier Neural Operators) or hybrid models that decouple the physical constraints from the neural network's core optimization loop are excluded from this scope, unless they introduce generalized loss-balancing mechanisms directly transferable to vanilla PINN formulations.

The article unfolds in seven meticulously structured sections:

- **Section 2: Loss Functions in Deep Neural Networks:** A foundational overview of loss functions in deep learning, highlighting their role in guiding optimization and shaping model performance.
- **Section 3: Physics-Informed Neural Networks:** An introduction to the PINN framework, emphasizing its mathematical formulation and the integration of physical laws into the loss.
- **Section 4: Foundational Designs: Core Components and Optimization Strategies in Loss Function:** A systematic presentation of standard loss components in PINNs, early weighting strategies, and the role of collocation point optimization in shaping effective training procedures.
- **Section 5: Literature Review: Loss Function Designs in PINNs:** A comprehensive survey of advanced loss function designs in PINNs, including adaptive weighting, alternative formulations, uncertainty quantification, and optimization enhancements. For a comparative summary of these methods and their attributes, see Table 10.

- **Section 6: Comparative Analysis of Loss Function Designs Across PINN Applications:** A detailed evaluation of the performance, strengths, and limitations of various loss designs when applied across different PINN variants and physical contexts.
- **Section 7: Future Challenges and Directions in Loss Function Design for PINNs:** A forward-looking discussion of open problems and potential research directions that can further enhance the scalability, robustness, and applicability of PINNs.
- **Section 8: Conclusions:** A synthesis of key findings and a summary of the critical role of loss function design in the advancement of physics-informed machine learning.

Through this review, our objective is to equip researchers and practitioners with a definitive resource that underscores the transformative potential of loss function design in PINNs. As we approach the threshold of groundbreaking discoveries, refining these designs holds the key to unlocking unprecedented precision and scalability, marking a significant advance in physics-informed machine learning that bridges data, theory, and application with demonstrating notable precision.

2 Loss Functions in Deep Neural Networks

Note on Scope and Structure: The following foundational overviews of deep neural networks (Sect. 2), basic PINN architectures (Sect. 3), and core optimization components (Sect. 4) are provided not as a general tutorial, but as a necessary diagnostic framework. Advanced loss function designs in PINNs are fundamentally responses to the optimization bottlenecks, gradient pathologies, and spatial sampling inefficiencies inherent in baseline formulations. By first rigorously defining the composite nature of standard objective functions and their underlying gradient dynamics, we establish the critical mathematical context required to critically analyze the sophisticated adaptive weighting and spatial-temporal balancing strategies reviewed in subsequent sections.

In the realm of deep learning, a loss function—also referred to as a cost function or objective function—serves as a fundamental metric that quantifies the discrepancy between the predictions generated by a neural network and the true target values. This quantification of error is essential, as the primary objective during the training of a neural network is to minimize the loss function, thereby enhancing the model's predictive accuracy and its ability to generalize to unseen data. The design and selection of an appropriate loss function are critical, as they directly shape the optimization process and significantly influence the performance of the resulting model.

Mathematically, consider a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i$ represents the input features and y_i denotes the corresponding target values. A neural network, parameterized by a set of weights and biases $\mathbf{w}$, produces predictions $\hat{y}_i = f(\mathbf{x}_i; \mathbf{w})$. The loss function $\mathcal{L}(\mathbf{w})$ aggregates the prediction errors across all samples in the dataset as follows:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n l(y_i, \hat{y}_i).$$

Here, $l(y_i, \hat{y}_i)$ represents the individual loss for a single sample, which varies depending on the nature of the task at hand.

For regression problems, where the goal is to predict continuous target values, the Mean Squared Error (MSE) is a widely adopted loss function due to its simplicity and interpretability.

ity. The MSE measures the average squared difference between the actual and predicted values:

$$l_{\text{MSE}}(y, \hat{y}) = (y - \hat{y})^2.$$

Thus, the total loss over the dataset becomes:

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

The squaring of errors in MSE amplifies the penalty for larger deviations, making it particularly effective for problems where large errors are undesirable, though it can render the model sensitive to outliers.

In classification tasks, where the objective is to assign inputs to discrete categories, the Cross-Entropy loss is a standard choice. For binary classification, the loss for a single sample is defined as:

$$l_{\text{CE}}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})].$$

The total loss is then:

$$\mathcal{L}_{\text{CE}}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

For multi-class classification with C classes, the Cross-Entropy loss generalizes to:

$$l_{\text{CE}}(y, \hat{y}) = -\sum_{c=1}^C y_c \log(\hat{y}_c),$$

and the total one is,

$$\mathcal{L}_{\text{CE}}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}).$$

where y_c is a binary indicator (one-hot encoded) of the true class, and $\hat{y}_c$ is the predicted probability for class c . Cross-Entropy loss is grounded in the principle of maximum likelihood estimation, making it well-suited for probabilistic outputs.

The choice of loss function is paramount for several reasons:

- **Optimization Dynamics:** The loss function defines the optimization landscape, influencing convergence speed and the risk of getting trapped in local minima.
- **Error Sensitivity:** For example, MSE's quadratic nature emphasizes large errors, whereas the Mean Absolute Error (MAE), defined as $l_{\text{MAE}}(y, \hat{y}) = |y - \hat{y}|$, offers robustness to outliers but may under-penalize significant deviations.
- **Task Alignment:** Cross-Entropy aligns with probabilistic classification, while custom loss functions can address specific challenges, such as class imbalance, by weighting rare classes more heavily.

A key practical consideration in deep learning is that the loss function must be differentiable with respect to the model parameters $\mathbf{w}$. This property enables the use of gradient-based optimization techniques, such as Stochastic Gradient Descent (SGD), which rely on the computation of gradients to iteratively update the weights. Common loss functions like MSE and Cross-Entropy satisfy this requirement, ensuring efficient training.

In summary, the loss function is a cornerstone of deep neural network design, bridging the gap between model predictions and real-world targets. The training process of a neural network, including forward and backward propagation. This iterative process adjusts the weights to minimize the loss function, enhancing the accuracy of the model. Its careful formulation lays the groundwork for effective learning, a principle that becomes even more intricate and fascinating when extended to Physics-Informed Neural Networks, as we will explore in subsequent sections.

The overall architecture and training process of a PINN is illustrated in Fig. 1, which shows how the loss function integrates physical laws, initial and boundary conditions, and available data.

To address inverse problems involving both unknown PDE parameters and neural network weights, the standard PINN framework can be extended by jointly optimizing over both sets of variables. Figure 2 illustrates this generalized formulation, where the loss function incorporates terms not only for enforcing the PDE, initial, and boundary conditions, but also for fitting available data. Automatic differentiation is used to compute derivatives with respect to spatial and temporal inputs, and the optimization simultaneously updates both the network parameters θ and the PDE parameters λ , enabling the recovery of hidden dynamics in time-space domains [5].

3 Physics-Informed Neural Networks

PINNs offer a novel approach to solving partial differential equations by merging deep learning techniques with the physical laws governing the problem. Unlike conventional numerical methods—such as finite difference or finite element methods—that rely on domain discretization and iterative solvers, PINNs employ neural networks to approximate PDE solutions while directly embedding the physics of the problem into the training process. This is achieved through a loss function that balances data-driven predictions with adherence to physical constraints, making PINNs versatile for both forward and inverse problems.

The core idea of PINNs is to train a neural network $\hat{u}(\mathbf{x}; \mathbf{w})$, parameterized by weights $\mathbf{w}$, to represent the solution $u(\mathbf{x})$ of a PDE. The network learns by minimizing a composite loss function that incorporates the PDE itself, boundary conditions, and any available data. This physics-informed approach enables PINNs to produce solutions that are consistent with the underlying physical principles, even when data is sparse or noisy. PINNs have been successfully applied to problems in fluid dynamics, heat transfer, and material science, demonstrating their ability to capture complex physical phenomena.

To illustrate how PINNs incorporate physics, consider a general PDE defined over a domain Ω :

$$\mathcal{N}[u(\mathbf{x})] = f(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

where $\mathcal{N}$ is a differential operator (e.g., involving spatial or temporal derivatives), $u(\mathbf{x})$ is the solution, and $f(\mathbf{x})$ is a source or forcing term. The PDE is accompanied by boundary conditions:

$$\mathcal{B}[u(\mathbf{x})] = g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega$$

In PINNs, the neural network $\hat{u}(\mathbf{x}; \mathbf{w})$ approximates $u(\mathbf{x})$. The physics of the problem is enforced by designing a loss function $\mathcal{L}(\mathbf{w})$ that penalizes deviations from the PDE, boundary conditions, and observed data. The total loss is typically a weighted sum of three terms:

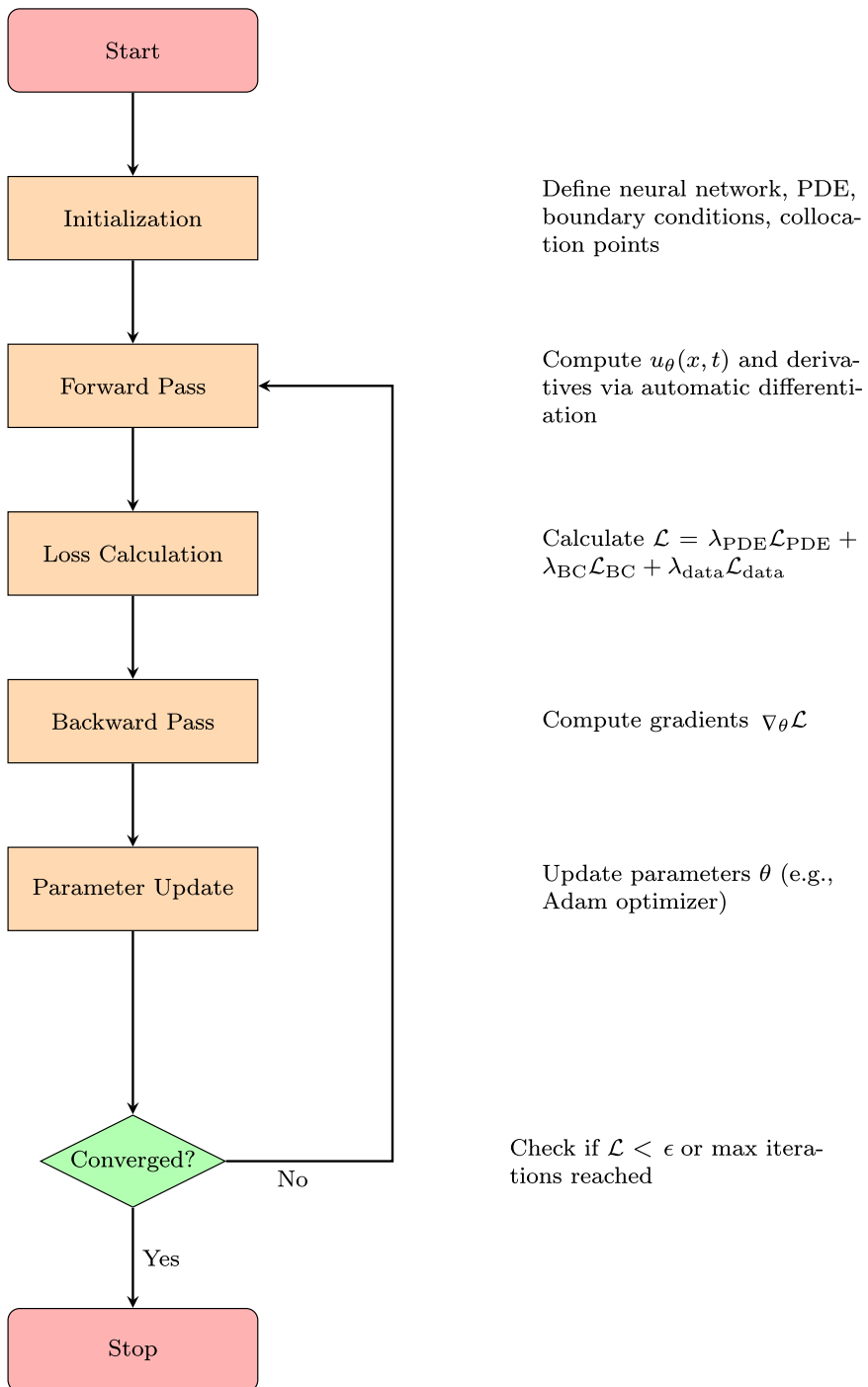


Fig. 1 Workflow diagram of the PINN training loop, illustrating the iterative process of optimizing the loss function to solve PDEs

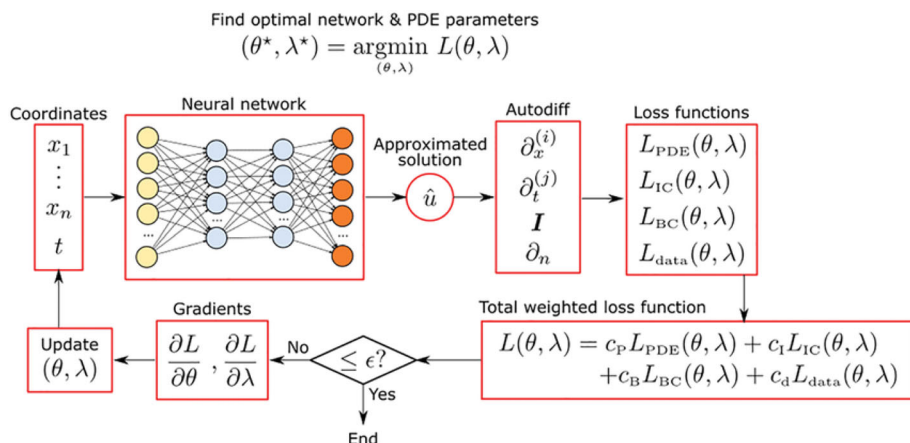


Fig. 2 Training loop for a Physics-Informed Neural Network (PINN). Starting from space–time coordinates $(x_1, \dots, x_n, t)$, the network yields an approximate solution $\hat{u}$. Automatic differentiation provides the derivatives needed to evaluate the four weighted loss components: the PDE residual L_{PDE} , initial condition loss L_{IC} , boundary condition loss L_{BC} , and data misfit loss L_{data} . These combine linearly as $L = c_P L_{\text{PDE}} + c_I L_{\text{IC}} + c_B L_{\text{BC}} + c_d L_{\text{data}}$. If $L > \varepsilon$ (a user-defined tolerance), gradients $\partial L / \partial \theta$ and $\partial L / \partial \lambda$ guide the update of the network weights θ and any unknown physical parameters λ towards a local minimum, thereby solving the optimisation problem $(\theta^*, \lambda^*) = \arg \min_{\theta, \lambda} L(\theta, \lambda)$ [5]

1. **Data Loss:** This term ensures the network matches any available measurements $\{(\mathbf{x}_i, u_i)\}_{i=1}^{N_d}$:

$$\mathcal{L}_{\text{data}}(\mathbf{w}) = \frac{1}{N_d} \sum_{i=1}^{N_d} (\hat{u}(\mathbf{x}_i; \mathbf{w}) - u_i)^2$$

2. **Boundary Condition Loss:** This enforces the boundary conditions at points $\{\mathbf{x}_j\}_{j=1}^{N_b} \in \partial\Omega$:

$$\mathcal{L}_{\text{bc}}(\mathbf{w}) = \frac{1}{N_b} \sum_{j=1}^{N_b} (\mathcal{B}[\hat{u}(\mathbf{x}_j; \mathbf{w})] - g(\mathbf{x}_j))^2$$

3. **PDE Residual Loss:** This term embeds the physics by minimizing the residual of the PDE at collocation points $\{\mathbf{x}_k\}_{k=1}^{N_c} \in \Omega$:

$$\mathcal{L}_{\text{pde}}(\mathbf{w}) = \frac{1}{N_c} \sum_{k=1}^{N_c} (\mathcal{N}[\hat{u}(\mathbf{x}_k; \mathbf{w})] - f(\mathbf{x}_k))^2$$

The overall loss function is:

$$\mathcal{L}(\mathbf{w}) = \lambda_d \mathcal{L}_{\text{data}}(\mathbf{w}) + \lambda_b \mathcal{L}_{\text{bc}}(\mathbf{w}) + \lambda_p \mathcal{L}_{\text{pde}}(\mathbf{w}),$$

where λ_d , λ_b , and λ_p are weighting factors that adjust the relative importance of each term. The PDE residual loss is particularly crucial, as it directly incorporates the physical laws into the training process. For example, in a heat transfer problem, $\mathcal{N}$ might represent the heat equation $\frac{\partial u}{\partial t} - \alpha \nabla^2 u$, and the residual loss ensures that the network's predictions satisfy this equation across the domain.

This formulation allows PINNs to leverage automatic differentiation to compute the derivatives in $\mathcal{N}[\hat{u}]$, making the approach mesh-free and adaptable to irregular geometries.

Furthermore, by including unknown parameters (e.g., material properties) as learnable variables in the network, PINNs can solve inverse problems, inferring these parameters while fitting the data and satisfying the physics.

The seminal work by Raissi et al. [54] demonstrated this approach with the Burgers' equation, a nonlinear PDE relevant to fluid dynamics. Their PINN combined sparse initial and boundary data with PDE residuals evaluated at random collocation points, yielding a solution that respected both the data and the physical constraints. This integration of physics into the loss function distinguishes PINNs from purely data-driven models, enabling physically consistent predictions and robust generalization.

It is important to clarify that the methodology described above, which forms the primary focus of this review, pertains specifically to *continuous PINNs*. In this continuous-time formulation, the neural network takes continuous spatial and temporal coordinates as inputs. However, as originally introduced by Raissi et al. [54], the PINN framework also encompasses *discrete PINNs*. Discrete PINN formulations employ classical numerical schemes, such as implicit Runge–Kutta methods, to handle discrete time-stepping, thereby transforming the continuous PDE into a system of structured equations. While discrete PINNs offer distinct advantages for certain dynamical systems, this review predominantly centers on the loss function topologies of continuous PINNs, given their widespread adoption and the specific optimization challenges they present in the literature.

4 Foundational Designs: Core Components and Optimization Strategies in Loss function

The foundational designs in PINNs encompass the core principles and components that define the architecture and training process of these models, forming the bedrock upon which advanced techniques are built. Central to PINNs is the loss function, which integrates data fidelity, boundary conditions, and the governing PDEs into a cohesive framework that guides the neural network toward an accurate solution. A thorough understanding of these foundational elements is essential, as they directly shape the construction and optimization of the loss function, ultimately determining the efficiency and effectiveness of PINN training. In this section, we explore the standard components of the loss function, tracing the early evolution of weighting strategies that balance these terms and laying the groundwork for more sophisticated approaches. We also examine techniques for optimizing collocation points, which play a critical role in enforcing physical constraints across the computational domain. Together, these foundational designs provide the necessary background for appreciating the innovations in loss function design discussed in subsequent sections. A visual overview of these concepts is presented in Fig. 3.

4.1 Standard Loss Function Components

The foundational framework of PINNs was established by Raissi et al. [54], who introduced the core loss function components: data loss, boundary condition loss, and PDE residual loss. Data loss measures the mismatch between the neural network predictions and observed data, ensuring fidelity to measurements; the loss of the boundary condition enforces the prescribed constraints at the domain boundaries; and the residual loss of the PDE ensures that the neural network satisfies the governing PDE at a set of collocation points within the domain. For example, in their application to Burgers' equation, Raissi et al. [54] define the loss as the sum

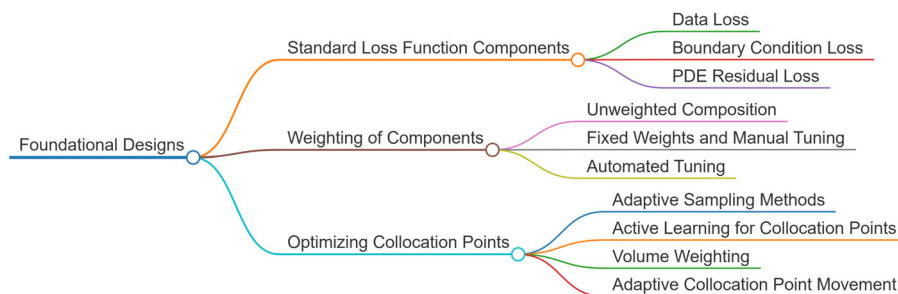


Fig. 3 Mindmap illustrating the foundational designs in PINNs, encompassing loss function components, weighting strategies, and collocation point optimization techniques

of the mean squared error of the data points (including initial and boundary conditions) and the PDE residuals, demonstrating the practical utility of these components in problems such as the Schrödinger equation and the Navier–Stokes equations. This seminal work provided a robust starting point for solving forward and inverse PDE problems using neural networks, although it did not address the potential challenges arising from the relative scales of these loss terms.

4.2 Weighting of Components

In Physics-Informed Machine Learning, particularly within the PINNs framework, the loss function serves as a cornerstone by quantifying the discrepancy between the predictions of the neural network and the known physical constraints derived from the PDEs. These constraints encompass boundary conditions, ODE/PDE residuals, and observational data. Training a PINN involves iteratively minimizing this loss to align the model's output with the underlying physical system. The general form of the loss function is given by:

$$L(\theta) = \sum_{\alpha \in C} m_{\alpha} \int_{\Omega_{\alpha}} r_{\alpha}(x, \theta) dx$$

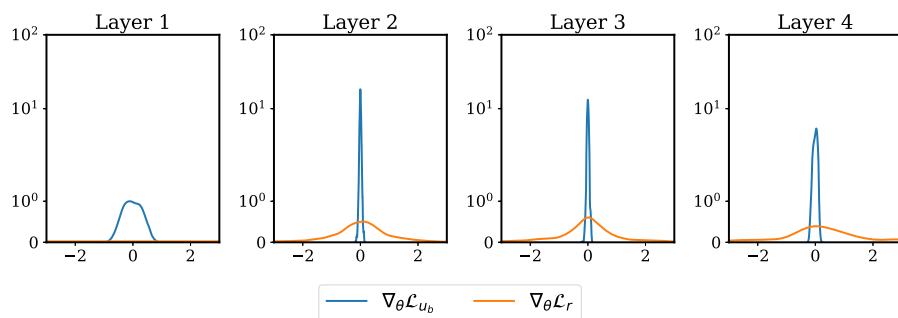
where: θ denotes the neural network parameters and potential hyperparameters in the PDE, $C = \{D, B, E, \dots\}$ represents the set of indices for distinct loss components, such as data fidelity (D), boundary conditions (B), and PDE residuals (E), m_{α} are global scalar weights that regulate the relative influence of each component, Ω_{α} is the domain corresponding to component α , $r_{\alpha}(x, \theta)$ is the residual for component α , measuring the deviation of the neural network's output from the expected physical behavior.

For computational feasibility, the continuous integral is approximated over discrete sub-domains $X_{\alpha} \subset \Omega_{\alpha}$, yielding the discretized loss function:

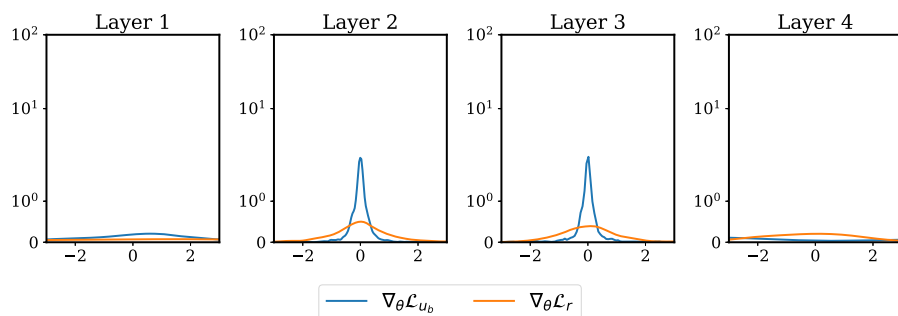
$$L(\theta, X_{\alpha}) = \sum_{\alpha \in C} m_{\alpha} \frac{1}{N_{\alpha}} \sum_{i=1}^{N_{\alpha}} \lambda_{\alpha,i} r_{\alpha}(x_i, \theta)$$

where: $X_{\alpha} \subset \Omega_{\alpha}$ is a finite batch of sampled points, N_{α} is the number of points in the batch X_{α} , $x_i \in X_{\alpha}$ are points sampled from Ω_{α} using a specified sampling strategy, $\lambda_{\alpha,i}$ are local weights that adjust the contribution of the residual at each point.

Crafting an effective loss function is paramount to ensure that the neural network's predictions satisfy the PDE solution accurately. The strategy for weighting these loss components



A: Histograms of back-propagated gradients $\nabla_{\theta} \mathcal{L}_r(\theta)$ and $\nabla_{\theta} \mathcal{L}_{u_b}(\theta)$ at each layer after 40,000 iterations using standard PINN.



B: Histograms of back-propagated gradients at each layer using learning rate annealing after 40,000 iterations.

Fig. 4 Comparison of gradient variations in standard and balanced PINNs for solving the Helmholtz equation [63]

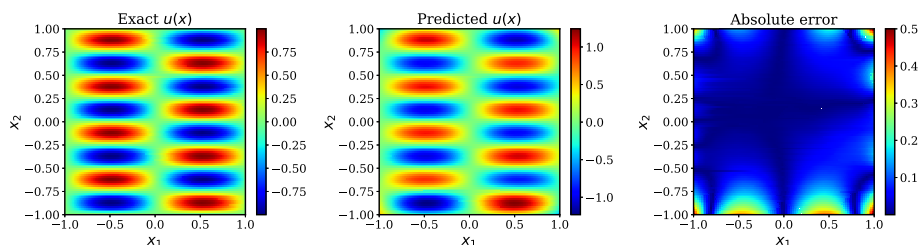
has undergone significant evolution since PINNs were first introduced, driven by the need to overcome training inefficiencies and enhance convergence.

The seminal work by Raissi et al. [54] adopted a simple, unweighted summation of loss components—combining data, boundary, and PDE residual terms without explicit weighting coefficients. While intuitive, this approach often resulted in suboptimal training dynamics due to the disparate scales of the loss terms. For example, in problems like the Burgers' equation, the PDE residual term, which involves higher-order derivatives, tends to dominate the loss, leading to unbalanced optimization and slow convergence.

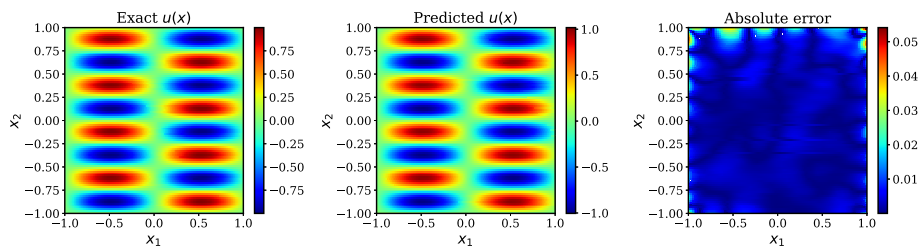
This limitation was later dissected by Wang et al. [63], who identified gradient pathologies as a fundamental issue in early PINN designs. Their analysis revealed that unweighted loss terms produce gradients with widely varying magnitudes, as evidenced by histograms of back-propagated gradients for a standard PINN applied to the Helmholtz equation (see Fig. 4A from Wang et al. [63]).

Early efforts to address this relied on fixed weights and manual tuning, where practitioners adjusted the global weights m_{α} through iterative experimentation. This method, while effective in some cases, was labor-intensive and highly dependent on the specific problem, lacking generalizability.

To move beyond this ad hoc approach, Wang et al. [63] introduced learning rate annealing, an automated technique that dynamically adjusts m_{α} based on the relative gradient



A: Exact vs. predicted solution using standard PINN (relative L_2 error: 1.81×10^{-1}).



B: Exact vs. predicted solution using learning rate annealing (relative L_2 error: 1.27×10^{-2}).

Fig. 5 Two-dimensional Poisson's equation with steep exact solution. Top panel: **A** Exact solution, **B** PINN prediction, and **C** PINN point-wise error. Bottom panel: **(D column)** h-refinement via different domain decomposition, **(E column)** hp-VPINN prediction, and **(F column)** hp-VPINN point-wise error. Adapted from Wang et al. [63]

magnitudes of each loss term. This innovation significantly improved training efficiency. For example, in solving the Helmholtz equation, a conventional PINN with unweighted loss achieved a relative L_2 error of 1.81×10^{-1} after 40,000 iterations (see Fig. 5A), while applying learning rate annealing reduced the error to 1.27×10^{-2} (see Fig. 5B).

The balanced gradient distributions resulting from this method, shown in Fig. 4B, stand in stark contrast to the imbalanced distributions of the unweighted approach in Fig. 4A, highlighting its efficacy.

More recently, Gao et al. [22] developed an adaptive weighting algorithm that adjusts m_α using a softmax-like formulation based on the relative magnitudes of the loss terms. This method has demonstrated superior performance compared to original standard PINNs across complex wave equations (e.g., the Benjamin-Ono, Sine-Gordon, and Mukherjee-Kundu equations). This superiority was established based on the criterion of achieving significantly lower relative prediction errors—evaluated using the L_2 norm between the exact and predicted solutions—thereby further refining the automation of loss balancing.

Subsequent advancements built on this foundation. McClenny and Braga-Neto [49] proposed self-adaptive PINNs, where the local weights $\lambda_{\alpha,i}$ are treated as trainable parameters optimized alongside θ via gradient descent. This approach enables the model to prioritize regions with large residuals or complex dynamics, enhancing convergence for challenging PDEs like the Allen-Cahn and Navier–Stokes equations.

In addition to the weighting strategies introduced in this subsection, various other approaches for adaptive and self-adaptive loss rebalancing exist and will be discussed in the upcoming subsection titled *Adaptive and Self-Adaptive Weighting Strategies*.

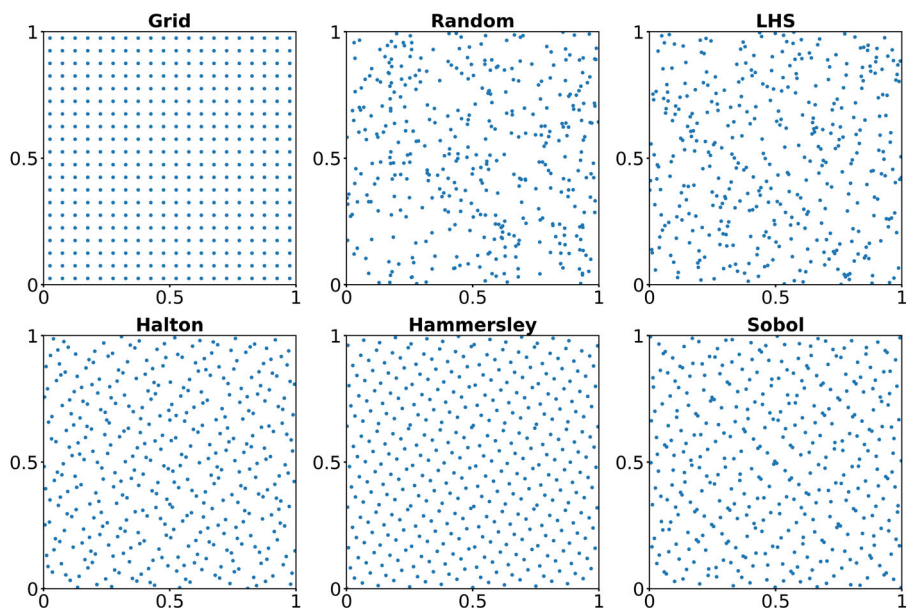


Fig. 6 Examples of 400 points generated in $[0, 1]^2$ using different uniform sampling methods [65]

Moreover, as seen in the mathematical formulation of the loss function, the choice of collocation points X_α plays a pivotal role in shaping the optimization landscape and the success of training. Different sampling strategies for these points—ranging from uniform and Latin hypercube sampling to residual-based refinement—will be explored in the next subsection titled *Optimizing Collocation Points*.

4.3 Optimizing Collocation Points

Techniques for optimizing the selection and weighting of collocation points within the domain where PDE residuals are evaluated have become critical to improving the efficiency and accuracy of PINNs, particularly for complex PDEs with sharp solutions or irregular domains. The distribution of these points significantly influences the training process, as it determines how effectively the neural network approximates the PDE solution across the domain. A summary of these strategies is provided in Table 1.

A comprehensive study by Wu et al. [65] systematically evaluated sampling methods for collocation points in PINNs, categorizing them into non-adaptive uniform sampling and adaptive nonuniform sampling. They investigated six uniform sampling methods: (1) equispaced uniform grid, (2) uniformly random sampling, (3) Latin hypercube sampling, (4) Halton sequence, (5) Hammersley sequence, and (6) Sobol sequence. Examples of 400 points generated in $[0, 1]^2$ using these methods are illustrated in Fig. 6. While uniform sampling provides a straightforward approach, it often proves inadequate for PDEs with localized features or sharp gradients, prompting the development of more sophisticated adaptive techniques.

Adaptive sampling methods dynamically adjust the distribution of collocation points during training to focus computational effort on regions where PDE residuals are large or the solution exhibits complex behavior. Mao and Meng [48] proposed a residual and gradient-

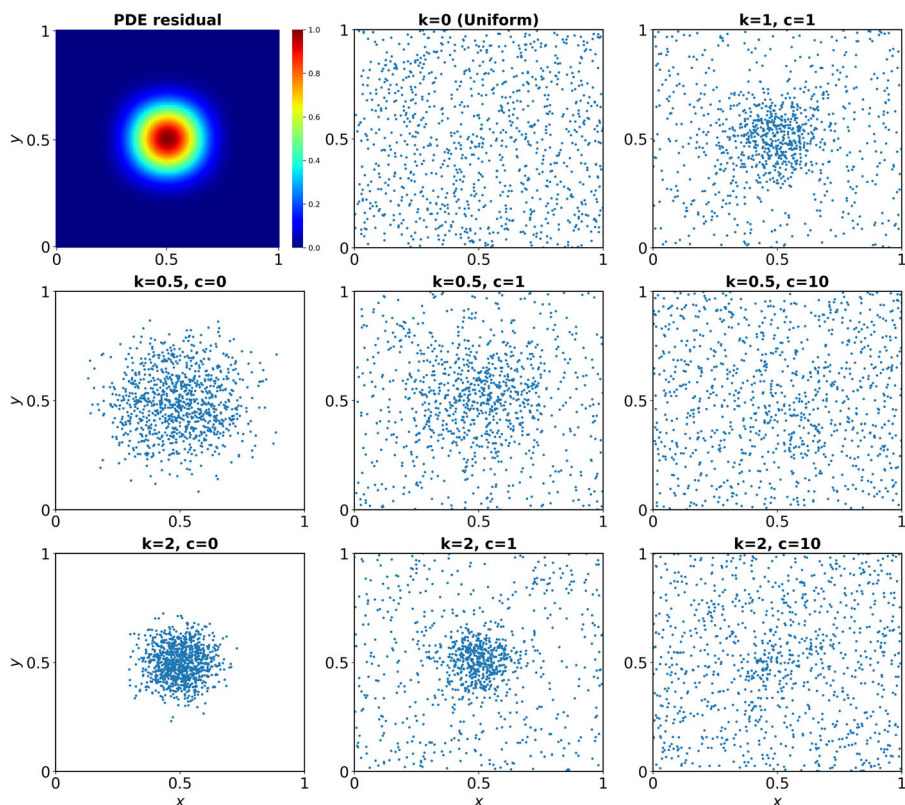


Fig. 7 Examples of 1000 residual points sampled by RAD with different values of k and c for the PDE residual $\varepsilon(x, y)$ [65]

based adaptive sampling strategy that redistributes points to areas with large PDE residuals or high solution gradients, proving effective for problems like the Burgers' and Korteweg-de Vries equations, which feature sharp solutions. Building on this foundation, Wu et al. [65] introduced two novel residual-based adaptive sampling methods: Residual-based Adaptive Distribution (RAD) and Residual-based Adaptive Refinement with Distribution (RAR-D). These methods enhance the distribution of collocation points by resampling or refining them based on PDE residuals during training.

In the RAD method, collocation points are resampled according to a probability density function (PDF) defined as:

$$p(x) \propto \frac{\varepsilon^k(x)}{E[\varepsilon^k(x)] + c},$$

where $\varepsilon(x)$ is the PDE residual at point x , $k \geq 0$ and $c \geq 0$ are hyperparameters controlling the distribution's focus, and $E[\varepsilon^k(x)]$ is the expected value of $\varepsilon^k(x)$ over the domain, approximated via Monte Carlo methods. By tuning k and c , RAD balances the emphasis between regions with large residuals and uniform coverage. Figure 7 demonstrates how varying k and c alters the sampling distribution for a sample PDE residual, providing visual insight into the method's flexibility.

The RAR-D method integrates residual-based refinement (akin to Mao and Meng [48]) with distribution-based RAD sampling, iteratively adding points according to the same PDF. Wu et al. [65] compared these methods with uniform sampling and existing adaptive strategies, such as RAR-G (Residual-based Adaptive Refinement with Greed) and Random-R (a baseline dynamic strategy where collocation points are periodically resampled across the domain following a uniform random distribution), in forward and inverse PDE problems. For the diffusion–reaction equation with an inverse problem setup, RAD and RAR-D achieved lower relative errors L_2 with fewer residual points than uniform methods, as shown in Fig. 8.

Further advancing adaptive techniques, Luo et al. [46] proposed a hybrid adaptive (HA) sampling method. This approach combines residual-based sampling with random resampling to balance exploration and exploitation, reducing the number of points required for efficient training in applications like the advection–diffusion and Navier–Stokes equations. The HA method alternates between two strategies: random resampling every N iterations to ensure stochasticity, and adaptive resampling that prioritizes points with large PDE residuals, as depicted in Fig. 9. Figure 10 illustrates the residual points resampled during training for a 2D Poisson equation, with blue points randomly sampled and red points adaptively selected, concentrating in regions with high residuals or solution peaks. Figure 11 demonstrates the superior performance of HA sampling, reducing the L_2 relative error of the solution u by approximately one order of magnitude and the parameter function f by 75% compared to vanilla PINNs in a 1D Poisson inverse problem.

Active learning offers another avenue for optimization by selecting collocation points based on uncertainty quantification. Aikawa et al. [1] utilized variational inference to estimate uncertainty in PINN predictions, preferentially sampling points in high-uncertainty regions, such as near boundaries in the heat or wave equations. This approach minimizes the number of points needed while preserving accuracy, showcasing the power of probabilistic methods in PINN training.

Volume weighting tackles nonuniform point distributions by adjusting each collocation point's contribution to the PDE residual loss. Song et al. [58] introduced Volume-Weighted PINNs (VW-PINNs), where residuals are weighted by the volume each point represents—computed via Voronoi tessellation—ensuring balanced PDE enforcement across regions of varying density, as demonstrated on the Poisson and Navier–Stokes equations.

Alternatively, adaptive collocation point movement iteratively adjusts the positions of a fixed set of points rather than resampling. Hou et al. [29] proposed the Adaptive Movement PINN (AM-PINN) framework. This method divides collocation points into fixed (PN_{rn}) and movable (PN_{rm}) sets, with the latter dynamically adjusted during training using gradient-based updates to concentrate in regions with high residuals or solution gradients, such as in the Burgers' and Allen–Cahn equations. The AM-PINN framework is illustrated in Fig. 12. Figure 13 shows an example distribution of 2000 collocation points, split evenly between fixed and movable sets, for a 2D function. Figure 14 demonstrates how the parameter k in the probability density function $p(x) = \frac{\omega^k(x)}{\int \omega^k(x)}$, where $\omega(x) = u(x, y)$, controls the concentration of movable points, enhancing adaptability to specific problem characteristics.

Collectively, these strategies—from uniform sampling baselines to advanced adaptive methods—demonstrate the diverse approaches to optimizing collocation points in PINNs. By addressing the shortcomings of uniform sampling, as highlighted by Wu et al. [65], and leveraging visual aids from key studies, these techniques enhance PINNs' applicability to challenging PDE problems, paving the way for further innovations in loss function design.

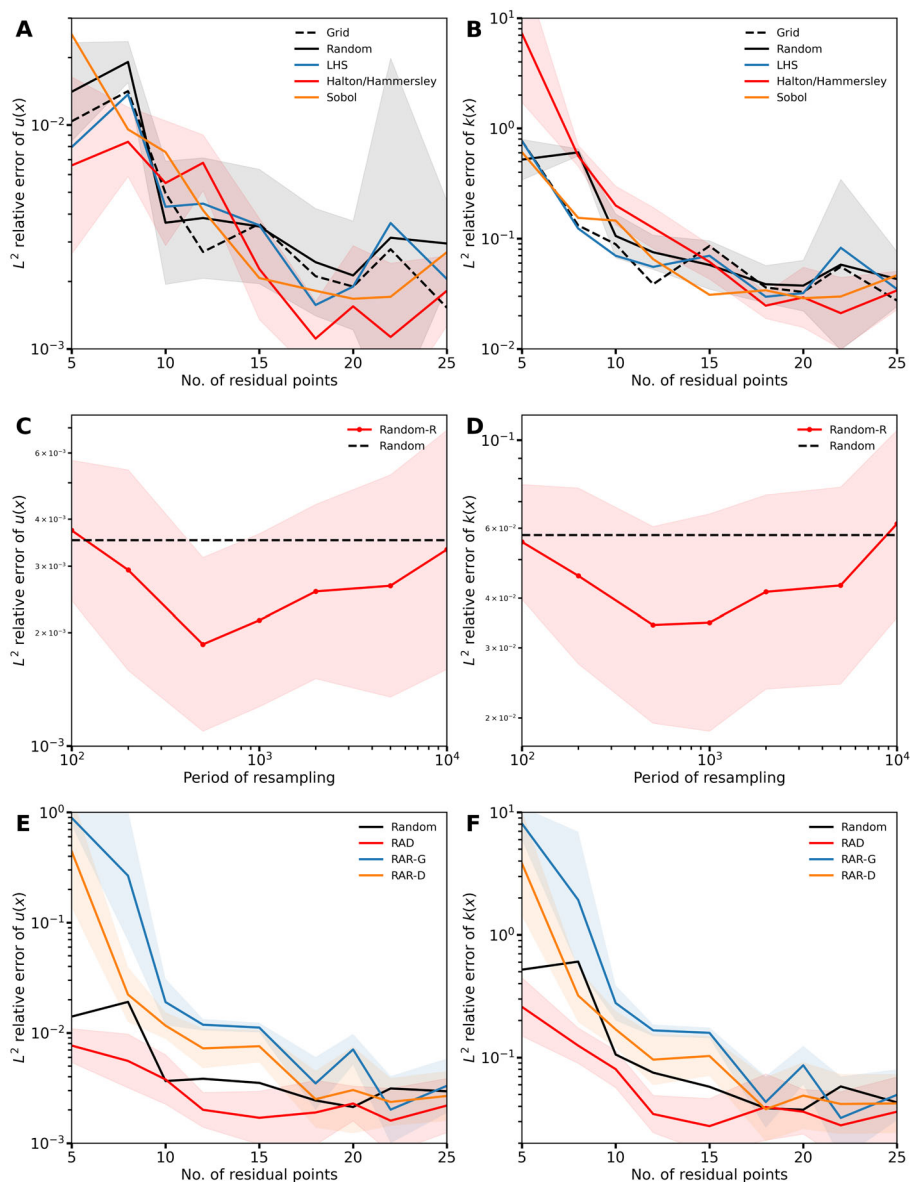


Fig. 8 L_2 relative errors of different sampling methods for u and k in the diffusion-reaction equation. **A, B** Six uniform sampling methods with fixed residual points. **C, D** Random-R with different periods of resampling using 15 residual points. **E, F** Comparison among Random, RAD ($k = 1, c = 1$), RAR-G, and RAR-D ($k = 2, c = 0$) for different numbers of residual points. The curves and shaded regions represent the geometric mean and one standard deviation of 10 runs [65]

5 Literature Review: Loss Function Designs in PINNs

The loss function serves as the cornerstone of PINNs, enabling these models to embed physical laws, boundary conditions, and observational data into a unified optimization framework

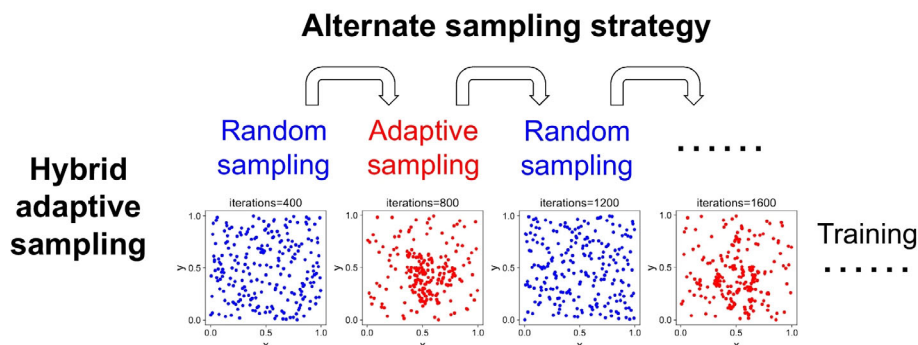


Fig. 9 The process of the hybrid adaptive sampling method [46]

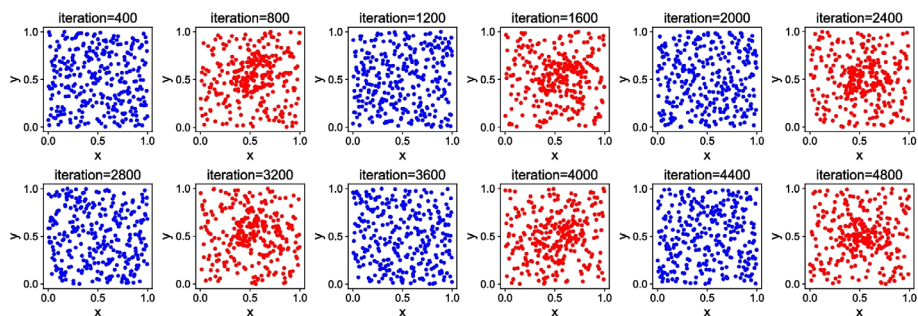


Fig. 10 Example of 2D Poisson equation. The residual points in the domain resampled by HA method during training. The number of residual points is 300. The blue points are randomly resampled during the iterations. The red points are acquired by adaptive sampling [46]

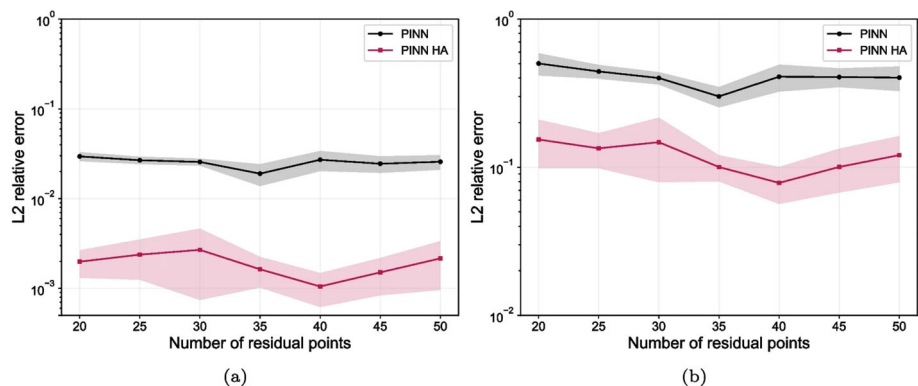


Fig. 11 Example of 1D Poisson equation (inverse). The number of training epochs is 20000. **a** L_2 relative error of u with different numbers of residual points. **b** L_2 relative error of f with different numbers of residual points [46]

for solving PDEs. Building upon the foundational formulations discussed in the previous chapter, this section provides a comprehensive review of advanced loss function designs in PINNs, covering developments up to 2025. We examine a broad range of innovations, including adaptive and self-adaptive weighting techniques that address gradient imbalances,

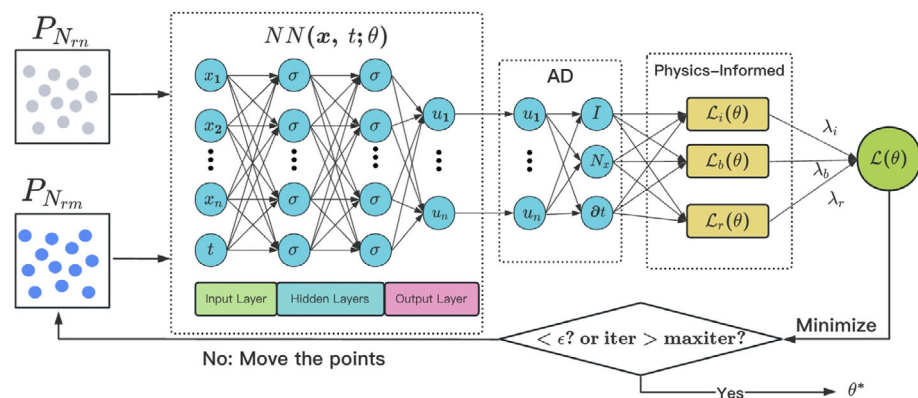


Fig. 12 AM-PINN: Adaptive collocation point movement for PINNs. $P_{N_{rn}}$ is the set of fixed collocation points, which are fixed during the model training and generated by the uniform sampling strategy. $P_{N_{rm}}$ is the set of movable collocation points which can be dynamically changed during the training process. Here, AD denotes Automatic Differentiation for computing the PDE residuals [29]

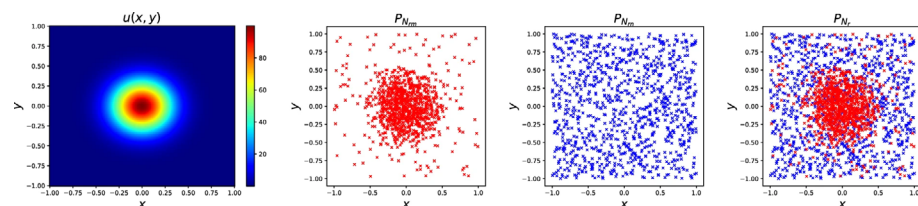


Fig. 13 An example of collocation points: $P_{N_r} = P_{N_{rn}} \cup P_{N_{rm}}$, $N_r = 2000$, $N_{rn} = N_{rm} = 1000$ [29]

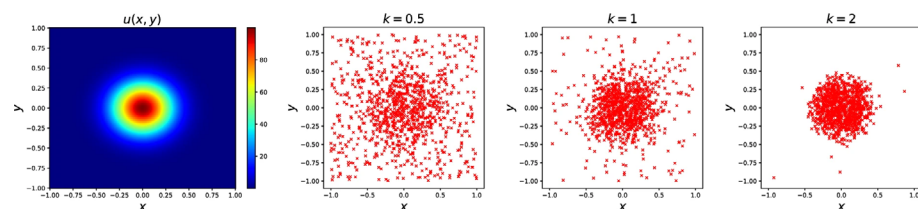


Fig. 14 The parameter k can control the degree of concentration of sampling points, where $\omega(x, y) = u(x, y)$ and $k = 0.5, 1, 2$ [29]

alternative formulations such as weak forms and energy-based methods that improve solution accuracy, and enhancements in training and optimization—such as meta-learning and loss landscape engineering—that boost learning efficiency. Additionally, we explore the integration of novel architectures and uncertainty quantification approaches, both of which have reshaped the design of loss functions to better handle noisy, sparse, or multiscale data. Together, these developments highlight the evolving role of loss function design in improving the accuracy, robustness, and generalization of PINNs across diverse scientific and engineering domains. A taxonomy of these designs is illustrated in Fig. 15.

Table 1 Summary of strategies for optimizing collocation points in PINNs

Strategy	References	Description
Adaptive Sampling Methods	Mao and Meng [48]	Physics-informed neural networks with residual/gradient-based adaptive sampling for sharp solutions
Residual-based Adaptive Sampling	Wu et al. [65]	Proposed RAD and RAR-D for adaptive sampling based on PDE residuals
Hybrid Adaptive Sampling	Luo et al. [46]	An enhanced hybrid adaptive sampling scheme for more efficient training
Active Learning for Collocation Points	Aikawa et al. [1]	Used variational inference and uncertainty quantification to select informative points
Volume Weighting	Song et al. [58]	VW-PINNs: volume-based weighting of residuals to handle nonuniform point distributions
Adaptive Collocation Point Movement	Hou et al. [29]	Dynamically moves collocation points based on local residual information

5.1 Adaptive and Self-Adaptive Weighting Strategies

In PINNs, the loss function is a critical component, typically comprising a weighted sum of terms that enforce data fidelity, boundary conditions, and the underlying PDEs. The challenge lies in determining the optimal weights for these terms, as their scales and contributions can vary significantly depending on the problem, often leading to gradient imbalances or poor convergence. Adaptive and self-adaptive weighting strategies offer a solution by dynamically adjusting these weights during training, ensuring a balanced optimization process. These strategies are vital for PINNs because they enhance training efficiency, prevent any single loss term from dominating, and enable the network to better capture complex PDE behaviors, such as multi-scale phenomena or nonlinear dynamics. By refining the loss function's structure, these methods improve the robustness and accuracy of PINN solutions. This subsection explores five key categories of such strategies and their influence on the loss function's design and optimization. An overview of these adaptive and self-adaptive weighting strategies, including their key references and core ideas, is summarized in Table 2.

5.1.1 Self-Adaptive Methods

To clarify the terminology within the PINN literature, *adaptive* methods generally refer to algorithms where the adjustment of loss weights or sampling distributions is governed by predefined heuristic rules, dynamic gradient analyses, or statistical metrics computed at specific training intervals. In contrast, *self-adaptive* methods internalize this process by treating the adjustment mechanisms—such as loss weights, penalty coefficients, or activation scaling factors—as fully trainable parameters. Rather than relying on external calculations, the neural network inherently learns to optimize these parameters simultaneously with the standard network weights (e.g., via gradient descent) to automatically find the optimal balance for the loss function components.

Applying this foundational concept, Xiang et al. [66] proposed an early approach leveraging Gaussian models to dynamically adjust weights, demonstrating its efficacy on problems such as the heat equation. Their self-adaptive loss balanced physics-informed neural networks

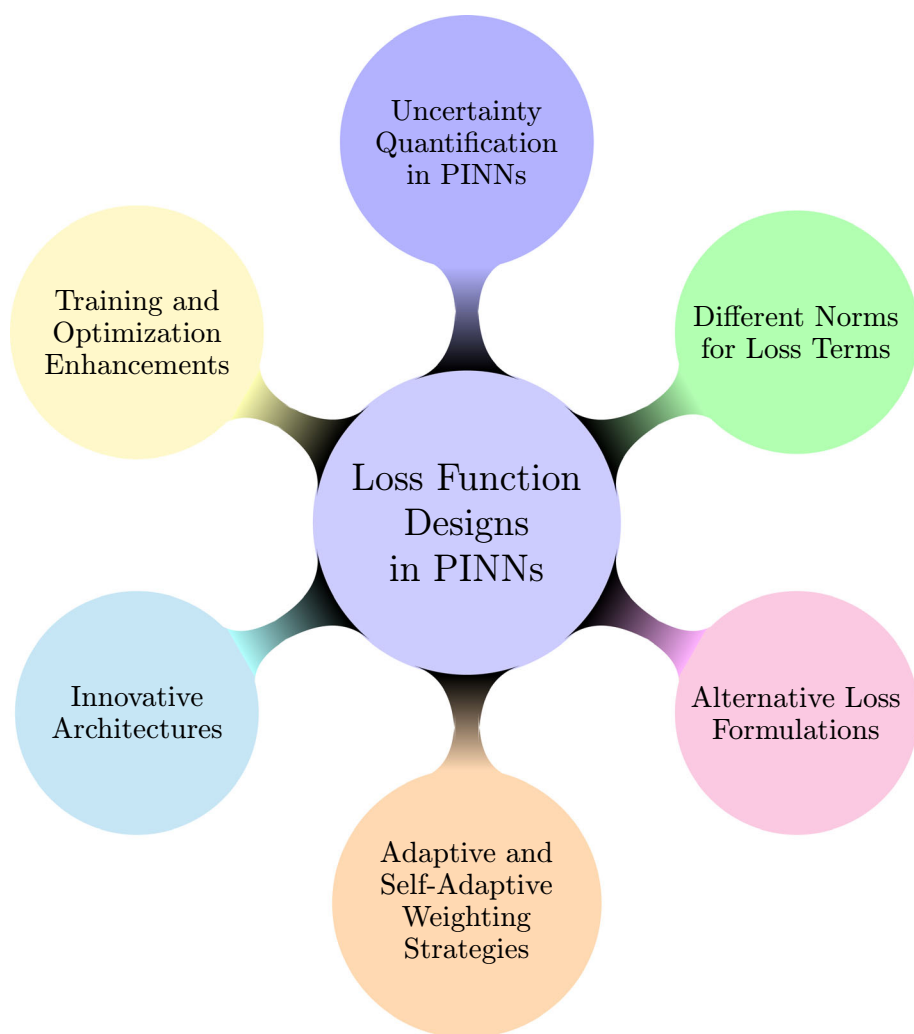


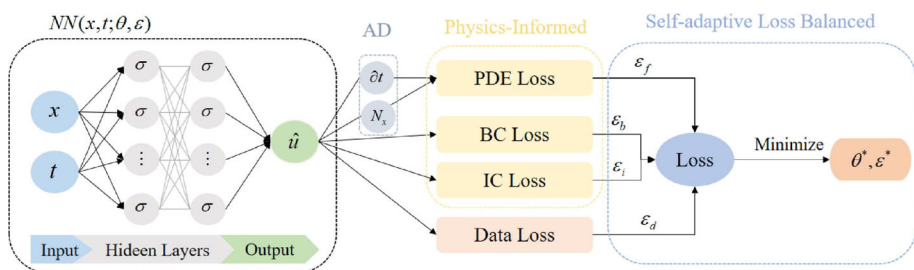
Fig. 15 Taxonomy chart of loss function designs in PINNs, showcasing six key categories

(lbPINNs) framework, illustrated in Fig. 16, employs a neural network that processes spatial and temporal coordinates to compute PDE residuals via automatic differentiation. The loss function comprises PDE, boundary condition, initial condition, and data losses, each assigned adaptive weights that are optimized during training to balance their contributions effectively. This method introduced a statistical basis for balancing data and PDE residual losses, reducing the dominance of larger-magnitude terms and improving convergence.

Building on this foundation, McClenny and Braga-Neto [49] developed self-adaptive PINNs, optimizing weights per collocation point via gradient descent. This refined the loss function by focusing computational effort on challenging regions, enhancing convergence for complex PDEs like the Allen-Cahn and Navier–Stokes equations. More recently, Zhou et al. [75] applied self-adaptive weighting to geotechnical problems, such as large strain consolidation models. By dynamically balancing data and residual losses, this approach

Table 2 Summary of adaptive and self-adaptive weighting strategies in PINNs

Strategy	References	Description
Self-adaptive methods	Xiang et al. [66]	Self-adaptive loss balanced PINNs using Gaussian models for adaptive weighting
	McClenny and Braga-Neto [49]	Introduced trainable weights per collocation point
	Zhou et al. [75]	Applied self-adaptive weighting to large-strain consolidation problems
Attention-based methods	Song et al. [59]	Loss-attentional PINNs using attention networks to dynamically weight squared errors
Gradient-based methods	entropy [40]	Dynamic weight strategy for Navier–Stokes: adjusts weights using gradient information
Optimization-based methods	Heldmann et al. [27]	PINN training formulated as a biobjective optimization problem for weight selection
	Zhu et al. [76]	BC-PINN: biased multiobjective coevolutionary algorithm for optimizing weights
	Cao et al. [6]	wbPINN: introduces correlation and penalty terms for balanced weighting
Spatial & temporal adaptation	Kharazmi et al. [36]	hp-VPINNs: domain decomposition for spatial adaptation
	Roy et al. [55]	Adaptive Interface-PINNs for transient diffusion with spatial–temporal weight adjustment
Multi-scale adaptation	Perez et al. [52]	Bayesian PINNs with adaptive weighting for multi-scale problems
	Liu et al. [43]	Variable-separated PINNs for blood flow: multi-scale hemodynamic weight adaptation

**Fig. 16** Schematic of self-adaptive loss balanced physics-informed neural networks (lbPINNs) adapted from Xiang et al. [66]. The neural network takes inputs x and t , producing output $\hat{u}$. PDE residuals are computed via automatic differentiation, contributing to the loss terms—PDE, boundary condition (BC), initial condition (IC), and data losses—each weighted by adaptive parameters ε_f , ε_b , ε_i , and ε_d , optimized during training to enhance solution accuracy

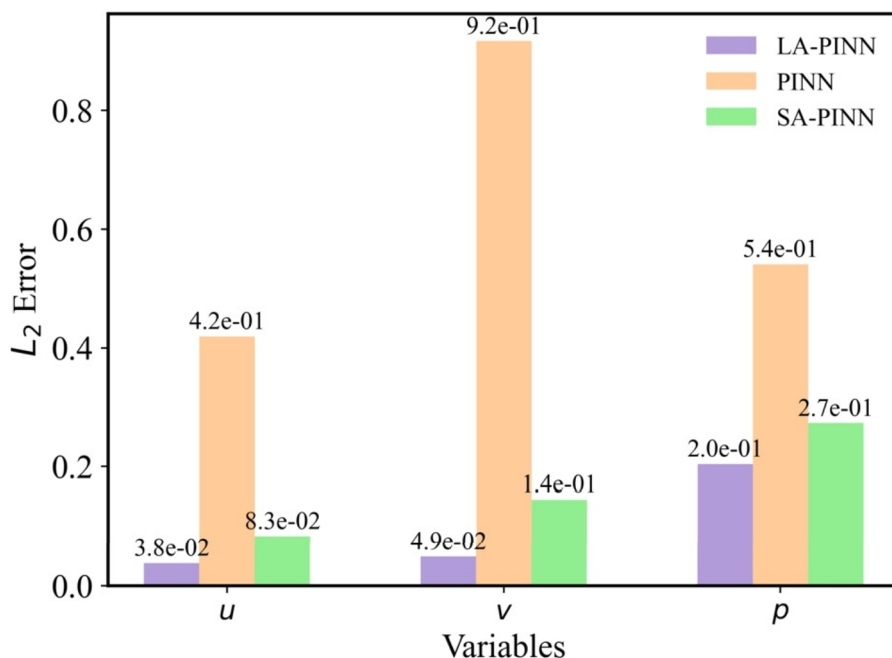


Fig. 17 Predictive performance of LA-PINN compared to vanilla PINN and SA-PINN for the Navier–Stokes equation. The figure shows the L_2 errors for the solution components u , v , and p after 100,000 training epochs, demonstrating the superior accuracy of LA-PINN [59]

tailored the loss function to handle nonlinear dynamics, ensuring stable optimization in highly variable systems.

5.1.2 Attention-Based Methods

Attention-based methods employ neural network architectures to assign weights to loss terms dynamically, adapting the loss function based on data-driven insights. Song et al. [59] introduced loss-attentional PINNs (LA-PINN), where attention networks weight squared errors according to their importance during training. Applied to the wave equation, this method adjusted the loss function to emphasize critical regions, such as areas with rapid changes, thereby boosting accuracy.

In their work, Song et al. [59] also applied LA-PINN to the Navier–Stokes equation, a classic benchmark in computational fluid dynamics. The LA-PINN model architecture includes multiple loss-attentional networks (LANs) that assign different weights to the squared errors of various loss components, such as boundary conditions and PDE residuals. This dynamic weighting allows the model to focus on hard-to-fit regions, improving convergence and accuracy.

As illustrated in Fig. 17, the LA-PINN model significantly outperforms both the vanilla PINN and the Self-adaptive PINN (SA-PINN) proposed by McClenny and Braga-Neto [49] in terms of predictive accuracy for the Navier–Stokes equation. After 100,000 training epochs, the L_2 errors for the solution components u , v , and p are notably lower for LA-PINN, highlighting its effectiveness in handling complex PDEs with challenging regions, such as those with rapid changes or stiffness.

Alternatively, gradient-based approaches, like the dynamic weight strategy from a 2022 *Entropy* study [40], adjust weights using gradient magnitudes. For the Navier–Stokes equations, this technique mitigated imbalances by ensuring equitable contributions from boundary condition and PDE residual losses, stabilizing the loss function’s optimization and improving overall performance.

5.1.3 Optimization-Based Methods

Optimization-based methods frame weight adjustment as a mathematical optimization problem, systematically refining the loss function’s structure. Heldmann et al. [27] proposed a biobjective optimization framework, treating data and PDE residual losses as competing objectives. By optimizing their weights using a dichotomic search method to approximate the Pareto front, this approach achieved a balanced trade-off, proving effective for predicting COVID-19 infection rates with an extended susceptible-vaccinated-infected-hospitalized-recovered (SVIHR) model. This method identified near-ideal knee solutions, often assigning weights greater than 0.99 to data loss, and demonstrated applicability to various systems of ordinary differential equations.

Zhu et al. [76] advanced this paradigm with BC-PINNs, employing a coevolutionary algorithm to dynamically adjust penalty coefficients. This method enhances convergence for nonlinear PDEs by exploring diverse weight configurations, prioritizing the fitting of boundary and initial conditions to ensure the PINN converges to the particular solution of the PDE. The framework of BC-PINN, illustrated in Fig. 18, involves a two-stage optimization mechanism: first, a biased multiobjective coevolutionary algorithm searches for optimal penalty coefficients by leveraging biased dominance ranking to favor data fidelity, and second, the PINN is trained with these coefficients via gradient descent. This iterative process balances convergence speed and accuracy, as depicted in the figure, which outlines the interplay between subpopulation evolution and network training.

Recently, Cao et al. [6] introduced weight-balanced PINNs (wbPINNs), incorporating correlation and penalty terms to prevent overfitting in weight adjustments. This approach divides the objective loss into three components and uses correlation regularization to reduce the generalization error, adaptively adjusting weights to optimize performance. Demonstrated on complex PDEs like the three-dimensional elliptic problem and the Navier–Stokes equation, wbPINNs achieve robust optimization and higher accuracy, outperforming traditional PINNs by orders of magnitude in convergence and accuracy.

Furthermore, a critical paradigm shifting away from conventional convex linear scalarization is the integration of *Chebyshev scalarization strategies* within multi-objective PINN optimization [28]. Traditional linear weighted combinations inherently fail when the Pareto front of the multi-component loss landscape (comprising $\mathcal{L}_{pde}$, $\mathcal{L}_{bc}$, and $\mathcal{L}_{ic}$) is non-convex. Chebyshev scalarization circumvents this by reformulating the objective to minimize the maximum weighted discrepancy from a predefined utopia or reference point $\mathbf{z}^* = [z_{pde}^*, z_{bc}^*]^T$:

$$\min_{\theta} \max_{i \in \{pde, bc\}} \{ \lambda_i (\mathcal{L}_i(\theta) - z_i^*) \} \quad (1)$$

By utilizing this max-affine objective, the optimizer is strictly driven to advance along directions that penalize the worst-performing loss component, establishing a mathematically sound mechanism for uniform Pareto-front traversal without heuristic tuning.

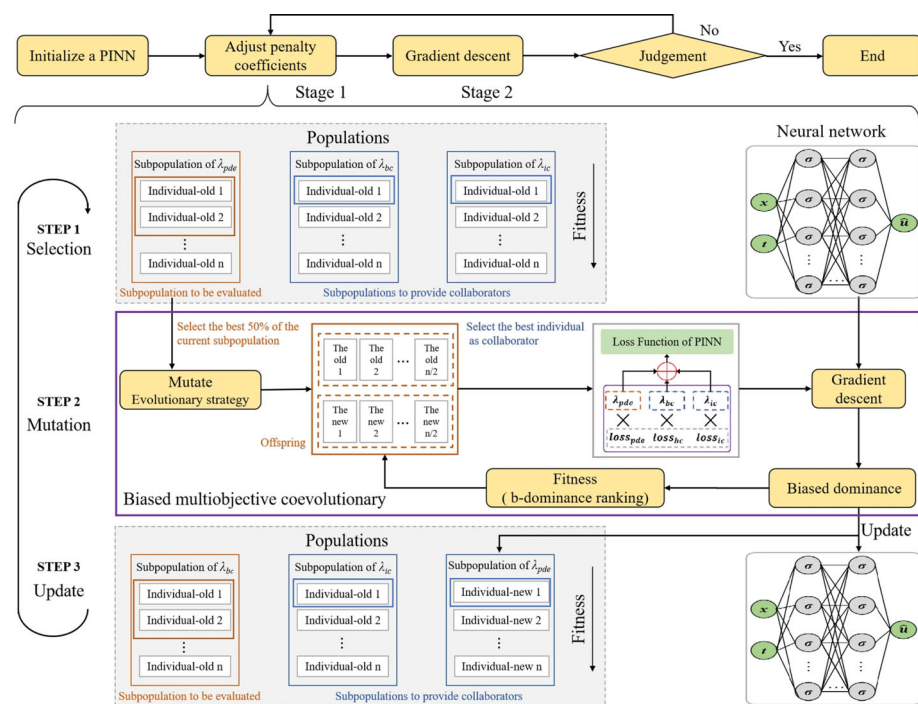


Fig. 18 The framework of BC-PINN, showcasing the two-stage optimization mechanism with the biased multiobjective coevolutionary algorithm for dynamic weight adjustment [76]

5.1.4 Spatial and Temporal Adaptation

Spatial and temporal adaptation techniques adjust weights based on domain properties or solution evolution, tailoring the loss function to specific problem characteristics. Kharazmi et al. [36] developed hp-Variational Physics-Informed Neural Networks (hp-VPINNs), using domain decomposition to spatially adapt weights across subdomains. For heterogeneous problems such as composite materials, this method improved the accuracy of the loss function by focusing on subdomain-specific challenges. In hp-VPINNs, the variational loss function is defined over nonoverlapping subdomains, employing high-order polynomial test functions and numerical integration techniques such as Gauss quadrature to accurately project the PDE residuals into a weak variational form. This formulation fundamentally reduces the smoothness requirements imposed on the neural network approximations and enables the model to effectively capture localized high-frequency dynamics and steep spatial variations. The effectiveness of the approach is shown in Fig. 19, which demonstrates the performance of hp-VPINNs in a two-dimensional Poisson equation with a steep exact solution. The figure compares the exact solution, the predictions of PINN and hp-VPINN, and their point errors, illustrating how the error converges as the number of subdomains increases through h-refinement, highlighting the ability of the method to capture steep spatial variations.

In the temporal domain, Roy et al. [55] proposed Adaptive Interface-PINNs (AdaI-PINNs) for transient diffusion problems. By adapting weights to prioritize regions of rapid change, such as heat diffusion across interfaces, this approach ensured that the loss function accurately captured time-dependent behaviors, enhancing the fidelity of the solution.

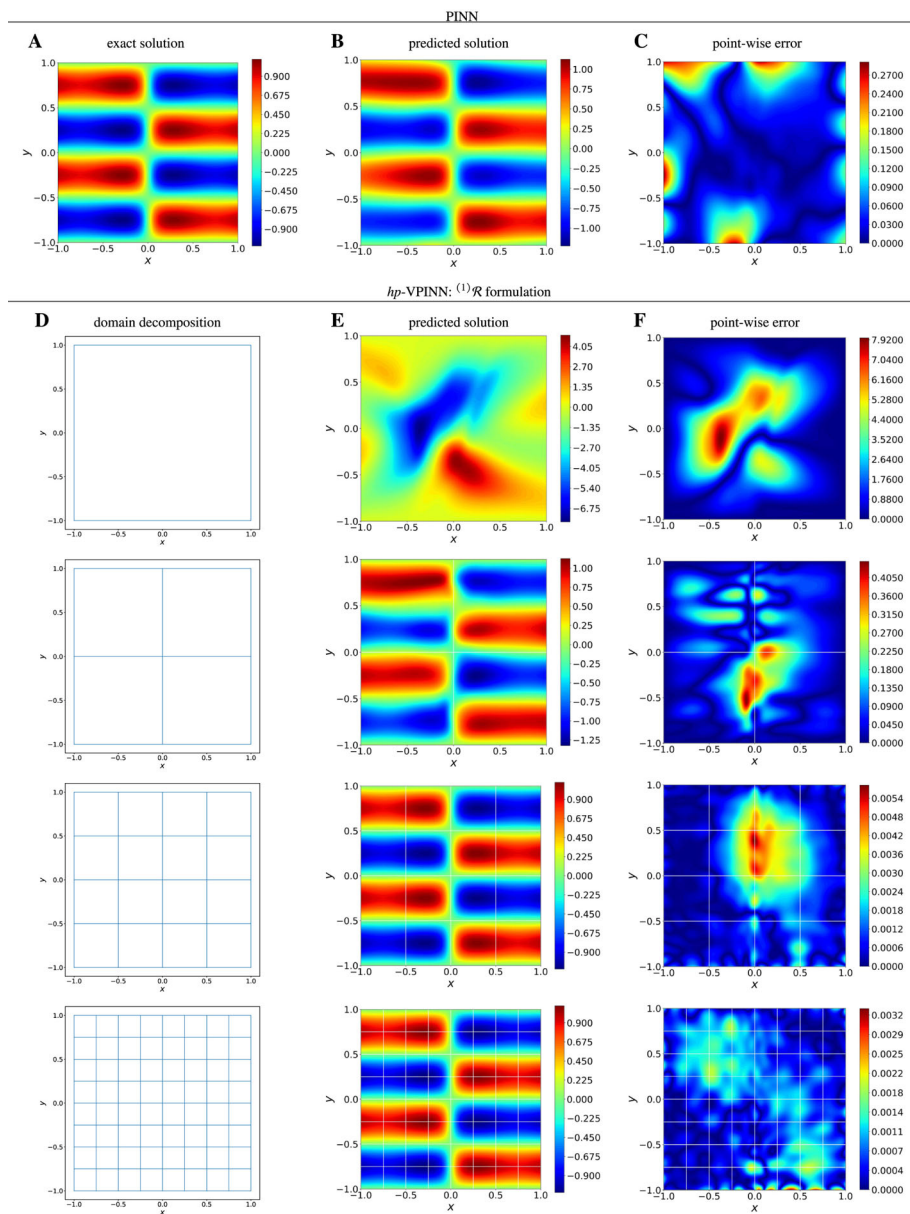


Fig. 19 Two-dimensional Poisson's equation with steep exact solution. Top panel: **A** Exact solution, **B** PINN prediction, and **C** PINN point-wise error. Bottom panel: **(D** column) h-refinement via different domain decomposition, **(E** column) hp-VPINN prediction, and **(F** column) hp-VPINN point-wise error [36]

5.1.5 Multi-Scale Adaptation

Multi-scale adaptation targets problems with disparate spatial or temporal scales, where loss term magnitudes can vary widely, complicating optimization. Perez et al. [52] introduced

adaptive weighting for Bayesian PINNs, using a Bayesian framework to balance weights in multi-scale fluid dynamics problems. This refined the loss function to prevent small-scale features from being overshadowed, improving overall accuracy. Similarly, Liu et al. [43] applied variable-separated PINNs to hemodynamic models, adapting weights to capture both macro- and micro-scale flow patterns. By adjusting the loss function to handle these scale differences, this method ensured precise modeling of complex multi-scale dynamics. Together, these strategies highlight how adaptive weighting improves the flexibility of the loss function, allowing PINNs to tackle a wide range of PDE challenges effectively.

5.1.6 Unifying Mathematical Principles and Recurring Failure Modes

When evaluating the adaptive and self-adaptive paradigms detailed in Sects. 5.1.1 through 5.1.5, a critical look beyond their diverse algorithmic implementations reveals that they are anchored to a few shared mathematical frameworks. Concurrently, they are highly susceptible to systemic optimization vulnerabilities. This subsection extracts these cross-cutting principles and recurring failure modes to provide a cohesive diagnostic blueprint.

Unifying Mathematical Principles

- **Gradient-Path Homogenization:** Vanilla PINN failure is frequently triggered by severe scale discrepancies between the gradients of the PDE residual ($\nabla_{\theta} \mathcal{L}_{pde}$) and boundary conditions ($\nabla_{\theta} \mathcal{L}_{bc}$). Whether utilizing explicit gradient norm tracking [40] or statistical self-adaptive weights [66], these strategies implicitly seek to homogenize the gradient trajectories, forcing the optimization landscape to be more isotropic by dynamically scaling weights λ_i such that $\|\nabla_{\theta}(\lambda_i \mathcal{L}_i)\|_2 \approx \|\nabla_{\theta}(\lambda_j \mathcal{L}_j)\|_2$.
- **Spatial-Temporal Error Pointing:** Point-wise adaptive methods [49] and attention-based networks [59] share an underlying conceptual identity: they reframe the local PDE residual as a dynamic spatial-temporal attention mask. Mathematically, by scaling weights based on local errors, the objective function is automatically regularized to penalize unresolved sharp gradients, boundary shocks, or high-frequency fields, acting as a continuous, automated grid-refinement surrogate.
- **Pareto Dominance and Multi-Objective Optimality:** In standard PINN training, the concept of Pareto dominance between competing loss contributions ($\mathcal{L}_{pde}$, $\mathcal{L}_{bc}$, and data fidelity) is fundamentally *neglected*. Vanilla linear scalarization ($\sum \lambda_i \mathcal{L}_i$) blindly aggregates conflicting gradients. The implication of this neglect is severe: when loss components conflict, a mathematically inferior update that drastically degrades the physical residual to marginally improve boundary fitting is often accepted. This traps the optimizer in sub-optimal local minima, entirely missing the true multi-objective optimum. In contrast, advanced optimization-based models [27, 76] explicitly *account for* Pareto dominance. Rather than treating weights as fixed parameters, these methods treat loss components as competing agents and employ non-dominated sorting or coevolutionary mechanisms. The implication of this paradigm shift is profound: it allows the network to systematically traverse and map the true Pareto front. Consequently, PINNs can locate "knee" solutions—representing the absolute optimal trade-off between physical fidelity and observational data—even when the underlying multi-objective landscape is highly non-convex.

Recurring Failure Modes Despite their theoretical advantages, these sophisticated designs frequently succumb to characteristic training traps, explaining the performance trade-offs visualized later in the comprehensive comparison:

- **The Computational Over-Indexing Bottleneck:** Methods requiring gradient-of-gradient evaluations (e.g., tracking the norms of gradients) or executing multi-objective coevolutionary loops [76] introduce severe wall-clock time penalties. The computational complexity often scales aggressively with the network parameters, meaning that while they require fewer training epochs to converge, their total clock runtime can vastly exceed a heavily-sampled vanilla PINN.
- **Trivial Flat Minima and Boundary Dominance:** In stiff or highly non-linear PDE regimes, adaptive algorithms frequently over-compensate for boundary condition errors. This creates an aggressive penalty on $\mathcal{L}_{bc}$, causing the loss landscape to warp into a steep, artificial valley. The optimizer gets trapped in a trivial, flat, unphysical minimum that satisfies the boundaries perfectly but yields zero-valued or completely unphysical interior solutions.
- **Weight-Optimizer Non-Convergence (Oscillation):** When loss weights λ and network weights θ are updated concurrently via joint gradient-based steps, the optimization mimics a min-max game. Without strict regularization (such as the correlation penalties proposed in wbPINNs [6]), these systems frequently fail to reach a stable Nash equilibrium. Instead, the weights enter an uncontrolled oscillatory cycle, causing sudden training divergence in the late stages of optimization.

To establish a systematic basis for comparing disparate literature claims regarding convergence, accuracy, and robustness, this review implements a normalized inferential framework. Because the empirical results in the surveyed studies are reported under highly variable conditions—including disparate hardware architectures, PDE benchmark scales, and hyperparameter tunings—a direct, absolute empirical comparison is infeasible. Therefore, the evaluations presented in this section, and visually synthesized in the accompanying radar chart, represent an *inferential synthesis*. This synthesis is based on the inherent theoretical capabilities, algorithmic structures, and reported relative performance gains of each advanced loss formulation compared to the baseline vanilla PINN (defined by static, uniform loss weights $\lambda_i = 1$) under identical PDE settings within each respective study.

The qualitative scores (ranging from 1 to 5) are systematically governed by the following relative criteria:

- **Convergence Speed:** Assessed by the method’s theoretical capability and reported efficiency in reducing the number of optimization epochs or wall-clock time required to reach a prescribed loss threshold (e.g., $\mathcal{O}(10^{-4})$). High scores indicate rapid error decay in early training phases, often facilitated by dynamic attention to stiff regions.
- **Accuracy:** Evaluated by the relative $\mathbb{L}_2$ error reduction compared to vanilla PINNs. A maximum score denotes a consistent, documented improvement of multiple orders of magnitude across complex or chaotic PDE regimes (e.g., high-Reynolds Navier–Stokes).
- **Computational Efficiency:** Reflects the per-iteration computational overhead introduced by the loss modification. Higher scores denote lower algorithmic friction, whereas lower scores reflect heavy penalties from calculating higher-order derivatives or executing secondary evolutionary optimization loops.
- **Generalization:** Assesses the model’s structural capacity to maintain predictive stability when evaluated outside the immediate training domain, or its robustness in inverse problem formulations involving unseen parameter spaces.
- **Robustness to Scale Imbalance:** Evaluated by the mathematical formulation’s capacity to maintain optimization stability in the presence of severe magnitude disparities ($\geq \mathcal{O}(10^3)$) between competing PDE residual, initial condition, and boundary condition loss terms.

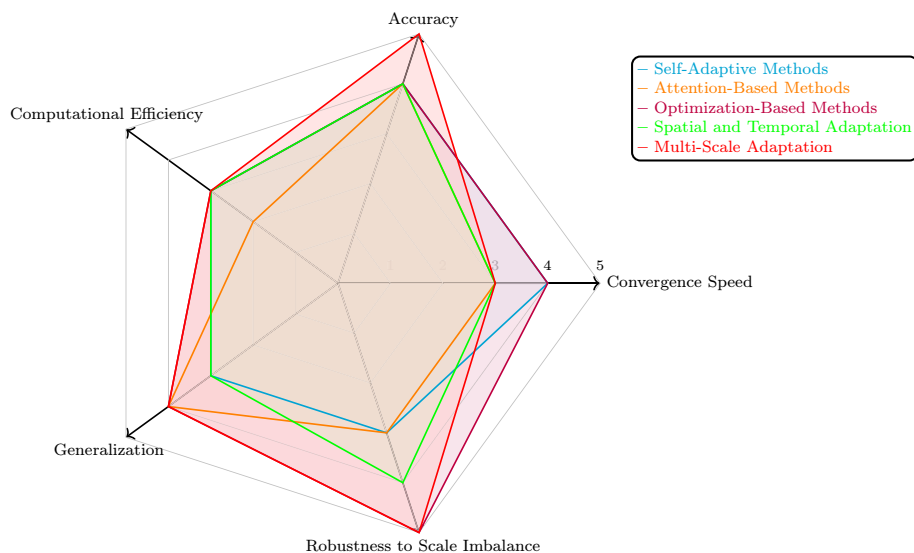


Fig. 20 Radar chart systematically comparing adaptive and self-adaptive weighting strategies in PINNs across five core performance metrics. Because absolute empirical cross-testing is infeasible across disparate literature, scores represent an inferential synthesis normalized based on relative performance gains over the baseline vanilla PINN formulation and the inherent algorithmic potential of each strategy

Figure 20 multi-axially synthesizes how these five core metrics manifest across the major adaptive weighting paradigms compiled in our study.

5.2 Alternative Loss Formulations

Alternative loss formulations in PINNs are modifications to the standard loss function designed to address specific challenges posed by certain PDEs or to enhance the training process. These formulations are critical because the standard loss function, typically a combination of data fidelity and PDE residual terms using the L2 norm, may not be optimal for all PDEs. For example, PDEs with high-order derivatives, steep gradients, or discontinuities can lead to numerical instability or poor convergence when using the standard approach. By tailoring the loss function to the specific characteristics of the PDE, alternative loss formulations improve the PINN's ability to solve a broader range of problems accurately and efficiently, reducing computational complexity, enhancing solution accuracy, and ensuring better adherence to physical principles. An summary of alternative loss formulations in Physics-Informed Neural Networks is presented in Table 3.

5.2.1 Weak Form (VPINNs)

Variational Physics-Informed Neural Networks (VPINNs) introduce an alternative loss formulation by employing the weak (variational) form of PDEs rather than the strong form used in standard PINNs. This approach, inspired by variational methods such as those in finite element analysis, reduces the computational burden associated with high-order derivatives and enhances training stability and accuracy, particularly for complex PDEs.

Table 3 Summary of alternative loss formulations in PINNs

Formulation	References	Description
Weak Form (VPINNs)	Kharazmi et al. [35]	Variational PINNs: introduced weak formulations for PDEs
	Chen and Lü [11]	PINN-wf: used weak form residuals for complex PDEs
Energy-Based Formulations	Chen et al. [9]	Energy-based PINNs for solid mechanics: computed loss via strain energy and virtual work
Gradient-Enhanced Formulations	Yu et al. [70]	Incorporated gradient terms into the loss
First-Order Formulations	Gladstone et al. [23]	FO-PINN: reduced derivative order for improved training stability
Godunov-Type Formulations	Cassia and Kerswell [7]	Godunov-type loss functions: used flux formulations for hyperbolic PDEs
Custom Loss Functionals	Frerichs-Mihov et al. [21]	Designed custom functionals for convection-dominated problems with steep gradients
	Bai et al. [3]	Proposed LSWR loss for 2D/3D solid mechanics
Symmetry-Enforced Formulations	Jiao et al. [33]	GsPINN: integrated Lie symmetry groups to introduce a first-order Linearized Symmetric Condition (LSC) constraint.

In standard PINNs, the strong form of a PDE is directly enforced by penalizing residuals at discrete points. Consider a general PDE:

$$\mathcal{L}u(x) = f(x), \quad x \in \Omega,$$

with boundary conditions:

$$u(x) = h(x), \quad x \in \partial\Omega,$$

where $\mathcal{L}$ is a differential operator, $u(x)$ is the solution, $f(x)$ is a known source term, and Ω is the domain with boundary $\partial\Omega$. The neural network approximation $u_{\text{NN}}(x; \theta)$ is used, and the strong-form loss function is typically:

$$\mathcal{L}^s = \mathcal{L}_r^s + \mathcal{L}_u,$$

where

$$\mathcal{L}_r^s = \frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{L}u_{\text{NN}}(x_{r_i}; \theta) - f(x_{r_i})|^2,$$

$$\mathcal{L}_u = \tau \frac{1}{N_u} \sum_{i=1}^{N_u} |u_{\text{NN}}(x_{u_i}; \theta) - h(x_{u_i})|^2,$$

with $\{x_{r_i}\}$ as interior collocation points, $\{x_{u_i}\}$ as boundary points, and τ as a penalty parameter. This formulation requires computing high-order derivatives of u_{NN} , which can be computationally expensive and numerically unstable for PDEs with complex operators.

In contrast, VPINNs, introduced by Kharazmi et al. [35], adopt the weak form by multiplying the PDE by a test function $v(x)$ and integrating over the domain:

$$\int_{\Omega} \mathcal{L}u(x)v(x) dx = \int_{\Omega} f(x)v(x) dx.$$

Using integration by parts, derivatives can be transferred from $u(x)$ to $v(x)$, lowering the derivative order required of u_{NN} . The variational residual is:

$$R(v) = \int_{\Omega} \mathcal{L}u_{NN}(x; \theta)v(x) dx - \int_{\Omega} f(x)v(x) dx.$$

A finite set of test functions $\{v_k(x)\}_{k=1}^K$ is chosen (e.g., Legendre polynomials for their orthogonality), and the loss function becomes:

$$\mathcal{L}^v = \mathcal{L}_R^v + \mathcal{L}_u,$$

where

$$\mathcal{L}_R^v = \frac{1}{K} \sum_{k=1}^K |R_k - F_k|^2,$$

$$R_k = \int_{\Omega} \mathcal{L}u_{NN}(x; \theta)v_k(x) dx, \quad F_k = \int_{\Omega} f(x)v_k(x) dx,$$

and $\mathcal{L}_u$ retains the boundary term as above. In practice, these integrals are computed using numerical quadrature (e.g., Gaussian quadrature), though analytical forms are possible for shallow networks.

The weak form offers distinct advantages over the strong form. By reducing derivative order, it lowers computational cost and mitigates instability, as demonstrated by Kharazmi et al. [35] in plate bending problems, where VPINNs outperformed standard PINNs in accuracy and stability. Similarly, Chen and Lü [11] applied VPINNs to the Hirota equation, a nonlinear PDE in optical communication and financial modeling, achieving accurate soliton solutions with enhanced efficiency.

Remark The strong form is a special case of the weak form when test functions are Dirac-delta functions, $v_k(x) = \delta(x - x_k)$, reducing $R_k - F_k$ to the pointwise residual $\mathcal{L}u_{NN}(x_k) - f(x_k)$.

The choice of test functions, such as polynomials or trigonometric functions, influences VPINN performance, offering flexibility to tailor the loss to specific PDEs. This adaptability, combined with reduced computational demands, positions VPINNs as a robust enhancement to PINN methodologies.

5.2.2 Energy-Based Formulations

Energy-based formulations leverage physical principles to construct loss functions, particularly in domains where energy conservation is a natural constraint. Chen et al. [9] proposed an energy-based PINN for solid mechanics, computing the loss using strain energy and virtual work principles instead of directly enforcing the PDE residuals. In their study of hyperelastic materials under large deformations, the loss was defined as the difference between the internal strain energy and the external work, ensuring that the neural network solution adheres to the principle of virtual work. This method proved effective in resolving stress singularities near cracks, as demonstrated on benchmark problems like a cantilever beam, where it outperformed standard PINNs by better capturing the physical behavior of the system.

5.2.3 Gradient-Enhanced Formulations

Gradient-enhanced formulations in PINNs aim to improve solution accuracy and smoothness by incorporating derivative information into the loss function. Introduced by Yu et al. [70], gradient-enhanced PINNs (gPINNs) address the limitations of standard PINNs, which often struggle with accuracy in problems featuring steep gradients, high-frequency components, or convection-dominated regimes, such as those encountered in the advection–diffusion equation.

In standard PINNs, the loss function enforces the PDE residual and boundary conditions for a general PDE of the form:

$$f\left(x; \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_d}; \frac{\partial^2 u}{\partial x_1 \partial x_1}, \dots, \frac{\partial^2 u}{\partial x_1 \partial x_d}; \dots; \lambda\right) = 0, \quad x = (x_1, \dots, x_d) \in \Omega,$$

with boundary conditions:

$$\mathcal{B}(u, x) = 0 \quad \text{on} \quad \partial\Omega.$$

Here, f represents the PDE residual, which depends on the spatial coordinates x , the solution u , its derivatives, and parameters λ . The neural network approximation $u_{\text{NN}}(x; \theta)$ minimizes:

$$\mathcal{L}_{\text{PINN}} = w_f \mathcal{L}_f + w_b \mathcal{L}_b,$$

where

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| f\left(x_{f_i}; \frac{\partial u_{\text{NN}}}{\partial x_1}, \dots, \frac{\partial u_{\text{NN}}}{\partial x_d}; \dots; \theta, \lambda\right) \right|^2,$$

$$\mathcal{L}_b = \frac{1}{N_b} \sum_{i=1}^{N_b} |\mathcal{B}(u_{\text{NN}}, x_{b_i})|^2,$$

with $\{x_{f_i}\}$ as interior collocation points, $\{x_{b_i}\}$ as boundary points, and w_f, w_b as weights. This approach, while effective for forward and inverse PDE problems, primarily constrains the residual f and may not adequately enforce the behavior of the solution's derivatives, leading to inaccuracies in gradient-sensitive regions like boundary layers.

To overcome this, gPINNs augment the loss function with terms that penalize the gradients of the PDE residual. Assuming sufficient smoothness, since $f(x) = 0$ for all $x \in \Omega$, the gradient of the residual must also vanish:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_d} \right) = 0, \quad x \in \Omega.$$

Thus, the gPINN loss is defined as:

$$\mathcal{L}_{\text{gPINN}} = w_f \mathcal{L}_f + w_b \mathcal{L}_b + \sum_{i=1}^d w_{g_i} \mathcal{L}_{g_i},$$

where

$$\mathcal{L}_{g_i} = \frac{1}{N_{g_i}} \sum_{j=1}^{N_{g_i}} \left| \frac{\partial f}{\partial x_i}(x_{g_j}; \theta, \lambda) \right|^2,$$

and $\{x_{g_j}\}$ are typically the same as $\{x_{f_i}\}$. The weights w_{g_i} are hyperparameters, often set equal for simplicity. The term $\frac{\partial f}{\partial x_i}$ is computed using automatic differentiation, accounting

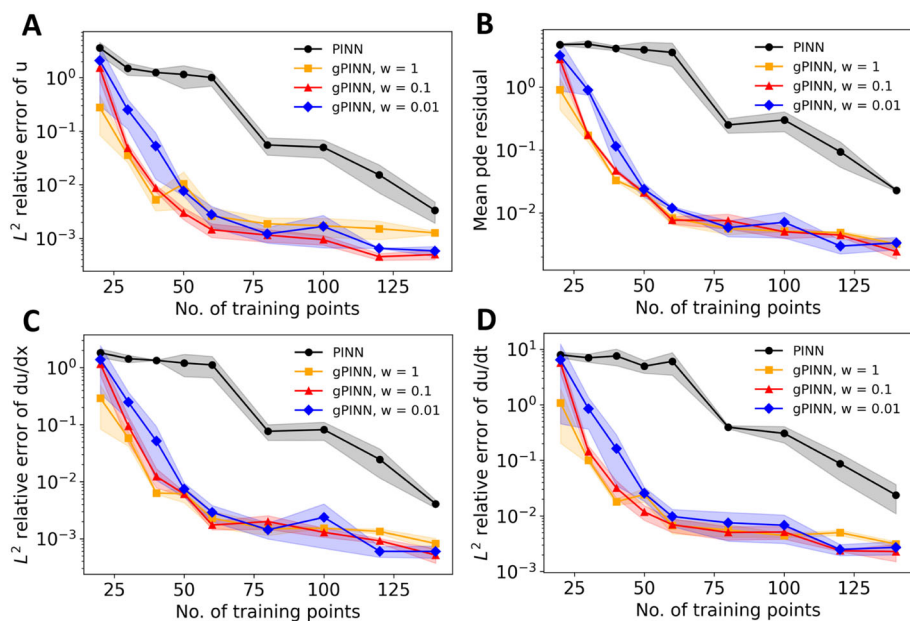


Fig. 21 Comparison between PINN and gPINN for the diffusion–reaction equation. **A** L^2 relative error of u for PINN and gPINN with $w = 1, 0.1$, and 0.01 . **B** Mean absolute value of the PDE residual. **C** L^2 relative error of $\frac{\partial u}{\partial x}$. **D** L^2 relative error of $\frac{\partial u}{\partial t}$. Adapted from Yu et al. [70]

for the dependence of f on x and the derivatives of u_{NN} . This formulation enhances the network’s ability to capture derivative behavior, yielding smoother and more accurate solutions, particularly in regions with sharp transitions.

For example, consider the diffusion–reaction equation:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + R(x, t), \quad x \in [-\pi, \pi], \quad t \in [0, 1],$$

where $D = 1$ and $R(x, t) = e^{-t} \left[32 \sin(2x) + \frac{8}{3} \sin(3x) + \frac{15}{4} \sin(4x) + \frac{63}{8} \sin(8x) \right]$, with boundary conditions $u(-\pi, t) = u(\pi, t) = 0$ and initial condition $u(x, 0) = \sum_{i=1}^4 \frac{\sin(ix)}{i} + \frac{\sin(8x)}{8}$. In this context, the PDE residual f is:

$$f = \frac{\partial u}{\partial t} - D \frac{\partial^2 u}{\partial x^2} - R(x, t).$$

The gPINN loss includes gradient terms:

$$\mathcal{L}_{\text{gPINN}} = \mathcal{L}_f + w \mathcal{L}_{g_x} + w \mathcal{L}_{g_t},$$

where $\mathcal{L}_{g_x}$ and $\mathcal{L}_{g_t}$ penalize $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial t}$, respectively. As shown in Fig. 21, gPINNs with weights $w = 1, 0.1, 0.01$ outperformed standard PINNs, achieving a 1% L^2 relative error in u with only 40 training points, compared to over 100 points required by PINNs. The figure illustrates lower errors in u , $\frac{\partial u}{\partial x}$, $\frac{\partial u}{\partial t}$, and the PDE residual.

The improved performance stems from reduced residual fluctuations, enhancing generalization in sparse data regions. Additionally, Yu et al. [70] combined gPINNs with

residual-based adaptive refinement (RAR), which adaptively adds training points in high-residual areas, further boosting accuracy for PDEs with steep gradients, such as the advection–diffusion equation. This hybrid approach ensures numerical stability and efficiency, making gPINNs a robust enhancement to PINNs for both forward and inverse problems.

In summary, gradient-enhanced formulations offer significant improvements over standard PINNs by enforcing both the PDE residual and its gradients, leading to more accurate, smoother solutions with fewer training points across a variety of PDE applications.

5.2.4 First-Order Formulations

First-order formulations in PINNs enhance training stability and computational efficiency for PDEs with high-order derivatives by reformulating them into systems of first-order equations. This approach reduces the computational burden of evaluating high-order derivatives via automatic differentiation, which can be numerically unstable and costly. Gladstone et al. [23] introduced First-Order PINNs (FO-PINNs), where the loss function is defined based on first-order residuals derived from the reformulated PDE system.

For example, consider the two-dimensional Helmholtz equation:

$$k^2 u + \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f, \quad (x, y) \in \Omega,$$

with Dirichlet boundary conditions $u = 0$ on $\partial\Omega$, where $k = 1$ is the wave number, $f = 0$ is the source term, and Ω is a square domain of side length 2. In standard PINNs, the neural network outputs the solution $\hat{u}$, and the loss function penalizes the second-order derivatives directly. In contrast, FO-PINNs introduce auxiliary variables $\hat{u}_x$ and $\hat{u}_y$ representing $\frac{\partial u}{\partial x}$ and $\frac{\partial u}{\partial y}$, reformulating the PDE as:

$$k^2 \hat{u} + \frac{\partial \hat{u}_x}{\partial x} + \frac{\partial \hat{u}_y}{\partial y} = f,$$

with compatibility constraints $\hat{u}_x = \frac{\partial \hat{u}}{\partial x}$ and $\hat{u}_y = \frac{\partial \hat{u}}{\partial y}$. The loss function incorporates these first-order residuals and compatibility terms, requiring only first-order derivatives. This reformulation, combined with exact boundary condition imposition using Automatic Differentiation Functions (ADFs), improved accuracy over standard PINNs, as demonstrated by Gladstone et al. [23].

Similarly, FO-PINNs have been effectively applied to higher-order PDEs, such as the fourth-order Kirchhoff plate bending problem:

$$\nabla^4 u(x, y) = 1, \quad (x, y) \in \Omega = \{(x, y) : x^2 + y^2 < 1\},$$

with clamped boundary conditions $u = 0$ and $\frac{\partial u}{\partial n} = 0$ on $\partial\Omega$. Here, FO-PINNs predict additional outputs for the first-, second-, and third-order derivatives ($\hat{u}_x, \hat{u}_y, \hat{u}_{xx}, \hat{u}_{yy}, \hat{u}_{xxx}, \hat{u}_{yyy}, \hat{u}_{xxy}$), reformulating the PDE to use only first-order derivatives in the loss function, supplemented by compatibility constraints. This approach achieved a 55% reduction in relative L^2 error and a 1.95x speedup compared to standard PINNs, as evidenced by the reduced prediction error for the plate deflection, shown in Fig. 22.

FO-PINNs have also shown promise in complex systems, such as the Navier–Stokes equations in an annular ring domain, where first-order reformulations of the continuity and momentum equations improved computational efficiency. These advancements highlight the

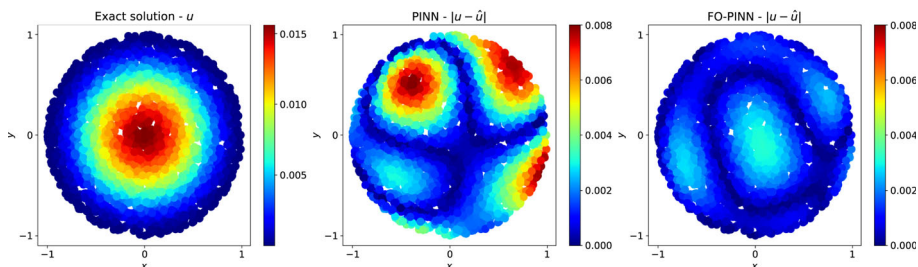


Fig. 22 Plots showing the exact solution along with the absolute error in the prediction of u obtained using standard PINN and FO-PINN for the clamped Kirchhoff plate. Adapted from Gladstone et al. [23]

robustness of FO-PINNs for diverse PDE applications, offering superior accuracy and faster convergence compared to traditional PINNs.

5.2.5 Symmetry-Enforced Formulations

An emerging and highly promising paradigm in alternative loss function design involves embedding geometric and algebraic invariants directly into the optimization objective to bypass computational bottlenecks. Recently, Jiao et al. [33] introduced GsPINN, a deep symmetric fundamental solver that integrates Sophus Lie's Lie symmetry groups into the PINN framework for solving Cauchy problems. Instead of relying solely on high-order PDE residuals that strain automatic differentiation mechanisms, this approach applies the classic Invariance Principle to derive a Linearized Symmetric Condition (LSC) as an explicit, additional first-order constraint.

The resulting total loss function is systematically formulated as:

$$\mathcal{L}(\theta) = \mathcal{L}_0(\theta) + \mathcal{L}_r(\theta) + \mathcal{L}_s(\theta)$$

where $\mathcal{L}_0(\theta)$ and $\mathcal{L}_r(\theta)$ represent the standard initial and interior PDE residual losses, respectively, and $\mathcal{L}_s(\theta)$ is the newly introduced symmetry-induced residual term evaluated over N_s symmetry collocation points:

$$\mathcal{L}_s(\theta) = \frac{1}{N_s} \sum_{i=1}^{N_s} |Q(t_i^s, x_i^s, y_i^s, K_\theta)|^2$$

Here, the operator Q encapsulates the first-order LSC residual:

$$Q(t, x, y, K) = \eta(t, x, y, K) - \xi(t, x, y, K)K_x - \tau(t, x, y, K)K_t = 0$$

where τ , ξ , and η are the infinitesimal generators representing the underlying tangent vector fields of the PDE's symmetry group. By structurally shifting a significant portion of the optimization burden from high-order automatic differentiation to these first-order symmetry constraints, this formulation achieves a deterministic 20% to 30% reduction in training time while fully preserving solution accuracy. This advancement highlights the immense potential of group-theoretic loss constraints in accelerating data-driven discovery and simulation of complex, high-dimensional physical systems.

5.2.6 Godunov-Type Formulations

Godunov-type formulations adapt numerical methods from computational fluid dynamics to design loss functions for hyperbolic PDEs, which often exhibit discontinuities like shocks. Cassia and Kerswell [7] introduced Godunov-type loss functions for PINNs, incorporating Godunov-type flux approximations into the PDE residual loss to handle discontinuities more effectively. In their study of the inviscid Burgers' equation, a hyperbolic PDE with shock formation, they used the Godunov flux to compute residuals at collocation points, ensuring that the loss function accounts for the upwind nature of the solution. This approach led to sharper resolution of shocks and better overall accuracy compared to standard PINN formulations, highlighting its potential for wave propagation and conservation law problems.

5.2.7 Custom Loss Functionals

Custom loss functionals in PINNs are tailored to address specific challenges in PDEs, such as steep gradients or domain-specific physical constraints, by designing loss functions that enhance stability and accuracy. These formulations adapt the loss to the problem's characteristics, overcoming limitations of standard PINN losses, particularly in multiscale or complex domains.

Frerichs-Mihov et al. [21] proposed novel loss functionals for steady-state convection-dominated convection-diffusion problems, characterized by high Péclet numbers and solutions with steep layers. Consider the PDE:

$$-\varepsilon \Delta u + \mathbf{b} \cdot \nabla u + cu = f \quad \text{in } \Omega \subset \mathbb{R}^d, \quad u = g \quad \text{on } \partial\Omega,$$

where $\varepsilon > 0$ is the diffusion coefficient, $\mathbf{b}$ is the convection field, c is the reaction coefficient, f is the source term, and g prescribes Dirichlet boundary conditions. For high Péclet numbers, standard PINN losses struggle with numerical oscillations near layers. Frerichs-Mihov et al. introduced loss functionals inspired by cost functionals from numerical methods, achieving up to 17.3% and 5.5% reductions in $L^2(\Omega)$ error for benchmark problems with interior and boundary layers, respectively, compared to vanilla and hp-variational PINN losses. Their approach enhances stability by carefully balancing residual terms, improving the approximation of solutions with steep gradients.

Similarly, Bai et al. [3] developed the Least Squares Weighted Residual (LSWR) loss functional for 2D and 3D solid mechanics problems, such as elasticity in domains like a 2D plate with a hole. The LSWR loss is formulated as:

$$\mathcal{L} = \chi_1 \int_{\Omega} R_g^2 d\Omega + \chi_2 \int_{\partial\Omega_t} R_t^2 d\partial\Omega_t,$$

where R_g is the residual of the equilibrium equations, R_t is the traction boundary condition residual, and $\chi_1 = \frac{h^2}{E}$, $\chi_2 = \frac{1}{E^2}$ are scaling factors (with E as Young's modulus and h as a characteristic length related to sample point spacing). For arbitrary point distributions, numerical integration is performed using Delaunay triangulation, assigning areas (or volumes in 3D) to collocation points. This dimensionless formulation mitigates biased training due to differing units and enhances generalization compared to collocation-based losses, achieving lower errors in displacement and stress predictions.

These custom loss functionals expand the applicability of PINNs by addressing specific PDE challenges. Frerichs-Mihov et al.'s approach excels in convection-dominated regimes, while Bai et al.'s LSWR loss ensures robust predictions in solid mechanics, collectively improving training efficiency and solution accuracy across diverse applications.

Table 4 Summary of different norms for loss terms in PINNs

Norm	References	Description
L1 norm	Wang et al. [40]	Used L1 norm for robustness in stiff-gradient problems such as Burgers' equation
Huber norm		Applied Huber loss to enhance robustness and smoothness
Sobolev norm	Yang et al. [67]	Incorporated Sobolev norms to enforce smoothness via derivative penalties
Custom norms	Yu et al. [70]	Designed custom losses with gradient penalties for convection-dominated problems

5.3 Different Norms for Loss Terms

In PINNs, the loss function is a critical component that guides the training process by quantifying the discrepancy between the neural network's predictions and the physical constraints derived from PDEs, boundary conditions, and observational data. The choice of norm used to measure these residuals defines how errors are penalized and aggregated, directly influencing the network's ability to converge to accurate solutions. The standard L2 norm, which computes the mean squared error, is widely adopted due to its differentiability and emphasis on penalizing large errors. However, its sensitivity to outliers, struggles with steep gradients, and limitations in capturing multi-scale dynamics have prompted the exploration of alternative norms. These alternatives, such as the L1, Huber, Sobolev, and custom norms, tailor the loss function to address specific challenges like robustness, smoothness, or problem-specific error handling. This subsection examines these norms, their mathematical formulations, and their impact on PINN performance, with key applications and references summarized in Table 4.

The general loss function in PINNs can be expressed as:

$$L(\theta) = \sum_{\alpha \in C} m_{\alpha} \int_{\Omega_{\alpha}} |r_{\alpha}(x, \theta)|^p dx,$$

where θ represents the neural network parameters, $C = \{D, B, E\}$ denotes the set of loss components (data fidelity D , boundary conditions B , and PDE residuals E), m_{α} are scalar weights, Ω_{α} is the domain for component α , $r_{\alpha}(x, \theta)$ is the residual, and $p \geq 1$ specifies the L^p norm. For computational efficiency, this is discretized over sampled points $X_{\alpha} \subset \Omega_{\alpha}$:

$$L(\theta, X_{\alpha}) = \sum_{\alpha \in C} m_{\alpha} \frac{1}{N_{\alpha}} \sum_{i=1}^{N_{\alpha}} \lambda_{\alpha,i} |r_{\alpha}(x_i, \theta)|^p,$$

where N_{α} is the number of points in X_{α} , and $\lambda_{\alpha,i}$ are local weights. The choice of p —whether 1 (L1 norm), 2 (L2 norm), or higher (e.g., L^{∞})—fundamentally alters the training dynamics, as explored below.

5.3.1 L1 Norm

The L1 norm ($p = 1$), or mean absolute error, penalizes residuals linearly, making it less sensitive to large errors than the L2 norm. This robustness is advantageous for PDEs with noisy data or discontinuities. For instance, in problems like the Burgers' equation, which

features stiff gradients and sharp transitions, the L1 norm stabilizes training by reducing the influence of large residuals near discontinuities [62]. Furthermore, Wang et al. [62] challenged the suitability of the L2 norm for high-dimensional nonlinear PDEs, such as the Hamilton-Jacobi-Bellman (HJB) equation in optimal control. They introduced the concept of stability, proving that for HJB equations, the learned solution approximates the true solution as the loss approaches zero only if the L^p norm has a sufficiently large p (e.g., L^∞). The L2 norm ($p = 2$) fails this stability criterion, while the L1 norm, though robust in some contexts, may not suffice for such equations. Thus, the L1 norm is effective for specific problems like Burgers' equation but requires careful consideration of the PDE's stability properties.

5.3.2 Huber Norm

The Huber norm blends L1 and L2 characteristics, applying a quadratic penalty ($|r|^2$) to small residuals ($|r| \leq \delta$) for smoothness and a linear penalty ($\delta|r| - \frac{1}{2}\delta^2$) to large residuals ($|r| > \delta$) for robustness. This adaptability suits PDEs with varying residual scales. Chen et al. [12] applied the Huber norm to PINNs solving the nonlinear Schrödinger equation, which governs optical solitons with complex dynamics. By using the Huber loss for the PDE residual, they balanced robustness against large errors near steep gradients with smoothness elsewhere, reducing oscillations and improving accuracy over the L2 norm when combined with multi-view transfer learning. The parameter δ controls the transition between L1 and L2 behavior, making the Huber norm a flexible choice for nonlinear, multi-scale problems.

5.3.3 Sobolev Norm

Sobolev norms extend traditional norms by penalizing both function values and their derivatives, enforcing solution smoothness. This is vital for PDEs requiring regularity, such as wave equations. Yang et al. [68] proposed Sobolev training for PINNs, defining the loss as:

$$L(\theta) = \|Pu_\theta - f\|_{W^{k,p}([0,T];W^{l,q}(\Omega))}^p + \|Iu_\theta - g\|_{W^{l,q}(\Omega)}^p + \|Bu_\theta - h\|_{W^{k,p}([0,T];W^{l,q}(\partial\Omega))}^p,$$

where $W^{k,p}$ denotes the Sobolev space with k derivatives in the L^p norm, and P , I , and B are the PDE, initial, and boundary operators, respectively. For the wave equation, they included penalties on first and second derivatives ($k = 2$, $p = 2$), reducing artifacts near wave fronts and enhancing stability in long-time simulations. This approach outperforms the L2 norm by ensuring physically meaningful smoothness, though it increases computational cost due to derivative calculations.

To clarify the distinction between Sobolev training and the gradient-enhanced PINNs (gPINNs) discussed in Sect. 5.2.3, it is important to highlight their respective scopes and functional spaces. While gPINNs primarily focus on adding the first-order derivatives of the interior PDE residual to the loss function using the standard L^2 norm, Sobolev training provides a more comprehensive functional framework. Specifically, Sobolev norms enforce regularity not only on the PDE residual but also systematically on the initial and boundary operators by incorporating their derivatives into the loss. Furthermore, while standard gPINNs are restricted to L^2 -based squared errors, Sobolev formulations generalize to L^p spaces, allowing for tailored penalization strategies that rigorously match the underlying well-posedness theory of the governing PDEs.

While standard Sobolev loss formulations typically minimize integrated squared derivatives (i.e., the $W^{k,2}$ or H^k seminorms) which act as global average smoothers, they can fail to suppress severe localized deviations, such as those occurring near sharp boundary layers

or shock fronts. To achieve rigorous worst-case error control, recent advanced architectures have transitioned toward Sobolev max-norm-based losses operating in the $W^{k,\infty}$ space [69]. A $W^{k,\infty}$ loss design penalizes the essential supremum of the residual and its differential variations up to order k over the physical domain Ω :

$$\mathcal{L}_{W^{k,\infty}} = \sum_{|\alpha| \leq k} \text{ess sup}_{x \in \Omega} |D^\alpha \mathcal{R}(x; \theta)| \quad (2)$$

By substituting the standard L^2 integration with an L^∞ min-max proxy, this framework guarantees a strict upper bound on local approximation errors, transforming the training into a robust optimization problem immune to localized physical violations.

5.3.4 Custom Norms

Custom norms are tailored to specific PDE challenges, adapting the loss beyond standard formulations. Yu et al. [70] introduced gradient-enhanced PINNs (gPINNs), augmenting the loss with gradient penalties:

$$L(\theta) = \sum_{\alpha \in \mathcal{C}} m_\alpha \frac{1}{N_\alpha} \sum_{i=1}^{N_\alpha} |r_\alpha(x_i, \theta)|^p + w_g \sum_{i=1}^{N_g} |\nabla r_E(x_i, \theta)|^p,$$

where w_g weights the gradient term, and ∇r_E is the gradient of the PDE residual. For the advection–diffusion equation with high Péclet numbers, gPINNs reduced oscillations in steep gradient regions, improving accuracy over L2-based PINNs with fewer training points. This custom norm exemplifies how problem-specific enhancements can stabilize training and address limitations of conventional norms.

In conclusion, the choice of norm in PINN loss functions shapes training efficacy and solution quality. While the L2 norm remains a default, alternatives like L1, Huber, Sobolev, and custom norms offer targeted solutions, enabling PINNs to tackle diverse PDEs with improved robustness, smoothness, and accuracy.

5.4 Uncertainty Quantification in PINNs

Uncertainty quantification (UQ) is a critical aspect of PINNs, particularly when dealing with noisy or incomplete data, as it provides confidence intervals or probabilistic estimates around the model's predictions. This is essential in scientific and engineering applications, where decision-making is based on accurate and reliable output. In standard PINNs, the loss function balances data fidelity and adherence to physical laws, but UQ requires modifications to incorporate uncertainty estimates. Two primary approaches have emerged: Bayesian methods, which use probabilistic frameworks to model uncertainty, and variance-involved methods, which predict uncertainty by estimating solution variance. This subsection explores these approaches, highlighting key studies that have advanced UQ in PINNs. A summary of these uncertainty quantification approaches is presented in Table 5.

5.4.1 Bayesian Methods

Bayesian methods enhance uncertainty quantification in PINNs by introducing a probabilistic framework, particularly suited for scenarios with noisy or incomplete data. Unlike standard PINNs, where the loss function balances data fidelity and PDE residuals deterministically,

Table 5 Summary of uncertainty quantification approaches in PINNs

Category	References	Description
Bayesian Methods	Yang et al. [67]	Introduced Bayesian framework for uncertainty in noisy data
	Perez and Poncet [51]	Applied auto-weighted Bayesian PINNs to pore-scale imaging
	Liu et al. [41]	Extended Bayesian methods to extreme learning machines
Variance-Involved Methods	Shan et al. [56]	Predicted variance alongside mean for uncertainty quantification

Bayesian approaches treat neural network parameters as random variables, enabling the estimation of posterior distributions and providing confidence intervals alongside predictions.

In the Bayesian Physics-Informed Neural Network (B-PINN) framework proposed by Yang et al. [67], the PDE solution $u(\mathbf{x})$ is approximated by a neural network surrogate $\tilde{u}(\mathbf{x}; \theta)$, where θ represents the network parameters (weights and biases). These parameters are assigned a prior distribution $P(\theta)$, typically independent Gaussian distributions for each component. The framework addresses a general PDE system:

$$\mathcal{N}_{\mathbf{x}}(u; \lambda) = f, \quad \mathbf{x} \in \mathcal{D},$$

$$\mathcal{B}_{\mathbf{x}}(u; \lambda) = b, \quad \mathbf{x} \in \partial\mathcal{D},$$

where $\mathcal{N}_{\mathbf{x}}$ is a differential operator, $\mathcal{D}$ is the physical domain, λ denotes PDE parameters, f is the forcing term, and $\mathcal{B}_{\mathbf{x}}$ enforces boundary conditions. For forward problems, λ is known, while inverse problems involve inferring λ alongside u .

The dataset comprises noisy measurements:

$$\mathcal{D} = \mathcal{D}_u \cup \mathcal{D}_f \cup \mathcal{D}_b,$$

with $\mathcal{D}_u = \{(\mathbf{x}_u^{(i)}, \tilde{u}^{(i)})\}_{i=1}^{N_u}$, $\mathcal{D}_f = \{(\mathbf{x}_f^{(i)}, \tilde{f}^{(i)})\}_{i=1}^{N_f}$, and $\mathcal{D}_b = \{(\mathbf{x}_b^{(i)}, \tilde{b}^{(i)})\}_{i=1}^{N_b}$, where observations are modeled as:

$$\tilde{u}^{(i)} = u(\mathbf{x}_u^{(i)}) + \epsilon_u^{(i)}, \quad \epsilon_u^{(i)} \sim \mathcal{N}(0, (\sigma_u^{(i)})^2),$$

and similarly for $\tilde{f}^{(i)}$ and $\tilde{b}^{(i)}$. The likelihood reflects these Gaussian noise assumptions:

$$P(\mathcal{D}|\theta) = P(\mathcal{D}_u|\theta)P(\mathcal{D}_f|\theta)P(\mathcal{D}_b|\theta),$$

where, for example,

$$P(\mathcal{D}_u|\theta) = \prod_{i=1}^{N_u} \frac{1}{\sqrt{2\pi}\sigma_u^{(i)}} \exp\left(-\frac{(\tilde{u}(\mathbf{x}_u^{(i)}; \theta) - \tilde{u}^{(i)})^2}{2(\sigma_u^{(i)})^2}\right).$$

The posterior distribution is then:

$$P(\theta|\mathcal{D}) \propto P(\mathcal{D}|\theta)P(\theta).$$

For inverse problems, the Bayesian framework assigns independent priors $P(\theta)$ and $P(\lambda)$, yielding a joint posterior:

$$P(\theta, \lambda|\mathcal{D}) \propto P(\mathcal{D}|\theta, \lambda)P(\theta)P(\lambda).$$

This posterior is integrated into the loss function through the likelihood terms, enhancing robustness against overfitting and noisy data compared to standard PINNs. Predictions and uncertainty estimates are obtained by sampling from the posterior, computing statistics such as the mean and standard deviation of $\tilde{u}(\mathbf{x}; \theta)$.

Posterior estimation employs methods such as Hamiltonian Monte Carlo (HMC) or Variational Inference (VI). HMC simulates a Hamiltonian system for sampling, while VI approximates the posterior with a simpler distribution, minimizing Kullback–Leibler divergence. Yang et al. [67] applied B-PINNs to a 2D nonlinear Allen–Cahn equation, demonstrating that HMC outperforms VI and dropout-based methods in accuracy and uncertainty quantification under varying noise levels, as shown in Fig. 23.

Further advancements include Perez and Poncet [51], who developed auto-weighted Bayesian PINNs for pore-scale imaging, using Bayesian inference to dynamically adjust loss weights, enhancing reconstruction accuracy. Liu et al. [41] extended Bayesian methods to extreme learning machines (ELMs) within PINNs. By integrating Bayesian inference, they accelerated training while retaining UQ capabilities. Applied to parameter identification in PDEs with limited data, this approach demonstrated efficiency and reliability, adapting the loss function for data-scarce scenarios.

5.4.2 Variance-Involving Methods

Variance-involved methods enhance the neural network architecture by enabling it to output both the mean and variance of the solution, thereby facilitating direct UQ. This capability is critical for addressing predictive uncertainty in PINNs, a challenge that remains significant in real-world applications where data imperfections are prevalent.

Shan et al. [56] introduced Variance-Involving Physics-Informed Neural Networks (VI-PINNs), an extension of standard PINNs. Unlike vanilla PINNs, which output a single value representing the mean prediction $\mu(\mathbf{x})$, VI-PINNs modify the final layer of the network to produce two outputs: the predicted mean $\mu_\theta(\mathbf{x})$ and the variance $\sigma_\theta^2(\mathbf{x})$. The variance serves as a direct measure of predictive uncertainty. To ensure positivity of the variance, the output $\sigma_\theta^2(\mathbf{x})$ is processed through a softplus function, $\log(1 + \exp(\cdot))$, and a small constant (10^{-6}) is added for numerical stability. The architecture of VI-PINNs is depicted in Fig. 24.

In traditional regression problems, the mean squared error (MSE) loss is widely used. However, MSE fails to capture aleatoric uncertainty, which arises from inherent noise in the data. To address this, Shan et al. [56] assume that the observed data follows a Gaussian distribution, a reasonable assumption supported by the central limit theorem and common noise patterns in practical applications. Under this assumption, the probability of observing a target y_i given an input $\mathbf{x}_i$ is:

$$P(y_i | \mathbf{x}_i) = \frac{1}{\sqrt{2\pi\sigma_\theta^2(\mathbf{x}_i)}} \exp\left(-\frac{[y_i - \mu_\theta(\mathbf{x}_i)]^2}{2\sigma_\theta^2(\mathbf{x}_i)}\right).$$

Taking the negative logarithm yields the negative log-likelihood (NLL) loss:

$$-\log P(y_i | \mathbf{x}_i) = \frac{\log[2\pi\sigma_\theta^2(\mathbf{x}_i)]}{2} + \frac{[y_i - \mu_\theta(\mathbf{x}_i)]^2}{2\sigma_\theta^2(\mathbf{x}_i)}.$$

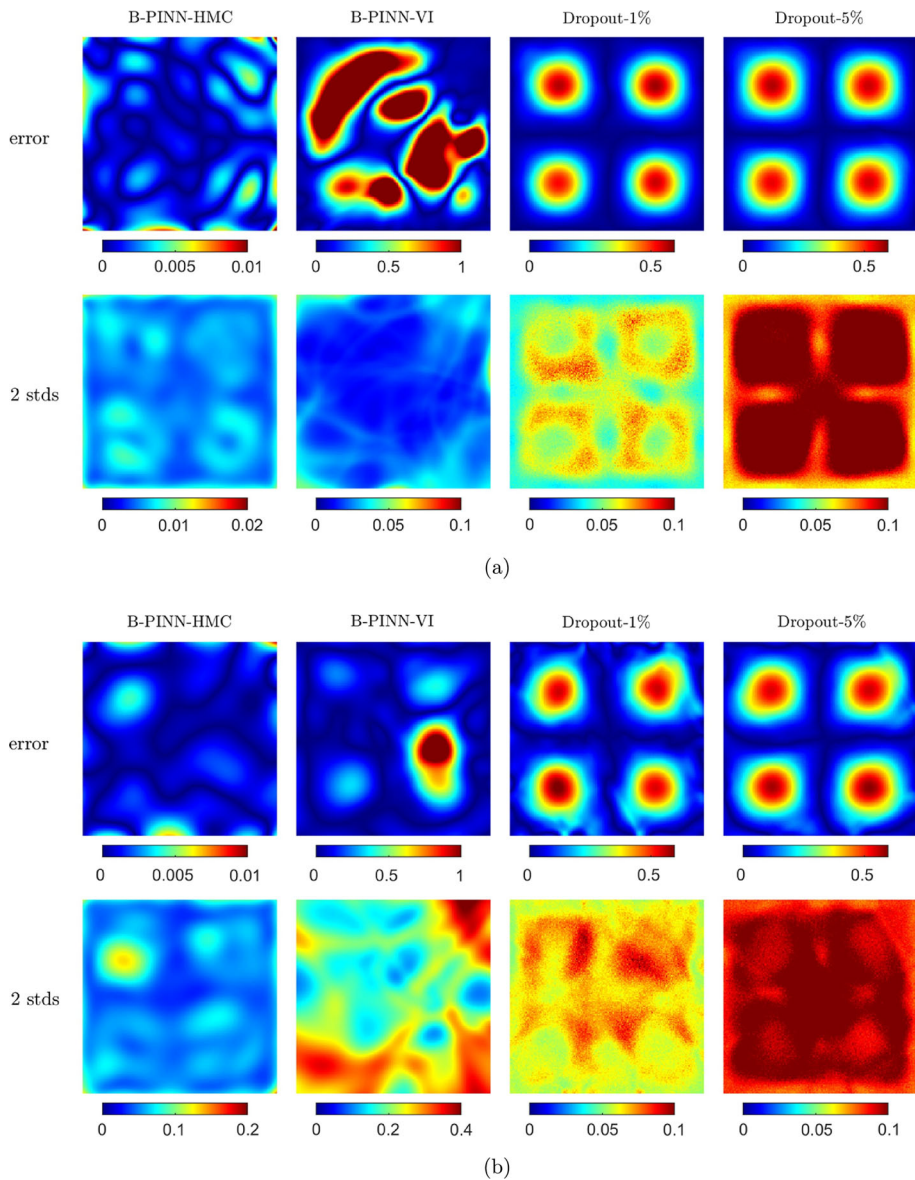


Fig. 23 2D Allen-Cahn equation - forward problem: Predicted errors and standard deviations for u from different methods with two data noise scales. **a** $\epsilon_f \sim \mathcal{N}(0, (0.01)^2)$, $\epsilon_b \sim \mathcal{N}(0, (0.01)^2)$. **b** $\epsilon_f \sim \mathcal{N}(0, (0.1)^2)$, $\epsilon_b \sim \mathcal{N}(0, (0.1)^2)$. Adapted from Yu et al. [67]

Ignoring the constant term for optimization purposes, the simplified NLL loss over N data points becomes:

$$\mathcal{L}_{NLL} = \frac{1}{N} \sum_{i=1}^N \left(\frac{\log[\sigma_{\theta}^2(\mathbf{x}_i)]}{2} + \frac{[y_i - \mu_{\theta}(\mathbf{x}_i)]^2}{2\sigma_{\theta}^2(\mathbf{x}_i)} \right).$$

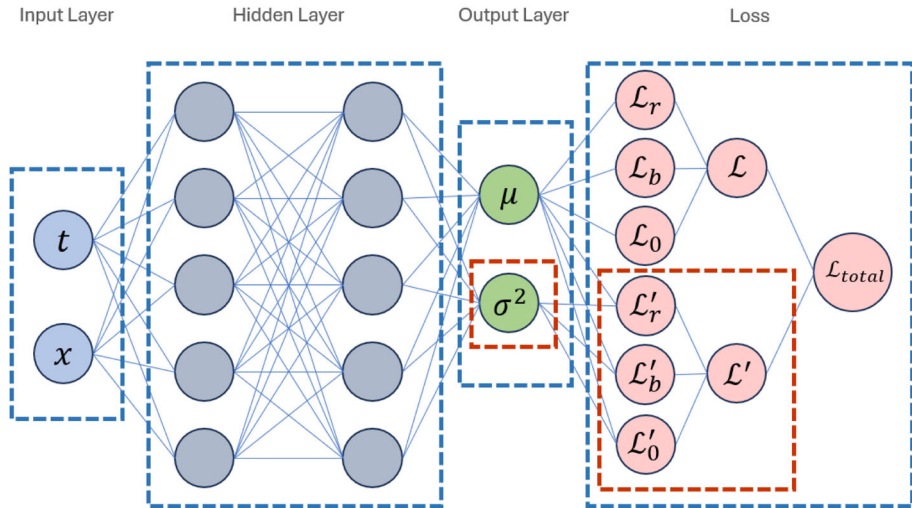


Fig. 24 Schematic of VI-PINNs. Terms marked with red dashed boxes are additional components compared to vanilla PINNs. Adapted from Shan et al. [56]

However, the authors noted that this standard NLL loss posed challenges during training. As $\sigma_\theta^2(\mathbf{x}_i)$ decreases, $\log \sigma_\theta^2(\mathbf{x}_i)$ becomes a large negative value, potentially destabilizing the optimization process. To remedy this, they proposed a modified NLL (MNLL) loss:

$$\mathcal{L}_{MNLL} = \frac{1}{N} \sum_{i=1}^N \left(\log[\sigma_\theta^2(\mathbf{x}_i) + 1] + \frac{[y_i - \mu_\theta(\mathbf{x}_i)]^2}{\sigma_\theta^2(\mathbf{x}_i)} \right).$$

The addition of 1 inside the logarithm ensures numerical stability even when the variance is small. In the context of PINNs, this MNLL loss is adapted to enforce the physical constraints of the PDE, boundary conditions, and initial conditions. For the PDE residuals, the loss term is:

$$\mathcal{L}'_{r,MNLL}(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left(\log[\sigma_\theta^2(\mathbf{x}_i^r) + 1] + \frac{[\mathcal{N}[\tilde{u}_\theta(\mathbf{x}_i^r)] - f(\mathbf{x}_i^r)]^2}{\sigma_\theta^2(\mathbf{x}_i^r)} \right),$$

where $\mathcal{N}$ is the differential operator, $\tilde{u}_\theta(\mathbf{x}) = \mu_\theta(\mathbf{x})$ is the predicted solution, and $\mathbf{x}_i^r = (t_i^r, x_i^r)$ are collocation points for the residuals. Similarly, the boundary condition loss is:

$$\mathcal{L}'_{b,MNLL}(\theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(\log[\sigma_\theta^2(\mathbf{x}_i^b) + 1] + \frac{[\mathcal{B}[\tilde{u}_\theta(\mathbf{x}_i^b)] - g(\mathbf{x}_i^b)]^2}{\sigma_\theta^2(\mathbf{x}_i^b)} \right),$$

and the initial condition loss is:

$$\mathcal{L}'_{0,MNLL}(\theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} \left(\log[\sigma_\theta^2(\mathbf{x}_i^0) + 1] + \frac{[\tilde{u}_\theta(\mathbf{x}_i^0) - h(\mathbf{x}_i^0)]^2}{\sigma_\theta^2(\mathbf{x}_i^0)} \right),$$

where $\mathbf{x}_i^b = (t_i^b, x_i^b)$ and $\mathbf{x}_i^0 = (0, x_i^0)$ denote boundary and initial points, respectively. The total loss for VI-PINNs combines these terms:

$$\mathcal{L}'(\theta) = \mathcal{L}'_{r,MNLL}(\theta) + \mathcal{L}'_{b,MNLL}(\theta) + \mathcal{L}'_{0,MNLL}(\theta).$$

Table 6 Summary of training and optimization enhancements in PINNs

Category	References	Description
Meta-Learning	Psaros et al. [53]	Optimized loss functions across multiple tasks
	Cheng and Alkhalifah [14]	Enhanced initialization for wavefield solutions
Transfer Learning	Liu et al. [41]	Guided optimization along low-loss paths using adaptive transfer learning
Loss Landscape Engineering	Gopakumar et al. [25]	Used sparse data to simplify optimization through loss landscape engineering
Regularization Techniques	Ma et al. [47] brid activation functions	
Optimization Algorithms	Urbán et al. [60]	Enhanced optimizers for better accuracy in PINN training
	Ko and Park [37]	Accelerated training for stiff PDEs using variable-scaling methods

This loss function enables VI-PINNs to simultaneously optimize the mean predictions and uncertainty estimates. When applied to the heat equation with stochastic boundary conditions, VI-PINNs demonstrated faster convergence and provided uncertainty estimates comparable to Monte Carlo simulations, while being computationally more efficient than Bayesian methods.

These advancements highlight the critical role of UQ in PINNs. By integrating variance-involved techniques into the loss function, VI-PINNs enhance the robustness of PINNs, making them better suited for real-world scenarios with noisy or incomplete data.

5.5 Training and Optimization Enhancements

In PINNs, the loss function serves as the cornerstone of training, integrating data fidelity terms with residuals of PDEs to enforce physical consistency. However, optimizing this composite loss function poses challenges such as gradient imbalances, stiff dynamics, and rugged loss landscapes. To address these issues, several strategies have been developed to enhance training efficiency and solution accuracy by refining the loss function's design, adaptation, or optimization process. This subsection examines five key approaches—Meta-Learning, Transfer Learning, Loss Landscape Engineering, Regularization Techniques, and Optimization Algorithms—and their contributions to improving PINN performance through loss function innovation. An organized summary of these loss-oriented enhancements, along with notable references and brief descriptions, is shown in Table 6.

5.5.1 Meta-Learning

Meta-learning enhances the training of PINNs by optimizing loss functions across a distribution of PDE tasks, enabling rapid adaptation to new problems. This approach leverages shared knowledge from related tasks to design loss functions that improve convergence speed and generalization, even for unseen PDEs.

Psaros et al. [53] proposed a gradient-based meta-learning framework for the offline discovery of PINN loss functions. They define *task distributions* based on parametrized

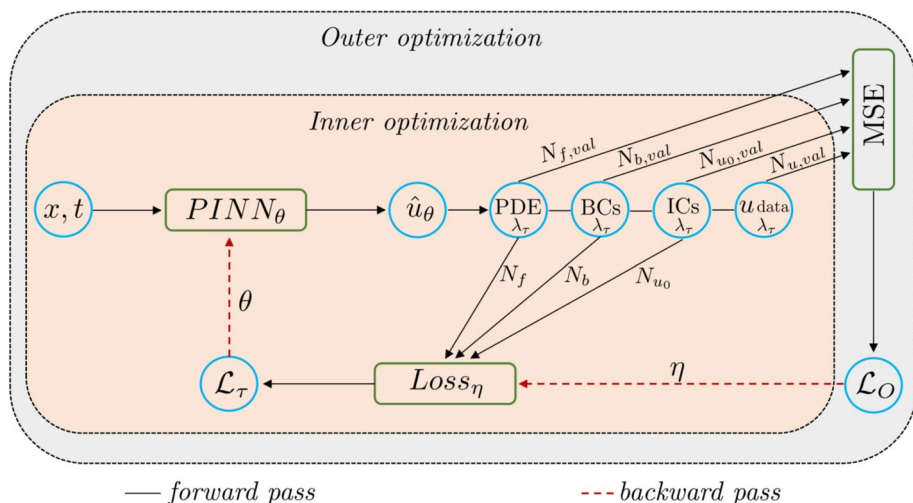


Fig. 25 Schematic illustration of the meta-learning method for discovering PINN loss functions. The inner optimization updates PINN parameters θ based on the current learned loss $\mathcal{L}_\tau$, evaluated on residuals for PDE, boundary, and initial conditions. The outer optimization updates loss parameters η based on the outer objective $\mathcal{L}_O$, typically the MSE of optimized parameters over validation data [53]

PDEs, where each task corresponds to a PDE instance governed by parameters $\lambda \sim p(\lambda)$. For instance, a PDE family such as $\mathcal{N}_x(u; \lambda) = f$ generates tasks by varying λ . The goal is to learn loss function parameters η that optimize PINN performance across these tasks.

The framework employs a bi-level optimization approach:

$$\min_{\eta} \mathcal{L}_O(\theta^*(\eta), \eta) \quad \text{s.t.} \quad \theta^*(\eta) = \{\theta_\tau^*(\eta)\}_{\tau=1}^T \quad \text{with} \quad \theta_\tau^*(\eta) = \arg \min_{\theta} \mathcal{L}_\tau(\theta, \eta),$$

where $\mathcal{L}_\tau(\theta, \eta)$ is the task-specific loss for task τ , parameterized by PINN parameters θ and loss parameters η , and $\mathcal{L}_O$ is the outer objective evaluating performance across T tasks. The inner optimization trains PINNs for individual tasks using the current loss function, while the outer optimization updates η to minimize the average error across tasks.

The meta-learning algorithm alternates between two steps: 1. Inner Optimization: PINNs solve sampled PDE tasks for a few iterations using $\mathcal{L}_\tau(\theta, \eta)$, tracking gradients with respect to η . 2. Outer Optimization: The loss parameters η are updated based on the mean squared error (MSE) of the optimized PINN parameters θ_τ^* across tasks.

This process is depicted in Fig. 25, which illustrates the interplay between updating θ and η .

Psaros et al. [53] further identified two desirable properties of meta-learned loss functions—enhancing optimization guidance and stability—and proposed regularization techniques to enforce them. These properties ensure the loss function effectively balances physics and data terms across diverse tasks. Their computational examples, including the heat and Burgers' equations, demonstrate that meta-learned loss functions accelerate convergence and reduce training time compared to standard PINN losses, even for out-of-distribution tasks and varying PINN architectures.

In a related development, Cheng and Alkhalifah [14] applied meta-learning to optimize initial weights for wavefield solutions in PINNs. Their method minimizes a meta-loss across

wave propagation tasks, enabling faster convergence to low-loss regions in seismic wave problems, further highlighting meta-learning's versatility in enhancing PINN efficiency.

This meta-learning approach underscores a significant advancement in loss function design. However, it is crucial to emphasize that the computational efficiency of this paradigm is highly conditional. The bi-level optimization framework incurs a massive upfront (offline) computational cost, as it requires training the PINN a high number of times across the task distribution to converge on the optimal loss parameters η . Therefore, this approach is only computationally efficient overall if the meta-learned loss function is subsequently deployed across a sufficiently large number of new, unseen PDE tasks (the online phase), thereby amortizing the heavy initial investment. Conversely, if one needs to solve only a single or a few isolated PDE instances, the immense offline cost of discovering the loss function renders this method significantly more computationally demanding than utilizing a vanilla PINN or standard heuristic weighting.

5.5.2 Transfer Learning

Transfer learning leverages knowledge from pre-trained PINN models to enhance training efficiency for new PDE tasks. The loss function is central to this process, guiding the adaptation of a pre-trained model to a target problem. Traditional transfer learning for PINNs (tPINN) initializes the network with weights θ_s from a source task (parameterized by λ_s) and fine-tunes them for a target task (parameterized by λ_t). However, tPINN is less effective when the parameter variation between λ_s and λ_t is large, as significant differences in solutions hinder direct knowledge transfer.

To address this, Liu et al. [42] proposed an adaptive transfer learning method for PINNs, termed AtPINN, which introduces the concept of the *minimum energy path* (MEP). The MEP is a low-loss trajectory in the parameter space connecting the source and target tasks, inspired by the loss landscape of neural networks. In AtPINN, PDE parameters λ are treated as trainable variables, initialized at λ_s and adaptively updated toward λ_t during training. This process constructs a stable optimization path, ensuring efficient transfer even for large parameter variations.

The AtPINN loss function is defined in an extended parameter space $V = V_1 \oplus V_2$, where V_1 contains the network weights θ and V_2 contains the PDE parameters λ . It is expressed as:

$$\mathcal{L}_{\text{AtPINN}}(\theta, \lambda) = \mathcal{L}_{\text{PINN}}(\theta, \lambda) + w_P \mathcal{L}_P(\lambda),$$

where $\mathcal{L}_{\text{PINN}}(\theta, \lambda)$ is the standard PINN loss (including PDE residuals, initial, and boundary conditions), and $\mathcal{L}_P(\lambda) = \|\lambda - \lambda_t\|^2$ drives λ toward the target parameters λ_t , with w_P as a weighting factor. The parameters are updated via gradient descent:

$$\lambda^{(i+1)} = \lambda^{(i)} - \eta \nabla_{\lambda} \mathcal{L}_{\text{AtPINN}}(\theta, \lambda),$$

while θ adapts to minimize $\mathcal{L}_{\text{PINN}}(\theta, \lambda)$, maintaining a low-loss path akin to the MEP. Figure 26 illustrates this process, showing intermediate pivots $\lambda_1, \lambda_2, \dots, \lambda_N$ guiding the transfer from λ_s to λ_t .

To ensure stability, adaptive weights are assigned to the loss terms. For instance, when λ appears in the PDE residuals, the loss is adjusted as:

$$\mathcal{L}_{\text{AtPINN}}(\theta, \lambda) = \frac{|\mathcal{L}_F(\theta, \lambda)|}{c} \mathcal{L}_F(\theta, \lambda) + \mathcal{L}_I(\theta) + \mathcal{L}_B(\theta) + \mathcal{L}_P(\lambda),$$

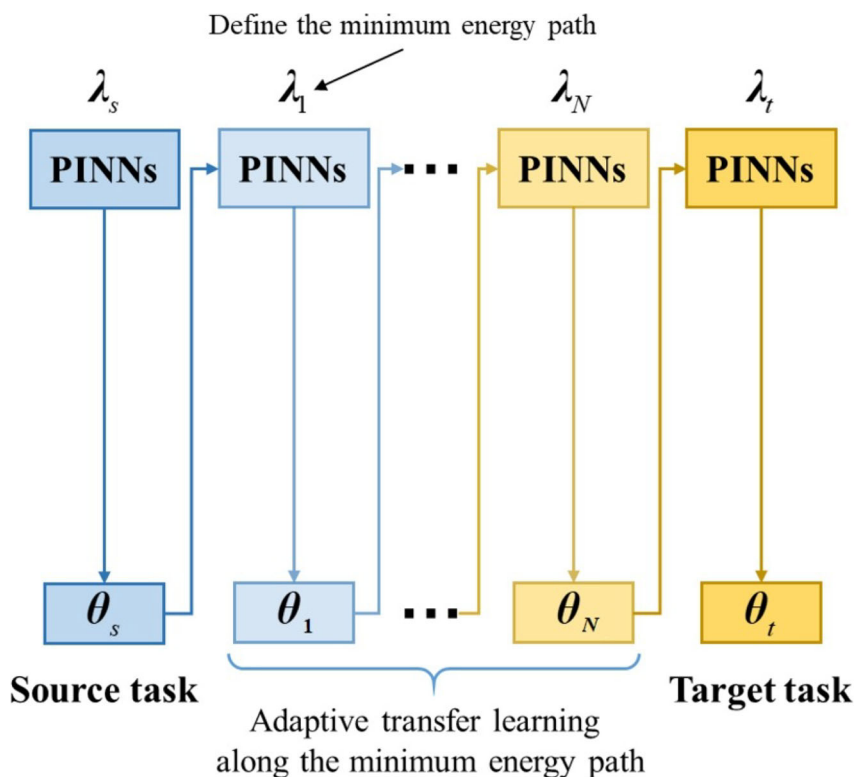


Fig. 26 Schematic of the adaptive transfer learning process in AtPINN. PDE parameters λ are updated from the source λ_s to the target λ_t via intermediate pivots, ensuring optimization along a low-loss path [42]

where $\mathcal{L}_F$, $\mathcal{L}_I$, and $\mathcal{L}_B$ represent the PDE residual, initial, and boundary condition losses, respectively, and c is a small constant (e.g., 0.001). This weighting keeps the residual loss balanced, preventing it from dominating the optimization.

Applied to variants of the Navier–Stokes equations with large Reynolds number variations, AtPINN significantly reduced training time compared to standard PINN and tPINN approaches. This efficiency stems from its ability to dynamically adjust the loss terms, guiding optimization along low-loss paths without sacrificing accuracy. AtPINN thus offers a practical solution for computationally intensive PDE tasks, enhancing the applicability of PINNs in scenarios with limited resources.

5.5.3 Loss Landscape Engineering

Loss landscape engineering aims to enhance the optimization of PINNs by reshaping the loss function's topography to facilitate convergence. By modifying the loss landscape, these methods reduce the risk of entrapment in local minima and improve training efficiency, particularly for complex PDEs with challenging optimization landscapes.

Gopakumar et al. [25] introduced a data regulation technique that incorporates sparse or coarse data points into the PINN loss function to smooth the loss landscape. Unlike vanilla PINNs, which rely on uniform sampling for PDE residuals, boundary, and initial conditions, their approach adds a reconstruction loss term for strategically selected data points. The loss

function for a sparse regulated PINN is defined as:

$$\mathcal{L}(\theta) = \mathcal{L}_r(\tilde{u}, \mathbf{x}, t) + \mathcal{L}_b(\tilde{u}, \mathbf{x}_b, t) + \mathcal{L}_0(\tilde{u}, \mathbf{x}_0) + \mathcal{L}_s(\tilde{u}, \mathbf{x}_s, t),$$

where $\mathcal{L}_r = \|\mathcal{N}[\tilde{u}(\mathbf{x}, t; \theta)] - f(\mathbf{x}, t)\|^2$ enforces the PDE residuals, $\mathcal{L}_b = \|\mathcal{B}[\tilde{u}(\mathbf{x}_b, t; \theta)] - g(\mathbf{x}_b, t)\|^2$ and $\mathcal{L}_0 = \|\tilde{u}(\mathbf{x}_0, 0; \theta) - h(\mathbf{x}_0)\|^2$ enforce boundary and initial conditions, respectively, and $\mathcal{L}_s = \|\tilde{u}(\mathbf{x}_s, t; \theta) - u(\mathbf{x}_s, t)\|^2$ is the reconstruction loss for sparse data points $\mathbf{x}_s$. Here, $\tilde{u}(\mathbf{x}, t; \theta)$ is the neural network's predicted solution, and $\mathbf{x}$, $\mathbf{x}_b$, $\mathbf{x}_0$, and $\mathbf{x}_s$ denote collocation points for the PDE, boundary, initial, and sparse data, respectively.

The sparse data points are chosen from high-informative regions, such as areas with steep gradients, to guide the optimizer toward the solution. This data regulation morphs the loss landscape, creating well-defined minima that are easier for the optimizer to reach. Applied to the Poisson equation, sparse regulated PINNs achieved an ℓ_2 error of 0.001 compared to 0.078 for vanilla PINNs after 5000 epochs, demonstrating faster convergence and higher accuracy.

For scenarios where sparse data is unavailable, Gopakumar et al. [25] extended the approach to coarse regulated PINNs, using data from inexpensive, coarse-grid numerical solvers. This method similarly smooths the loss landscape, enabling faster convergence. In tests with the Navier–Stokes equations, coarse regulated PINNs exhibited comparable regulation effects to sparse data, leveraging the mesh-invariant nature of PINNs to refine coarse approximations. However, when using experimental data (e.g., linear profiles from specific domain regions), the regulation was less effective due to insufficient coverage, highlighting the importance of data representativeness.

This hybrid unsupervised–supervised approach reduces both approximation and optimization errors by biasing the network toward the solution's vicinity, allowing fine-tuning via PDE constraints. The need for pre-existing data is mitigated by using coarse or sparse samples, which act as partial guides rather than complete solutions, making loss landscape engineering a versatile strategy for enhancing PINN performance.

5.5.4 Regularization Techniques

Regularization techniques modify the loss function to enforce desirable solution properties or prevent overfitting, directly influencing training outcomes. Ma et al. [47] integrated Lasso regression into the PINN loss function to perform sparse coefficient regression in PDEs. By adding an L1 penalty term, their method identifies critical PDE coefficients while discarding insignificant ones, enhancing interpretability and accuracy. Tested on the advection–diffusion equation, this approach successfully recovered sparse solutions with fewer iterations. Complementing this, Zafar et al. [72] explored hybrid activation functions within the loss function framework. By combining activation types (e.g., ReLU and tanh), they tailored the loss to improve gradient flow and convergence speed. Applied to chaotic PDE systems, their technique outperformed standard activations, achieving greater stability and efficiency in training.

5.5.5 Optimization Algorithms

Advanced optimization algorithms are critical for addressing the challenges of PINN loss functions, such as multi-scale dynamics and stiffness, by tailoring the training process to navigate complex loss landscapes. These methods enhance convergence speed and solution accuracy, particularly for stiff PDEs with sharp transitions or high-frequency behaviors.

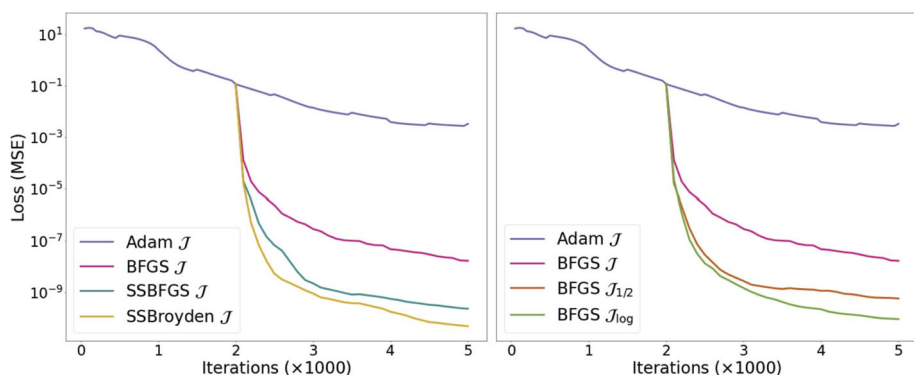


Fig. 27 Evolution of the loss function for the current-free Grad-Shafranov equation. Left: Comparison of Adam, BFGS, SSBFGS, and SSBroyden using standard MSE loss $\mathcal{L}$. Right: BFGS with modified loss functions $\mathcal{L}$, $\mathcal{L}_{1/2}$, and $\mathcal{L}_{\log}$. SSBFGS and SSBroyden achieve up to three orders of magnitude lower loss than Adam, while $\mathcal{L}_{\log}$ accelerates convergence [60]

Urbán et al. [60] introduced self-scaled quasi-Newton methods, namely SSBFGS and SSBroyden, to improve PINN optimization. These methods modify the Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm by incorporating scaling factors τ_k and ϕ_k to the inverse Hessian approximation H_k . For SSBFGS, the scaling is defined as:

$$\tau_k^{(1)} = \min \left\{ 1, \frac{\mathbf{y}_k \cdot \mathbf{s}_k}{\mathbf{s}_k \cdot H_k^{-1} \mathbf{s}_k} \right\}, \quad \phi_k = 1,$$

where $\mathbf{y}_k = \nabla \mathcal{L}(\Theta_{k+1}) - \nabla \mathcal{L}(\Theta_k)$, $\mathbf{s}_k = \Theta_{k+1} - \Theta_k$, and Θ_k are the network parameters. For SSBroyden, $\tau_k^{(2)}$ and $\phi_k^{(1)}$ are computed using auxiliary variables (e.g., a_k , σ_k , θ_k) to ensure super-linear convergence. These methods precondition the loss landscape, transforming the parameter space to a better-conditioned space $\mathbf{z} = H_k^{-1/2} \Theta$, reducing the Hessian condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ from $\sim 10^{12}$ to a more manageable range. Applied to the current-free Grad-Shafranov equation, SSBFGS and SSBroyden reduced the loss by two and three orders of magnitude, respectively, compared to standard BFGS, achieving relative ℓ_2 errors as low as 1.0×10^{-7} for the flux function $\mathcal{P}$.

Additionally, Urbán et al. [60] explored loss function modifications by applying a monotonic function g , such as $\mathcal{L}_g = \sqrt{\mathcal{L}}$ or $\mathcal{L}_g = \log \mathcal{L}$, to the mean squared error (MSE) loss $\mathcal{L}$. This adjusts the Hessian as $\text{hess}(\mathcal{L}_g) \approx g'(\mathcal{L}) \text{hess}(\mathcal{L})$ near the minimum, improving convergence by up to two orders of magnitude, with $\mathcal{L}_{\log}$ slightly outperforming $\mathcal{L}_{1/2}$. Figure 27 illustrates the convergence behavior for different optimizers and loss modifications.

Complementing this, Ko and Park [37] proposed VS-PINN, a variable-scaling method to address stiff PDEs by rescaling the spatial domain via $\mathbf{x} \mapsto \mathbf{x}/N$. For a PDE $\mathcal{F}(\mathbf{x}; u, Du, D^2u, \dots, D^k u) = 0$ on domain Ω , the scaled PDE becomes $\mathcal{F}(\mathbf{x}/N; v, N Dv, N^2 D^2v, \dots, N^k D^k v) = 0$ on $N\Omega$, where $v(\mathbf{x}) = u(\mathbf{x}/N)$. The corresponding loss function is:

$$\begin{aligned} \mathcal{L}_{\text{scaled}} = & \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} \left\| \mathcal{F} \left(\frac{\mathbf{x}_i^r}{N}; v(\mathbf{x}_i^r), N Dv(\mathbf{x}_i^r), \dots, N^k D^k v(\mathbf{x}_i^r) \right) \right\|_2^2 \\ & + \frac{\lambda_b}{N_b} \sum_{j=1}^{N_b} \left\| v(\mathbf{x}_j^b) - g \left(\frac{\mathbf{x}_j^b}{N} \right) \right\|_2^2, \end{aligned}$$

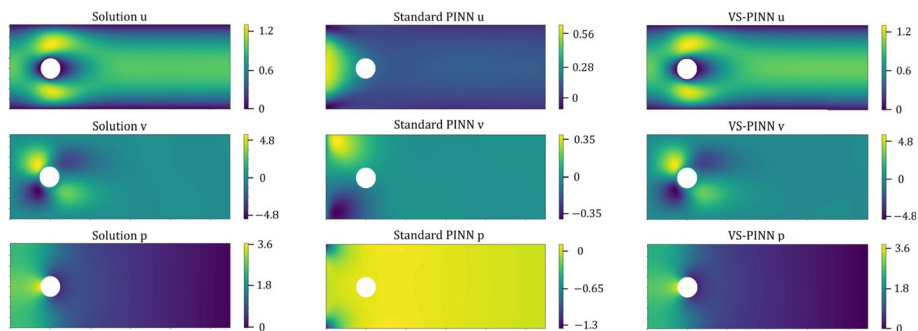


Fig. 28 Contour plots for the Navier–Stokes equations. Left: Exact solutions for velocity and pressure. Middle: Predictions using standard PINN ($N = 1$). Right: Predictions using VS-PINN ($N = 10$). VS-PINN significantly improves accuracy, especially near sharp transitions [37]

where $\mathbf{x}_i^r \sim \mathcal{U}(N\Omega)$ and $\mathbf{x}_j^b \sim \mathcal{U}(N\partial\Omega)$ with $\mathcal{U}(\cdot)$ explicitly denoting the uniform probability distribution over the given domain for sampling collocation points, and λ_r, λ_b are weights adjusted to balance the scaling effects. This approach mitigates sharp transitions in stiff PDEs by stretching the domain, reducing gradient imbalances. Applied to the Allen–Cahn equation, VS-PINN achieved a mean relative ℓ_2 error of 0.44×10^{-2} (supervised) compared to 1.93×10^{-2} for standard PINNs. For the Navier–Stokes equations, VS-PINN with $N = 10$ reduced errors to 1.85×10^{-2} (velocity) and 0.24×10^{-2} (pressure) using Adam+L-BFGS, as shown in Fig. 28.

These advancements highlight the synergy between optimizer design and loss function modifications. SSBFGS and SSBroyden address ill-conditioned Hessians, while VS-PINN mitigates stiffness through domain scaling, collectively enabling robust and efficient PINN training for complex PDEs.

These enhancement strategies collectively demonstrate the pivotal role of the loss function in PINN training. Whether through meta-learning’s task-adaptive losses, transfer learning’s guided optimization, landscape engineering’s simplified topography, regularization’s tailored constraints, or advanced algorithms’ dynamic adjustments, each approach refines how the loss function is crafted and optimized. Together, they enable PINNs to tackle a broader range of PDE problems with improved efficiency and precision, advancing their applicability in scientific computing.

5.6 Innovative Architectures

Innovative architectures in PINNs extend beyond traditional fully connected networks, incorporating designs that better capture the spatial, temporal, or multi-scale nature of PDEs. These architectures often influence how the loss function is evaluated or optimized, leading to more efficient training and improved solution accuracy. This subsection explores three such architectures: Convolutional PINNs, which leverage convolutional layers for spatial and temporal features; Incremental Learning approaches, which build solutions sequentially; and Variable-Separated Networks, which decompose solutions for multi-scale problems. Each of these designs impacts the loss function’s role in guiding the network toward accurate PDE solutions. Table 7 provides a structured summary of these architectural innovations along with key references and descriptions of their contributions.

Table 7 Summary of innovative architectures in PINNs

Category	References	Description
Convolutional PINNs	Zhang et al. [74]	Used convolutional layers for spatial feature extraction in multi-receptive-field CNNs
	Yuan et al. [71]	Designed physics-informed CNNs for space-time PDEs
Incremental Learning	Dekhovitch et al. [15]	Learned PDEs sequentially with subnetworks for incremental learning
Variable-Separated Networks	Liu et al. [43]	Separated variables for multi-scale predictions in blood flow simulations

5.6.1 Convolutional PINNs

Convolutional PINNs leverage convolutional neural networks (CNNs) to enhance the solution of PDEs by exploiting their ability to capture local spatial and temporal patterns. These architectures streamline the evaluation of PDE residuals in the loss function, improving convergence and accuracy for problems with complex dynamics.

Zhang et al. [74] introduced MRF-PINN, a multi-receptive-field CNN framework designed to handle multi-scale spatial features. MRF-PINN employs six encoder-decoder pairs, each with a distinct receptive field size determined by a convolutional kernel of size $k_s \times k_s$ (typically $k_s = 3$) and dilation rate d_r , yielding receptive fields $RF = k_s + (k_s - 1) \cdot d_r$. These fields range from $1/1$ to $1/32$ of the global domain scale X , capturing both fine and coarse features. The forward propagation for the i -th encoder-decoder pair is given by:

$$u_{i,1} = \sigma(u_0 \otimes W_{i,1} + b_{i,1}), \quad u_{i,j} = \mathcal{D}_4(\sigma(u_{i,j-1} \otimes W_{i,j} + b_{i,j})), \quad j = 2, 3,$$

$$u_{i,j} = \mathcal{U}_2(\sigma(u_{i,j-1} \otimes W_{i,j} + b_{i,j})), \quad j = 4, 5, 6, \quad u_i = \mathcal{U}_2(\sigma(u_{i,6} \otimes W_{i,7} + b_{i,7})),$$

where u_0 is the initial field (mean-filtered from boundary values), σ is the PReLU activation, $\otimes$ denotes convolution, $\mathcal{D}_4$ and $\mathcal{U}_2$ represent bilinear downsampling and upsampling, and $W_{i,j}, b_{i,j}$ are weights and biases. The final prediction is a weighted superposition:

$$\hat{u}_{\text{MRF-PINN}} = \sum_{i=1}^6 \theta_i u_i,$$

where θ_i are learnable scalars. To ensure accurate derivative computation, MRF-PINN uses a Taylor polynomial for boundary padding, enabling high-order finite difference (e.g., 6th-order central difference) for PDE residuals. A dimensional balance approach normalizes loss terms:

$$\mathcal{L}_{\text{PDE}} = X^{\beta_1} U^{\beta_2} \frac{1}{N} \sum_{i \in \Omega} f^2(\epsilon u_i, \nabla_{\mathbf{x}} \epsilon u_i, \nabla_{\mathbf{x}}^2 \epsilon u_i, \dots),$$

$$\mathcal{L}_{\text{BC}} = X^{\beta_3} U^{\beta_4} \frac{1}{N_{\partial\Omega}} \sum_{i \in \partial\Omega} (\epsilon u_i - \epsilon u_i)^2, \quad \mathcal{L}_{\text{data}} = X^{\beta_5} U^{\beta_6} \frac{1}{N_{\text{data}}} \sum_{i \in D} (\epsilon u_i - \epsilon u_i)^2,$$

with weights $\lambda_{\text{PDE}} = X^{-\beta_1} U^{-\beta_2}$, $\lambda_{\text{BC}} = X^{-\beta_3} U^{-\beta_4}$, and $\lambda_{\text{data}} = X^{-\beta_5} U^{-\beta_6}$, where X and U explicitly denote the characteristic physical scales (e.g., global reference maximums) of the spatial domain and the target solution field, respectively. Consequently, $\epsilon u = u/U$ represents the dimensionless target variable, $\epsilon x = x/X$ is the dimensionless spatial coordinate,

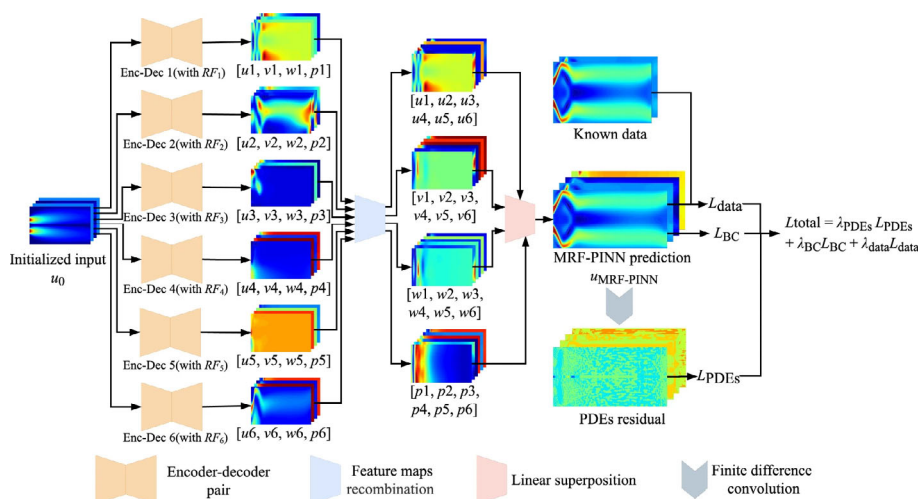


Fig. 29 The MRF-PINN framework with six encoder-decoder pairs, each assigned distinct convolutional receptive fields (RFs) ranging from $1/1$ to $1/32$ of the global domain scale, enabling multi-scale feature extraction [74]

and $\beta_1, \dots, \beta_6$ are derived from PDE dimensions. Applied to the Navier–Stokes equations, MRF-PINN reduced solving errors compared to MLP-PINN and UNet-PINN, with faster convergence. Figure 29 illustrates the MRF-PINN framework.

Yuan et al. [71] developed f-PICNN, a physics-informed convolutional neural network (CNN) for space-time PDEs, utilizing nonlinear convolutional units (NCUs) to approximate differential operators via finite difference schemes, such as Euler, Crank–Nicolson, and 4th-order Runge–Kutta. The network operates autoregressively, taking the state at time t , u_t , as input to predict the next time step:

$$u_{t+1} = f_{\text{CNN}}(u_t),$$

where f_{CNN} represents the f-PICNN model. NCUs employ convolutional kernels (e.g., size 3 for 1D, 3×3 for 2D) to compute derivatives, such as:

$$\frac{\partial u_t}{\partial x} \approx \frac{1}{\Delta x} \left(\frac{1}{2}, 0, \frac{1}{2} \right) [u_{t,i-1}, u_{t,i}, u_{t,i+1}]^T, \quad \frac{\partial^2 u_t}{\partial x^2} \approx \frac{1}{\Delta x^2} (1, -2, 1) [u_{t,i-1}, u_{t,i}, u_{t,i+1}]^T,$$

for 1D cases, with analogous formulations for 2D using finite difference filters. Initial conditions are hard-encoded as the first time-step solution, Dirichlet boundary conditions are softly enforced via a loss term, and Neumann conditions (which specify the normal derivative of the solution on the boundary, i.e., $\nabla u \cdot \mathbf{n} = g$ on $\partial\Omega$, where $\mathbf{n}$ is the outward unit normal vector) are hard-encoded through padding operations to maintain domain consistency. The loss function is unsupervised, relying on PDE residuals:

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{m=1}^{N_f} \|R_m(x_i, y_j, t_k, \theta, \gamma)\|_2^2,$$

where $R(x, t, \theta, \gamma) = \Delta u_\theta + \mathcal{N}[u_\theta, \nabla_{\mathbf{x}} u_\theta, \nabla_{\mathbf{x}}^2 u_\theta, \dots, \gamma]$ represents the PDE residual, θ denotes network parameters, and γ is the PDE coefficient. For Dirichlet boundary conditions,

a boundary loss is included:

$$\mathcal{L}_u = \frac{1}{N_u} \sum_{i=1}^{N_u} \|\hat{u}_i - u_i\|_2^2,$$

where $\hat{u}_i$ and u_i are the predicted and ground truth values at N_u boundary points, respectively. The total loss is:

$$\mathcal{L} = \omega_u \mathcal{L}_u + \omega_f \mathcal{L}_f,$$

with weights $\omega_u = \omega_f = 1$ typically, though $\mathcal{L} = \mathcal{L}_f$ when boundary conditions are hard-encoded. Applied to the 2D Burgers equation:

$$\frac{\partial u}{\partial t} + u \cdot \left(\frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} \right) = \vartheta \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \quad (x, y, t) \in [0, 1] \times [0, 1] \times (0, 1],$$

with $\vartheta = 0.02$, initial condition $u_0 = \sin(\pi x) \cos(\pi x)$, and non-zero boundary conditions $u(0, y, t) = u(1, y, t) = 0$, $u(x, 0, t) = e^{-2\pi^2 t} \sin(\pi x)$, $u(x, 1, t) = -e^{-2\pi^2 t} \sin(\pi x)$, or Dirichlet conditions with $\vartheta = 0.025$, $u_0 = \sin(2\pi x) \cos(2\pi x)$, and zero boundary conditions, f-PICNN achieved a mean absolute error of $\sim 10^{-3}$, compared to $\sim 10^{-2}$ for standard PINNs. The training time was significantly reduced to 459 s versus 3584 s for PINNs, demonstrating superior efficiency and accuracy. Figure 30 presents solution comparisons at time points $t = 0.10, 0.40, 0.80$.

These convolutional architectures enhance PINN performance by leveraging local feature extraction and tailored derivative approximations, making them well-suited for complex, time-dependent PDEs.

5.6.2 Incremental Learning

Incremental learning in PINNs involves training the network sequentially, using task-specific subnetworks to solve simpler or partial versions of PDEs before addressing more complex ones. This approach tailors the loss function to focus on specific PDE terms or domain regions at each stage, enhancing convergence and generalization.

Dekhovitch et al. [15] introduced iPINNs, an incremental learning framework that creates a subnetwork N_k with parameters $\theta_k \subset \theta$ for each PDE task k , allowing subnetworks to overlap and share knowledge. The method learns a sequence of PDEs, such as the 1D convection equation:

$$\frac{\partial u}{\partial t} + \beta_k \frac{\partial u}{\partial x} = 0, \quad x \in [0, 2\pi], \quad t \in [t_0, T], \quad u(x, 0) = \sin x, \quad u(0, t) = u(2\pi, t),$$

or the reaction-diffusion equation:

$$\begin{aligned} \frac{\partial u}{\partial t} - v \frac{\partial^2 u}{\partial x^2} - \rho u(1 - u) &= 0, \quad x \in [0, 2\pi], \quad t \in [0, 1], \quad u(x, 0) = e^{-\frac{(x-\pi)^2}{2(\pi/4)^2}}, \\ u(0, t) &= u(2\pi, t), \end{aligned}$$

where β_k , v , and ρ are PDE coefficients. Each subnetwork is trained on a task-specific dataset $D_k = \{(x, t, k) : x \in [0, 2\pi], t \in [t_0, T], k \in \mathbb{N}\}$, with the loss for task k defined as:

$$\mathcal{L}_k(\theta_k; D_k) = \frac{1}{N_f} \sum_{m=1}^{N_f} \|\mathcal{F}_k(u_{\theta_k}(x_m, t_m))\|_2^2 + \frac{1}{N_b} \sum_{i=1}^{N_b} \|u_{\theta_k}(x_i, t_i) - g(x_i, t_i)\|_2^2,$$

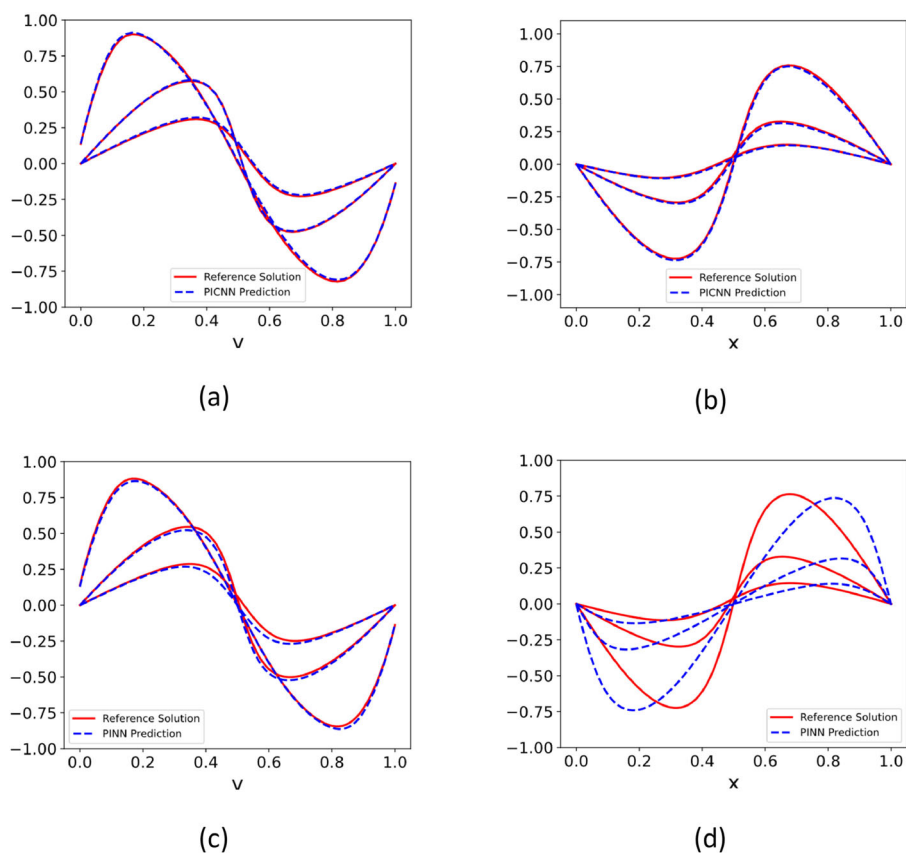


Fig. 30 Comparisons of f-PICNN and PINN predictions for the 2D Burgers equation at $t = 0.10, 0.40, 0.80$. **a** f-PICNN (non-zero BC); **b** f-PICNN (Dirichlet BC); **c** PINN (non-zero BC); **d** PINN (Dirichlet BC). f-PICNN achieves lower errors ($\sim 10^{-3}$) and faster convergence [71]

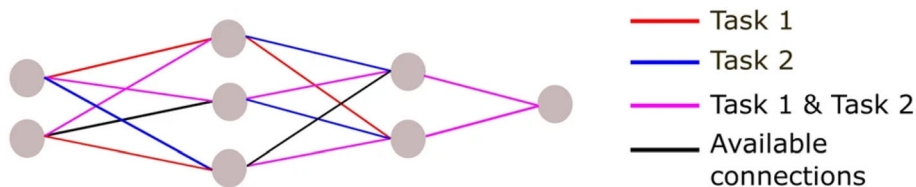
where $\mathcal{F}_k$ is the PDE operator, u_{θ_k} is the predicted solution, and g represents boundary conditions. The total loss is:

$$\mathcal{L} = \sum_{j=1}^k \mathcal{L}_j,$$

with gradients $\frac{\partial \mathcal{L}}{\partial w} = \sum_{j:w \in N_j} \frac{\partial \mathcal{L}_j}{\partial w}$, ensuring only parameters in subnetwork N_j contribute to task j 's gradient. The iPINNs algorithm employs NNrelief, an iterative pruning method, to create sparse subnetworks. For each task k , NNrelief trains the subnetwork, prunes connections based on importance scores:

$$s_{ij}^{(l)} = \frac{|w_{ij}^{(l)} y_i^{(l-1)}|}{\sum_{k=1}^m |w_{kj}^{(l)} y_k^{(l-1)}| + |b_j^{(l)}|},$$

where $w_{ij}^{(l)}$ and $b_j^{(l)}$ are weights and biases, and $y_i^{(l-1)}$ is the input signal. Connections with scores below a threshold (controlled by hyperparameter α) are pruned, followed by retraining



$$\mathcal{L}(\theta; \mathcal{D}_1 \cup \mathcal{D}_2) = \mathcal{L}_1(\theta_1; \mathcal{D}_1) + \mathcal{L}_2(\theta_2; \mathcal{D}_2)$$

Fig. 31 iPINNs framework with two PDE tasks, where subnetworks correspond to distinct tasks. Colors indicate connections: red for task 1, blue for task 2, magenta for shared connections, and black for unassigned connections [15]

to maintain performance. This process allows subnetworks to overlap (shared parameters) while preserving task-specific parameters, as shown in Fig. 31.

The iPINNs approach was validated on two scenarios: (1) learning a family of convection equations with varying β_k , and (2) learning reaction-diffusion by first training on reaction and diffusion components separately. In both cases, iPINNs achieved lower prediction errors than standard PINNs, with improved generalization due to pretraining on simpler tasks. The use of the sine activation function further enhanced performance for modeling complex signals, and the absence of catastrophic forgetting allowed iPINNs to retain solutions for all tasks without architecture extension.

5.6.3 Variable-Separated Networks

Variable-separated networks in PINNs decompose the solution into components that capture distinct physical variables or scales, particularly effective for multi-scale PDEs like blood flow simulations. By tailoring the loss function to each component, this approach mitigates gradient imbalances and enhances optimization efficiency.

Liu et al. [43] proposed AW-vsPINN, a variable-separated PINN with adaptive weighted loss functions for modeling 1D blood flow, governed by the equations:

$$\frac{\partial \hat{A}}{\partial t} + \frac{\partial(\hat{A}\hat{u})}{\partial x} = 0, \quad \frac{\partial(\hat{A}\hat{u})}{\partial t} + \frac{\partial(\hat{A}\hat{u}^2)}{\partial x} + \frac{\hat{A}}{\rho} \frac{\partial \hat{p}}{\partial x} = 0, \quad \hat{p} - p_{\text{ext}} + k(x) \left(\sqrt{\frac{\hat{A}}{A_0}} - 1 \right) = 0,$$

where $\hat{A}$, $\hat{u}$, and $\hat{p}$ are the predicted cross-sectional area, velocity, and pressure, respectively, p_{ext} is the external pressure, ρ is the fluid density, $k(x)$ is the stiffness, and A_0 is the reference area. The AW-vsPINN framework uses sub-neural networks to approximate each variable ($\hat{A}$, $\hat{u}$, $\hat{p}$) via a fully connected feed-forward neural network $vsNN(\mathbf{x}, t; \Theta)$, defined as:

$$\mathcal{N}_0(\mathbf{z}) = \mathbf{z}, \quad \mathcal{N}_l(\mathbf{z}) = \sigma_l(W_l \mathcal{N}_{l-1}(\mathbf{z}) + \mathbf{b}_l), \quad 1 \leq l \leq L-1, \quad \mathcal{N}_L(\mathbf{z}) = W_L \mathcal{N}_{L-1}(\mathbf{z}) + \mathbf{b}_L,$$

where $\mathbf{z} = (\mathbf{x}, t)$, $\mathcal{N}_l(\mathbf{z})$ explicitly denotes the mathematical mapping or output vector of the l -th layer of the network (where $\mathcal{N}_0$ and $\mathcal{N}_L$ represent the initial embedding input and final network output layers, respectively), $\Theta = \{W_l, \mathbf{b}_l\}_{1 \leq l \leq L}$ are the weights and biases, and σ_l is the activation function (e.g., tanh). The network maps $[x, t] \rightarrow [\hat{A}, \hat{u}, \hat{p}]$.

The loss function comprises residuals for the PDE, initial conditions, and boundary conditions:

$$\mathcal{L}_r(\Theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} (\|r_A(x_i^r, t_i^r; \Theta)\|_2^2 + \|r_u(x_i^r, t_i^r; \Theta)\|_2^2 + \|r_p(x_i^r, t_i^r; \Theta)\|_2^2),$$

$$r_A = \frac{\partial \hat{A}}{\partial t} + \frac{\partial(\hat{A}\hat{u})}{\partial x}, \quad r_u = \frac{\partial(\hat{A}\hat{u})}{\partial t} + \frac{\partial(\hat{A}\hat{u}^2)}{\partial x} + \frac{\hat{A}}{\rho} \frac{\partial \hat{p}}{\partial x}, \quad r_p = \hat{p} - p_{\text{ext}} + k(x) \left(\sqrt{\frac{\hat{A}}{A_0}} - 1 \right),$$

$$\mathcal{L}_0(\Theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} \|\mathcal{I}(x_i^0, \hat{A}, \hat{u}, \hat{p})\|_2^2, \quad \mathcal{L}_b(\Theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} \|\mathcal{B}(x_i^b, t_i^b, \hat{A}, \hat{u}, \hat{p})\|_2^2,$$

where $\{x_i^r, t_i^r\}$, $\{x_i^0\}$, and $\{x_i^b, t_i^b\}$ are collocation, initial, and boundary points, respectively, sampled via Sobol sequences or Latin hypercube sampling. The total loss is:

$$\mathcal{L}(\Theta, \lambda) = \lambda_0 \mathcal{L}_0(\Theta) + \lambda_b \mathcal{L}_b(\Theta) + \lambda_r \mathcal{L}_r(\Theta),$$

where $\lambda = \{\lambda_0, \lambda_b, \lambda_r\}$ are the adaptive loss weights updated via a minmax algorithm:

$$\Theta_{k+1} = \Theta_k - \eta_k \nabla_{\Theta_k} \mathcal{L}(\Theta_k, \lambda_k), \quad \lambda_{k+1}^i = \lambda_k^i + \eta_k^i \nabla_{\lambda_k^i} \mathcal{L}(\Theta_k, \lambda_k), \quad i \in \{0, b, r\},$$

with learning rates η_k (for the network parameters Θ) and η_k^i (for the loss weights λ_i) following an inverse time decay schedule. Crucially, to maintain optimization stability, the adaptive loss weights λ_k^i are updated only when their gradient satisfies the condition $\nabla_{\lambda_k^i} \mathcal{L} < \nabla_{\lambda_{k-1}^i} \mathcal{L}$, thereby ensuring algorithmic convergence. The training uses a two-stage approach: Adam optimizer for initial steps, followed by L-BFGS until convergence (relative difference $\epsilon = 10^{-16}$).

Applied to 1D blood flow simulations, AW-vsPINN outperformed standard PINNs, achieving higher prediction accuracy and better generalization by effectively capturing macro-scale vessel dynamics and micro-scale flow patterns. The adaptive weights mitigated gradient imbalances, enhancing convergence stability.

These innovative architectures demonstrate the evolving landscape of PINN design, where the network structure is closely tied to the loss function's effectiveness. By tailoring the architecture to the problem's characteristics, whether through convolutional layers for spatial features, incremental learning for sequential refinement, or variable separation for multi-scale dynamics, these approaches enhance the loss function's ability to guide the network towards accurate and efficient PDE solutions.

5.6.4 Physics-Informed Residual and Adaptive Networks (PirateNets)

While conventional PINNs have demonstrated success on various forward and inverse PDE problems, their performance paradoxically degrades when scaled to larger and deeper network architectures. This counter-intuitive behavior stems from the use of standard Multi-Layer Perceptron (MLP) architectures coupled with conventional initialization schemes (e.g., Glorot or He initialization). As network depth increases, these initializations result in poor trainability of the network's derivatives, leading to severe instability when minimizing the PDE residual loss [64].

To mitigate these gradient pathologies, early architectural modifications introduced residual connections and specialized embedding layers to PINNs [63]. While these modifications

improved gradient flow and mitigated spectral bias, training very deep PINNs remained challenging. To fundamentally resolve this, Wang et al. [64] introduced Physics-Informed Residual Adaptive Networks (PirateNets). PirateNets are designed to facilitate the stable and efficient training of deep PINN models by reimagining how residual connections and network initializations are handled.

The core innovation of PirateNets is the *adaptive residual connection*. In a standard ResNet, residual blocks are fully active from the beginning of training. In contrast, PirateNets introduce a trainable parameter $\alpha^{(l)}$ within the skip connection of the l -th block:

$$x^{(l+1)} = \alpha^{(l)} h^{(l)} + (1 - \alpha^{(l)}) x^{(l)} \quad (3)$$

Crucially, PirateNets initialize $\alpha^{(l)} = 0$ for all blocks. This initialization effectively renders the deep network as a shallow, single-layer network (a linear combination of the first-layer embeddings) at the start of training. Because shallow networks do not suffer from the aforementioned initialization pathologies, the early phase of PDE residual minimization is highly stable. As training progresses via gradient descent, the $\alpha^{(l)}$ parameters are learned, and the network progressively and smoothly deepens, unlocking the representational capacity required to capture complex, non-linear PDE solutions.

Furthermore, because PirateNets act as a linear combination of basis functions (e.g., random Fourier features) at initialization, they provide a unique mechanism to encode physical inductive biases directly into the architecture. Practitioners can initialize the final linear layer using a simple least-squares fit against available initial/boundary data or linearized PDE approximations [64]. This ensures that the deep network starts its non-linear optimization from an optimal initial guess, dramatically enhancing convergence speed, robustness, and the ability to scale PINNs to state-of-the-art depths.

6 Comparative Analysis of Loss Function Designs Across PINN Variants and Applications

Following the comprehensive review of loss function designs in Sect. 3, this section provides a comparative analysis of their practical implementation and performance. The effectiveness of a given loss function is not absolute but is deeply contextual, depending on the specific characteristics of the governing PDE, the nature of the physical domain, and the objectives of the simulation. Here, we systematically evaluate how different loss function strategies—from adaptive weighting to specialized physical formulations—perform across a spectrum of PINN variants and scientific applications. By examining their impact on accuracy, robustness, and computational efficiency in real-world scenarios, we aim to elucidate the critical trade-offs involved and provide a clear framework for selecting the most appropriate design for a given problem.

6.1 Performance Analysis in Key Application Domains and PDE Types

The principle of tailoring loss functions to specific problems is fundamental to advancing PINN capabilities. This approach involves customizing the loss function to incorporate domain-specific knowledge or to address particular problem features—such as discontinuities, multi-scale dynamics, or intricate boundary conditions—enabling PINNs to more effectively solve a wide range of scientific and engineering challenges. This subsection analyzes the

performance of such custom-made designs across various application areas and PDE types, highlighting key advancements.

6.1.1 By Application Area

Fluid Dynamics: Tailored loss functions have been developed to address the unique challenges of modeling complex flow phenomena in fluid dynamics. Hou et al. [30] focused on reconstructing the wake field of the propellers of the ship, a task complicated by the turbulent and highly nonlinear nature of the flow. They introduced a loss function that incorporates both the PDE residuals and additional terms to account for the specific characteristics of wake fields, such as vorticity and pressure gradients. This customization led to more accurate reconstructions of the wake field, as demonstrated in their experiments with real ship propeller data. Similarly, Liu et al. [43] applied PINNs to hemodynamic simulations, where the multi-scale nature of blood flow—from large arteries to microcapillaries—poses significant challenges. They proposed a variable-separated PINN approach, which decomposes the solution into components corresponding to different scales and customizes the loss function accordingly. This method improved the accuracy of blood flow predictions in complex vascular geometries, showcasing the benefits of tailoring the loss function to the specific scales and dynamics of the problem.

Solid Mechanics: Customized loss functions in solid mechanics have been designed to handle challenges such as stress singularities and the accurate prediction of displacement and stress fields. Chen et al. [9] introduced energy-based PINNs, where the loss function is derived from the principle of minimum potential energy rather than directly from the PDE residuals. This approach is particularly effective for problems involving stress singularities, such as those near cracks or sharp corners, as it naturally enforces the physical constraints of the system. Their results on benchmark problems like a cantilever beam with a crack demonstrated superior accuracy in capturing stress concentrations compared to standard PINN formulations. Complementing this, Bai et al. [3] proposed a modified loss function for solid mechanics that incorporates both displacement and stress residuals, weighted to optimize the balance between accuracy in displacement fields and stress predictions. Applied to 2D and 3D elasticity problems, this method achieved higher fidelity in stress field predictions, particularly in regions of high stress gradients, illustrating the value of tailoring the loss to the specific outputs of interest in solid mechanics applications.

Heat Transfer: For heat transfer applications, Farmer et al. [20] developed a tailored loss function for modeling laser bio-effects on skin, a problem characterized by rapid temperature changes and complex boundary conditions. They employed an empirical approach to optimize the weights of the loss terms, using experimental data to guide the weighting strategy. This customization led to more accurate predictions of temperature distributions in skin tissue under laser exposure, as validated against experimental measurements. Their work highlights the importance of tailoring the loss function to incorporate empirical insights, particularly in applications where physical measurements can inform the training process.

Wave Propagation: In wave propagation, Nosrati and Emami Niri [50] focused on the 1D wave equation, a fundamental hyperbolic PDE. They manipulated the loss function to better capture wave field predictions by introducing a custom term that penalizes deviations in the wave speed. This approach improved the accuracy of wave front tracking and reduced numerical dispersion, as demonstrated in their simulations of wave propagation in heterogeneous media. Their study underscores the benefits of tailoring the loss function to the specific characteristics of wave phenomena, such as propagation speed and dispersion.

Geotechnical Applications: Geotechnical applications often involve complex soil behaviors and multi-phase interactions, necessitating customized loss functions. Chen et al. [13] addressed the modeling of water flow in unsaturated soils, where the nonlinear and hysteretic nature of soil–water interactions poses significant challenges. They balanced the loss function to account for both the Richards equation residuals and the constitutive relations governing soil moisture, leading to improved predictions of soil moisture profiles under varying rainfall conditions. Similarly, Zhou et al. [75] applied self-adaptive PINNs to large strain consolidation problems, where the loss function and network architecture are dynamically adjusted to handle the evolving stress and deformation fields. Specifically, the self-adaptive PINN (SA-PINN) framework explicitly adjusts the weight parameters (ω_f , ω_i , ω_b , and ω_d) assigned to the PDE residual, initial conditions, boundary conditions, and data-driven loss components, alongside the adaptive scaling of the slopes (a) of the hidden layer activation functions. These adjustments are determined through a rigorous procedure: initially, a pre-training phase of 50,000 iterations identifies the uncertain parameters to calculate the optimal initial weight values (ω) for each loss term. During the formal training process, the gradients of the total loss with respect to the network parameters (θ) and activation slopes (a) are continuously calculated via backpropagation. Consequently, the SA-PINN synchronously updates the optimal weights, network parameters, activation slopes, and unknown physical parameters (λ_r) based on the minimized loss value at each iteration. Their method enhanced the accuracy of settlement predictions in soft soils, demonstrating the effectiveness of adaptive loss strategies in geotechnical contexts.

Environmental Modeling: In environmental modeling, Jia et al. [32] tailored the loss function for predicting extreme temperatures in greenhouses, a critical factor for crop management. They incorporated additional terms in the loss to account for the thermal dynamics of greenhouse structures and the influence of external weather conditions. This customization improved the accuracy of temperature forecasts, particularly during extreme weather events, as validated against sensor data from operational greenhouses. Their work illustrates the value of tailoring the loss function to incorporate domain-specific environmental factors.

A summary of these application-specific loss function designs is provided in Table 8.

6.1.2 By PDE Type

Customizing loss functions for specific PDE classes allows PINNs to better handle their unique mathematical characteristics.

Parabolic PDEs: Customizing loss functions for specific types of PDEs allows PINNs to better handle the mathematical characteristics unique to each class, such as diffusive behavior in parabolic PDEs or wave propagation in hyperbolic PDEs. For parabolic PDEs, Chen et al. [10] developed PF-PINNs tailored for phase field equations like the Allen-Cahn and Cahn-Hilliard equations. They modified the loss function to emphasize the interfacial dynamics and phase separation processes, leading to more accurate simulations of microstructure evolution in materials science applications. This approach highlights the importance of aligning the loss function with the key physical processes governed by the PDE.

Hyperbolic PDEs: For hyperbolic PDEs, Cassia and Kerswell [7] introduced Godunov loss functions, incorporating Godunov-type flux approximations into the PINN loss to handle discontinuities in conservation laws. Applied to the inviscid Burgers' equation, this method enabled sharper resolution of shocks and better overall accuracy compared to standard PINN formulations, demonstrating the effectiveness of tailoring the loss to the upwind nature of hyperbolic problems.

Table 8 Application-specific PINN loss designs across scientific domains

Domain	References	Description
Fluid dynamics	Hou et al. [30]	Tailored loss for wake field reconstruction in ship propeller wake
	Liu et al. [43]	Customized loss for hemodynamic simulations in blood flow
Solid mechanics	Chen et al. [9]	Energy-based loss designed for stress singularities
	Bai et al. [3]	Modified loss optimized for displacement and stress fields
Heat transfer	Farmer et al. [20]	Empirical loss weight optimization for laser bio-effects on skin
Wave propagation	Nosrati and Emami Niri [50]	Modified loss for wave field prediction in 1D wave equation
Geotechnical applications	Chen et al. [13]	Balanced loss for modeling water flow in unsaturated soils
	Zhou et al. [75]	Self-adaptive loss customized for large strain consolidation
Environmental modeling	Jia et al. [32]	Improved loss for predicting greenhouse extreme temperatures

Nonlinear PDEs: In the realm of nonlinear PDEs, Zhang et al. [73] proposed advanced PINNs for the coupled Schrödinger–KdV system, customizing the loss function to account for the interactions between the two equations. Their approach improved the stability and accuracy of solutions for this complex system, which models wave interactions in plasma physics. Similarly, Chen and Lü [11] applied a weak form-based PINN (PINN-wf) to the Hirota equation, relevant to optical communication and financial modeling. By using a variational formulation in the loss, they achieved higher accuracy in capturing soliton solutions, showcasing the benefits of tailoring the loss to the specific nonlinear dynamics of the PDE.

These problem-tailored designs demonstrate the versatility and adaptability of PINNs, enabling them to tackle a wide range of scientific and engineering challenges by customizing the loss function to the specific needs of the application or PDE type. By leveraging domain knowledge and mathematical insights, these approaches have significantly advanced the state of the art in physics-informed machine learning.

Table 9 summarizes these representative PINN formulations categorized by PDE type.

6.2 Comparative Synthesis and Performance Trade-offs

The detailed applications and PDE-specific examples presented above reveal a clear pattern of performance trade-offs. This comparative synthesis distills these observations into key strengths and limitations inherent to different loss design philosophies:

- **Accuracy:** Adaptive and specialized loss functions consistently outperform standard, fixed-weight designs in complex systems. For instance, self-adaptive strategies [49] and energy-based losses [9] yield significant accuracy improvements in challenging fluid and structural domains by mitigating issues like gradient imbalance and enforcing physical conservation principles.

Table 9 Specialized PINN approaches by PDE type

PDE Type	References	Description
Parabolic PDEs	Chen et al. [10]	Tailored for phase field equations like Allen-Cahn and Cahn-Hilliard
Hyperbolic PDEs	Cassia and Kerswell [7]	Designed for conservation laws using Godunov loss functions
Nonlinear PDEs	Zhang et al. [73]	Customized for coupled nonlinear systems like Schrödinger-KdV
	Chen and Lü [11]	Tailored for communication and finance models using PINN-wf for the Hirota equation

- **Robustness:** Techniques that dynamically balance loss terms, such as gradient-based weighting [40] and optimization-based balancing [4], enhance training stability and resilience against noisy data or stiff PDE systems. These methods prevent individual loss components from dominating the optimization landscape, leading to more reliable convergence.
- **Computational Efficiency:** While sophisticated loss functions can introduce overhead, strategies like domain decomposition [26] and adaptive sampling [46] can improve overall efficiency. By concentrating computational resources in critical regions of the domain (e.g., near shocks or boundaries), they reduce training costs while maintaining or even improving accuracy.
- **Generalizability vs. Specificity:** A primary trade-off exists between broad applicability and problem-specific performance. Standard and general adaptive approaches offer versatility across a wide range of problems. In contrast, specialized designs, such as energy-based or Godunov-type losses, provide superior performance for their target applications but are less transferable to other problem classes.

This analysis underscores that the choice of a loss function is not a one-size-fits-all decision but a strategic choice that depends on the specific goals and constraints of the scientific problem at hand.

7 Future Challenges and Directions in Loss Function Design for PINNs

Despite significant advancements in the design of loss functions for PINNs, several challenges persist, and numerous opportunities for innovation remain. Crucially, as the field matures, it is necessary to distinguish between optimization pathologies that can be mitigated heuristically (e.g., via adaptive weight tuning) and fundamental mathematical limitations inherent to deep learning that remain unresolved by loss formulation alone. As the field evolves, addressing these deeper fundamental challenges will be crucial for enhancing the robustness, scalability, and applicability of PINNs to increasingly complex problems. This section outlines these key limits and proposes novel research directions that could drive the next wave of breakthroughs, moving beyond heuristic fixes.

7.1 Challenges

- **Balancing Competing Loss Terms:** One of the most persistent challenges in PINNs is the effective balancing of loss terms corresponding to data fidelity, boundary conditions, and PDE residuals. The scales of these terms often differ by orders of magnitude, leading to gradient pathologies where one term dominates the optimization process [63]. While adaptive weighting strategies have been proposed, they often require problem-specific tuning or may not generalize well across different PDEs.
- **Handling High-Dimensional and Multi-Scale Problems:** As PINNs are applied to higher-dimensional PDEs or problems with multiple spatial and temporal scales, the loss function must account for the increased complexity. Current methods struggle with the curse of dimensionality and the need to capture phenomena at vastly different scales, which can lead to inefficient training or inaccurate solutions [45].
- **Incorporating Complex Physical Constraints:** Many real-world problems involve PDEs with intricate physical constraints, such as non-linear constitutive laws, multi-physics couplings, or stochastic elements. Designing loss functions that can seamlessly integrate these constraints while maintaining computational tractability remains a significant hurdle [34].
- **Uncertainty Quantification:** While some progress has been made in incorporating UQ into PINNs, such as through Bayesian methods or variance-involved approaches, these techniques often come with increased computational costs and complexity. Developing efficient and scalable UQ methods that can be naturally embedded into the loss function is an ongoing challenge [67].

7.2 Novel Research Directions

To address these challenges and push the boundaries of PINN capabilities, the following research directions are proposed. To provide a pragmatic roadmap for the community, we have critically categorized these directions into paradigms likely to yield near-term practical engineering impacts versus those that remain high-risk, speculative theoretical frontiers:

- **Automated Loss Term Balancing via Reinforcement Learning** [*High-Risk / Speculative*]: Drawing on pre-existing insights in hyperparameter optimization, leveraging reinforcement learning (RL) to dynamically adjust the weights of loss terms during training could offer a more flexible solution. By formulating the weighting process as a sequential decision-making problem [2], RL agents can learn to adapt weights based on the evolving loss landscape. However, this remains highly speculative due to the massive computational overhead and compounded instability of wrapping a delicate PINN training loop inside an inherently sample-inefficient RL environment.
- **Hierarchical and Multi-Fidelity Loss Functions** [*Near-Term Practical Impact*]: For multi-scale problems, hierarchical loss functions that operate at different levels of fidelity could be developed. This approach allows the network to learn coarse-scale features first, progressively refining the solution at finer scales. Because this is heavily inspired by well-established multi-grid methods in traditional numerical solvers, it represents an immediate, highly practical avenue to reduce computational burden. Here, multi-fidelity denotes a strategy that leverages both low-accuracy (rapid and inexpensive) and high-accuracy (computationally intensive) simulations.
- **Physics-Aware Regularization Techniques** [*Near-Term Practical Impact*]: Incorporating physics-based regularization directly into the loss function helps enforce complex

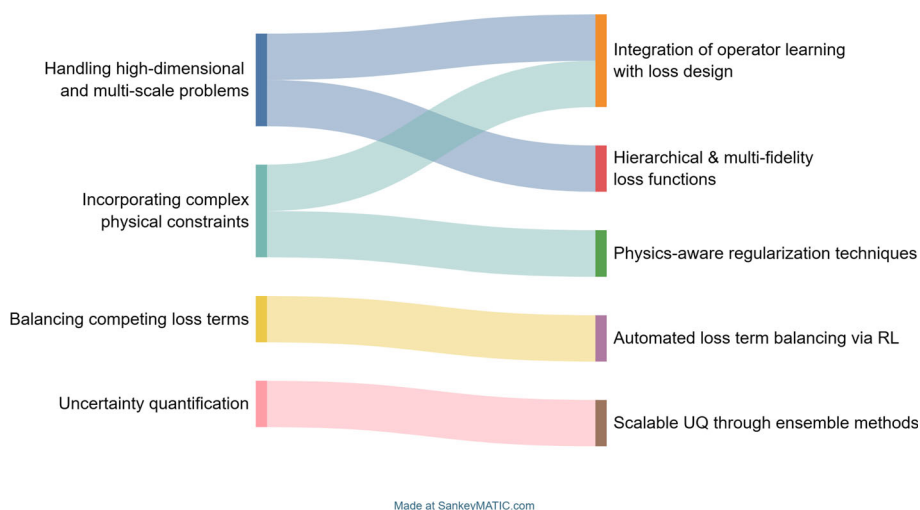


Fig. 32 This Sankey diagram visualizes how specific challenges (left) in the design of loss functions for PINNs map to corresponding research directions (right), offering a structured view of emerging solutions in the field

constraints without explicitly solving them. This is a highly feasible near-term direction; for example, using energy minimization principles or conservation laws as algebraic penalty terms [9] requires minimal architectural changes while forcefully guiding the network towards physically plausible solutions.

- **Integration of Operator Learning with Loss Design** [*High-Risk / Speculative*]: Recent advances in operator learning, such as DeepONets [44], suggest that learning the mapping between function spaces could be integrated into PINN loss functions. While promising, embedding infinite-dimensional operator mappings directly into the gradient flow of a localized PINN loss design requires fundamental architectural overhauls, making it a high-risk mathematical frontier.
- **Scalable UQ through Ensemble Methods** [*Near-Term Practical Impact*]: Developing ensemble-based methods that distribute the computational load across multiple PINN instances provides a scalable approach to UQ. Unlike complex Bayesian inference (e.g., HMC), training a network ensemble with varied loss configurations or initializations is trivially parallelizable. This allows prediction uncertainty to be estimated far more efficiently [39], making it an immediately deployable strategy for robust engineering applications.

These proposed directions aim to address the current limitations of PINN loss functions while opening new avenues for research. By synthesizing cutting-edge techniques from machine learning and numerical analysis, the next generation of loss function designs could significantly enhance the precision, efficiency, and applicability of PINNs, solidifying their role as a powerful tool in scientific computing. Figure 32 visualizes the mapping of core PINN loss function challenges to emerging solution approaches, highlighting the research landscape and its targeted advancements.

8 Conclusion

Over the past decade, the design of loss functions in PINNs has evolved remarkably, transitioning from simple unweighted combinations of data fidelity and physics constraints to advanced formulations incorporating adaptive weighting, weak-form methods, and uncertainty quantification. These innovations have significantly enhanced the precision, computational efficiency, and versatility of PINNs, enabling their successful application across diverse fields such as fluid dynamics, solid mechanics, wave propagation, and environmental modeling. By embedding physical laws directly into the loss function, PINNs have bridged the gap between data-driven approaches and traditional numerical methods, offering mesh-free solutions that respect PDE constraints while leveraging sparse data.

Despite these advancements, substantial challenges persist in the present landscape. Balancing competing loss terms remains delicate, often leading to gradient imbalances that hinder convergence. High-dimensional and multi-scale systems exacerbate optimization complexities, while incorporating intricate physical constraints—such as nonlinear dynamics or stochastic elements—demands greater ingenuity. Addressing these issues is crucial for expanding PINNs' applicability to real-world problems with noisy or incomplete data.

Looking ahead, the future implications of refined loss function designs are profound, promising to unlock unprecedented capabilities in scientific computing. By enabling more robust and scalable PINNs, these advancements could revolutionize fields like climate modeling, materials science, and biomedical engineering, where accurate PDE solutions drive innovation. Transversal research opportunities abound, such as integrating PINNs with reinforcement learning for dynamic loss balancing, enhancing uncertainty quantification through Bayesian or variance-involved methods to provide reliable confidence intervals, and exploring hybrid-symbolic AI to combine neural networks with symbolic regression for interpretable physical models. These interdisciplinary synergies could further mitigate challenges like overfitting and computational cost, fostering hybrid systems that blend the strengths of machine learning and domain-specific knowledge.

As we stand at the threshold of a new era in computational science, the ongoing refinement of loss function designs in PINNs holds immense potential to decipher the intricacies of the physical world. The path forward is both exhilarating and demanding, offering boundless opportunities for pioneering discoveries that could redefine our understanding of nature and propel transformative applications across science and engineering.

9 Appendix: Overview of key advancements in Loss Function Designs in PINNs

The diverse landscape of loss function designs in PINNs, spanning foundational approaches to advanced adaptive and problem-specific strategies, underscores the rapid evolution of the field. For a complete overview of these methods, their advantages, and key references, see Table 10.

Table 10 Comparison of loss function designs in PINNs

Method	Advantages	References
Standard PINN	Simplicity, foundational approach	Raissi et al. [54]
Learning rate annealing	Automated weight tuning, improved convergence	Wang et al. [63]
Self-adaptive PINNs	Dynamic weight optimization, enhanced convergence	McClenny and Braga-Neto [49]
Variational PINNs (VPINNs)	Reduced computational burden, improved stability	Kharazmi et al. [35]
Energy-based PINNs	Better handling of stress singularities	Chen et al. [9]
Godunov-type loss functions	Sharper resolution of discontinuities	Cassia and Kerswell [7]
Sobolev training	Enforced smoothness, reduced artifacts	Yang et al. [67]
Variable-separated PINNs	Improved handling of disparate scales	Liu et al. [43]
Residual & gradient-based sampling	Focus on critical regions, efficient training	Mao and Meng [48]
Hybrid adaptive sampling	Combines residual and domain knowledge	Luo et al. [46]
Active learning for points	Reduces points via uncertainty quantification	Aikawa et al. [1]
Volume-Weighted PINNs	Handles nonuniform point distributions	Song et al. [58]
Adaptive point movement	Concentrates points in high-residual areas	Hou et al. [29]
Self-adaptive loss with Gaussian models	Statistical weight balancing	Xiang et al. [66]
Loss-attentional PINNs	Dynamic weighting via attention	Song et al. [59]
Dynamic weight strategy	Mitigates gradient imbalances	Entropy [40]
Biobjective optimization	Systematic weight balancing	Heldmann et al. [27]
BC-PINNs with coevolutionary algorithms	Explores diverse weight configurations	Zhu et al. [76]
Weight-balanced PINNs (wbPINNs)	Prevents overfitting in weight adjustments	Cao et al. [6]
hp-VPINNs with domain decomposition	Spatial adaptation, subdomain focus	Kharazmi et al. [36]
Adaptive Interface-PINNs	Temporal adaptation, prioritizes rapid changes	Roy et al. [55]
Adaptive weighting for Bayesian PINNs	Prevents overshadowing of small-scale features	Perez et al. [52]

Table 10 continued

Method	Advantages	References
PINN-wf for Hirota equation	Captures soliton solutions accurately	Chen and Lü [11]
Gradient-enhanced PINNs	Smoother solutions, reduces oscillations	Yu et al. [70]
FO-PINNs with first-order formulations	Simplifies training, reduces computational cost	Gladstone et al. [23]
Custom loss for convection-dominated problems	Stabilizes training, captures boundary layers	Frerichs-Mihov et al. [21]
Modified loss with LSWR	Better enforcement in high-stress regions	Bai et al. [3]
Huber norm for NLSE	Balances robustness and smoothness	
Tailored loss for wake field	Accounts for vorticity and pressure gradients	Hou et al. [30]
Empirical loss weight optimization	Incorporates experimental data for weighting	Farmer et al. [20]
Manipulated loss for wave speed	Improves wave front tracking, reduces dispersion	Nosrati and Emami Niri [50]
Balanced loss for unsaturated flow	Handles nonlinear soil–water interactions	Chen et al. [13]
Tailored loss for greenhouse temperature	Accounts for thermal dynamics and weather	Jia et al. [32]
B-PINNs for UQ	Handles noisy data, provides uncertainty estimates	Yang et al. [67]
VI-PINNs with variance estimation	Direct uncertainty estimation, efficient	Shan et al. [56]
Meta-learning for loss optimization	Generalizes across PDE tasks, accelerates convergence	Psaros et al. [53]
Meta-learning for wavefield initialization	Improves initialization, reduces effort	Cheng and Alkhalifah [14]
Adaptive transfer learning	Guides optimization, reduces training time	Liu et al. [41]
Loss landscape engineering	Simplifies optimization, avoids local minima	Gopakumar et al. [25]
Lasso regression in PINN loss	Sparse solutions, interpretability	Ma et al. [47]
Hybrid activation functions	Improves gradient flow, stability	Zafar et al. [72]
Enhanced optimizers for PINNs	Stabilizes updates, improves accuracy	Urbán et al. [60]
VS-PINN with variable scaling	Mitigates gradient imbalances, accelerates convergence	Ko and Park [37]
MRF-PINN with CNN	Captures multi-scale features, improves convergence	Zhang et al. [74]
f-PICNN for space-time PDEs	Unified space-time handling, coherent optimization	Yuan et al. [71]
iPINNs with incremental learning	Sequential refinement, guided optimization	Dekhovitch et al. [15]

Acknowledgements The authors express their sincere gratitude to the handling editor and anonymous reviewers for their constructive comments and insightful suggestions, which have significantly contributed to improving the quality and clarity of this manuscript.

Funding Open access funding provided by Università degli Studi di Napoli Federico II within the CRUI-CARE Agreement. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data Availability No new data were created or analyzed in this study.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose. The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Aikawa, Y., Ueda, N., Tanaka, T.: Improving the efficiency of training physics-informed neural networks using active learning. *N. Gener. Comput.* **42**, 739–760 (2024)
2. Anonymous: Emerging approaches formulation loss balancing as sequential decision-making. In: *ICLR 2024 Workshop on AI for Science* (2024). (Accelerating PINN training via RL-based adaptive loss control)
3. Bai, J., Rabczuk, T., Gupta, A., Alzubaidi, L., Gu, Y.: A physics-informed neural network technique based on a modified loss function for computational 2D and 3D solid mechanics. *Comput. Mech.* **71**, 543–562 (2023)
4. Barreau, M., Shen, H.: Accuracy and robustness of weight-balancing methods for training PINNs. Preprint at [arXiv:2501.18582](https://arxiv.org/abs/2501.18582) (2025)
5. Cai, S., Mao, Z., Wang, Z., Yin, Y., Karniadakis, G.E.: Physics-informed neural networks (PINNs) for fluid mechanics: a review. *Acta. Mech. Sin.* **37**(1), 1727–1738 (2021)
6. Cao, F., Guo, X., Dong, X., Yuan, D.: wbPINN: weight balanced physics-informed neural networks for multi-objective learning. *Appl. Soft Comput.* **170**, 112632 (2025)
7. Cassia, R., Kerswell, R.: Godunov loss functions for modelling of hyperbolic conservation laws. *Comput. Methods Appl. Mech. Eng.* **437**, 117782 (2025)
8. Chatrabgoun, O., Esmailbeigi, M., Cheraghi, M., Daneshkhah, A.: Stable likelihood computation for machine learning of linear differential operators with Gaussian processes. *Int. J. Uncertain. Quantif.* **12**, 75–99 (2022)
9. Chen, J., Ma, J., Zhao, Z., Zhou, X.: Energy-based PINNs using the element integral approach and their enhancement for solid mechanics problems. *Int. J. Solids Struct.* **313**, 113315 (2025)
10. Chen, N., Lucarini, S., Ma, R., Chen, A., Cui, C.: Pf-PINNs: Physics-informed neural networks for solving coupled Allen-Cahn and Cahn-Hilliard phase field equations. *J. Comput. Phys.* **529**, 113843 (2025)
11. Chen, Y., Lü, X.: PINN-wf: A PINN-based algorithm for data-driven solution and parameter discovery of the Hirota equation appearing in communications and finance. *Chaos, Solitons Fractals* **190**, 115669 (2025)
12. Chen, Y., Xiao, H., Teng, X., Liu, W., Lan, L.: Enhancing accuracy of physically informed neural networks for nonlinear Schrödinger equations through multi-view transfer learning. *Inf. Fusion* **102**, 102041 (2024)
13. Chen, Y., et al.: Modeling water flow in unsaturated soils through physics-informed neural network with principled loss function. *Comput. Geotech.* **161**, 105546 (2023)
14. Cheng, S., Alkhalifah, T.: Meta learning for improved neural network wavefield solutions. *Surv. Geophys.* **46**, 145–167 (2025)

15. Dekhovich, A., et al.: iPINNs: incremental learning for physics-informed neural networks. *Eng. Comput.* **41**, 389–402 (2025)
16. Esmailbeigi, M., Cheraghi, M.: Hybrid kernel approach to improving the numerical stability of machine learning for parametric equations with gaussian processes in the noisy and noise-free data assumptions. *Eng. Comput.* **40**, 761–794 (2024)
17. Esmailbeigi, M., Chatrabgoun, O., Daneshkhah, A., Shafa, M.: On the impact of prior distributions on efficiency of sparse Gaussian process regression. *Eng. Comput.* **39**, 2905–2925 (2023)
18. Esmailbeigi, M., Chatrabgoun, O., Cavoretto, R., De Rossi, A., Shafa, M.: Sparse Gaussian process regression via compactly supported kernels: a trade-off between accuracy and computational efficiency. *Inf. Sci.* **717**, 122306 (2025)
19. Ezati, M., Esmailbeigi, M., Kamandi, A.: Novel approaches for hyper-parameter tuning of physics-informed gaussian processes: application to parametric PDEs. *Eng. Comput.* **40**, 3175–3194 (2024)
20. Farmer, J., Oian, C.A., Bowman, B.A., Khan, T.: Empirical loss weight optimization for PINN modeling laser bio-effects on human skin for the 1d heat equation. *Mach. Learn. Appl.* **16**, 100563 (2024)
21. Fericich-Mihov, D., Henning, L., John, V.: On loss functionals for physics-informed neural networks for steady-state convection-dominated convection-diffusion problems. *Commun. Appl. Math. Comput.* **8**(1), 287–308 (2024)
22. Gao, B., Yao, R., Li, Y.: Physics-informed neural networks with adaptive loss weighting algorithm for solving partial differential equations. *Comput. Math. Appl.* **181**, 216–227 (2025)
23. Gladstone, R.J., Nabian, M.A., Sukumar, N., Srivastava, A., Meidani, H.: FO-PINN: first-order formulation. *Eng. Anal. Bound. Elem.* **174**, 106161 (2025)
24. Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. MIT Press (2016)
25. Gopakumar, V., Pamela, S., Samaddar, D.: Loss landscape engineering via data regulation on PINNs. *Mach. Learn. Appl.* **12**, 100464 (2023)
26. Heinlein, A., Klawonn, A., Lanser, M., Weber, J.: Combining machine learning and domain decomposition methods for the solution of partial differential equations—a review. *GAMM-Mitteilungen* **44**(1), e202100001 (2021)
27. Heldmann, F., Berkahn, S., Ehrhardt, M., Klamroth, K.: Pinn training using biobjective optimization: the trade-off between data loss and residual loss. *J. Comput. Phys.* **488**, 112211 (2023)
28. Hotegni, S.S., Berkemeier, M., Peitz, S.: Multi-objective optimization for sparse deep multi-task learning. In 2024 International Joint Conference on Neural Networks (IJCNN). pp. 1–9 (2024). <https://doi.org/10.1109/IJCNN60899.2024.10650994>
29. Hou, J., Li, Y., Ying, S.: Enhancing PINNs for solving PDEs via adaptive collocation point movement and adaptive loss weighting. *Nonlinear Dyn.* **111**, 15233–15261 (2023)
30. Hou, X., Zhou, X., Liu, Y.: Reconstruction of ship propeller wake field based on self-adaptive loss balanced physics-informed neural networks. *Ocean Eng.* **309**, 118341 (2024)
31. Jagtap, A.D., Kawaguchi, K., Karniadakis, G.E.: Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *J. Comput. Phys.* **404**, 109136 (2020)
32. Jia, Y., Qu, L., Huang, S., Chen, X., Li, X.: Better prediction of greenhouse extreme temperature base on improved loss function. *Comput. Electron. Agric.* **227**, 109581 (2024)
33. Jiao, X., Shi, J.C., Chen, X., Xiong, F.: GsPINN: a novel fast green kernel solver based on symmetric physics-informed neural networks. *Neurocomputing* **672**, 132706 (2026)
34. Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Yang, L.: Physics-informed machine learning. *Nat. Rev. Phys.* **3**(6), 422–440 (2021)
35. Kharazmi, E.: Variational physics-informed neural networks for solving partial differential equations. Preprint at [arXiv:1912.00873](https://arxiv.org/abs/1912.00873) (2019)
36. Kharazmi, E., et al.: hp-VPINNs: variational physics-informed neural networks with domain decomposition. *Comput. Methods Appl. Mech. Eng.* **374**, 113547 (2021)
37. Ko, S., Park, S.: VS-PINN: a fast and efficient training of physics-informed neural networks using variable-scaling methods for solving pdes with stiff behavior. *J. Comput. Phys.* **529**, 113860 (2025)
38. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. *Commun. ACM* **60**(6), 84–90 (2017)
39. Lakshminarayanan, B., Pritzel, A., Blundell, C.: Simple and scalable predictive uncertainty estimation using deep ensembles. *Adv. Neural. Inf. Process. Syst.* **30**, 6402–6413 (2017)
40. Li, S., Feng, X.: Dynamic weight strategy of physics-informed neural networks for the 2D Navier-Stokes equations. *Entropy* **24**(9), 1254 (2022). <https://doi.org/10.3390/e24091254>
41. Liu, X., Yao, W., Peng, W., Zhou, W.: Bayesian physics-informed extreme learning machine for forward and inverse PDE problems with noisy data. *Neurocomputing* **549**, 126425 (2023)
42. Liu, Y., Liu, W., Yan, X., Guo, S., Zhang, C.A.: Adaptive transfer learning for PINN. *J. Comput. Phys.* **490**, 112291 (2023)

43. Liu, Y., Cai, L., Chen, Y., Ma, P., Zhong, Q.: Variable separated physics-informed neural networks based on adaptive weighted loss functions for blood flow model. *Comput. Math. Appl.* **153**, 108–122 (2024)
44. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E.: Learning nonlinear operators via deepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **3**(3), 218–229 (2021)
45. Lu, L., Meng, X., Mao, Z., Karniadakis, G.E.: DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.* **63**(1), 208–228 (2021)
46. Luo, K., Liao, S., Guan, Z., Liu, B.: An enhanced hybrid adaptive physics-informed neural network for forward and inverse PDE problems. *Appl. Intell.* **55**, 255 (2025)
47. Ma, M., Fu, L., Guo, X., Zhai, Z.: Incorporating LASSO regression to physics-informed neural network for inverse PDE problem. *Comput. Model. Eng. Sci.* **141**, 385–399 (2024)
48. Mao, Z., Meng, X.: Physics-informed neural networks with residual/gradient-based adaptive sampling methods for solving partial differential equations with sharp solutions. *Appl. Math. Mech.* **44**(7), 1069–1084 (2023)
49. McClenny, L.D., Braga-Neto, U.M.: Self-adaptive physics-informed neural networks. *J. Comput. Phys.* **474**, 111722 (2023)
50. Nosrati, H., Niri, M.E.: Manipulating the loss calculation to enhance the training process of physics-informed neural networks to solve the 1d wave equation. *Eng. Comput.* **40**, 1741–1769 (2024)
51. Perez, S., Poncet, P.: Auto-weighted Bayesian physics-informed neural networks and robust estimations for multitask inverse problems in pore-scale imaging of dissolution. *Comput. Geosci.* **28**, 1175–1215 (2024)
52. Perez, S., Maddu, S., Sbalzarini, I.F., Poncet, P.: Adaptive weighting of Bayesian physics informed neural networks for multitask and multiscale forward and inverse problems. *J. Comput. Phys.* **491**, 112342 (2023)
53. Psaros, A.F., Kawaguchi, K., Karniadakis, G.E.: Meta-learning PINN loss functions. *J. Comput. Phys.* **458**, 111121 (2022)
54. Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019)
55. Roy, S., et al.: Adaptive interface-PINNs (Adal-PINNs) for transient diffusion: applications to forward and inverse problems in heterogeneous media. *Finite Elem. Anal. Des.* **244**, 104305 (2025)
56. Shan, B., Li, Y., Huang, S.-J.: VI-PINNs: Variance-involved physics-informed neural networks for fast and accurate prediction of partial differential equations. *Neurocomputing* **623**, 129360 (2025). <https://doi.org/10.1016/j.neucom.2025.129360>
57. Soleimani, M., Esmailbeigi, M., Cavoretto, R., De Rossi, A.: Analyzing the effects of various isotropic and anisotropic kernels on critical heat flux prediction using gaussian process regression. *Eng. Appl. Artif. Intell.* **133**, 108351 (2024)
58. Song, J., Cao, W., Liao, F., Zhang, W.: VW-PINNs: a volume weighting method for PDE residuals in physics-informed neural networks. *Acta. Mech. Sin.* **41**, 324140 (2025)
59. Song, Y., Wang, H., Yang, H., Taccari, M.L., Chen, X.: Loss-attentional physics-informed neural networks. *J. Comput. Phys.* **501**, 112781 (2024)
60. Urbán, J.F., Stefanou, P., Pons, J.A.: Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be? *J. Comput. Phys.* **523**, 113656 (2025)
61. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. *Adv. Neural. Inf. Process. Syst.* **30**, 5998–6008 (2017)
62. Wang, C., Li, S., He, D., Wang, L.: Is L2 physics informed loss always suitable for training physics informed neural network? *Adv. Neural. Inf. Process. Syst.* **35**, 8278–8290 (2022)
63. Wang, S., Teng, Y., Perdikaris, P.: Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM J. Sci. Comput.* **43**(5), A3055–A3081 (2021)
64. Wang, S., Li, B., Chen, Y., Perdikaris, P.: Piratenets: Physics-informed deep learning with residual adaptive networks. *J. Mach. Learn. Res.* **25**(402), 1–51 (2024)
65. Wu, C., Zhu, M., Tan, Q., Kartha, Y., Lu, L.: A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Comput. Methods Appl. Mech. Eng.* **403**, 115671 (2023)
66. Xiang, Z., Peng, W., Liu, X., Yao, W.: Self-adaptive loss balanced physics-informed neural networks. *Neurocomputing* **496**, 11–34 (2022)
67. Yang, L., Meng, X., Karniadakis, G.E.: B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. *J. Comput. Phys.* **425**, 109913 (2021)
68. Yang, L., Meng, X., Karniadakis, G.E.: Sobolev training for physics-informed neural networks. Preprint at [arXiv:2101.08932](https://arxiv.org/abs/2101.08932) (2021)
69. Yang, Y., He, J.: Deeper or wider: A perspective from optimal generalization error with Sobolev loss. Preprint at [arxiv:2402.00152](https://arxiv.org/abs/2402.00152) (2026)

70. Yu, J., Lu, L., Meng, X., Karniadakis, G.E.: Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. *Comput. Methods Appl. Mech. Eng.* **393**, 114823 (2022)
71. Yuan, B., Wang, H., Heitor, A., Chen, X.: f-PICNN: a physics-informed convolutional neural network for partial differential equations with space-time domain. *J. Comput. Phys.* **515**, 113284 (2024)
72. Zafar, H., Ahmad, X.L., Sadiq, M.N.: Optimizing physics-informed neural networks with hybrid activation functions: a comparative study on improving residual loss and accuracy using partial differential equations. *Chaos Solit. Fractals* **191**, 115727 (2025)
73. Zhang, Q., Qiu, C., Hou, J., Yan, W.: Advanced physics-informed neural networks for numerical approximation of the coupled Schrödinger–KdV equation. *Commun. Nonlinear Sci. Numer. Simul.* **138**, 108229 (2024)
74. Zhang, S., Zhang, C., Han, X., Wang, B.: MRF-PINN: a multi-receptive-field convolutional physics-informed neural network for solving partial differential equations. *Comput. Mech.* **75**(3), 1137–1163 (2025)
75. Zhou, H., Wu, H., Sheil, B., Wang, Z.: A self-adaptive physics-informed neural networks method for large strain consolidation analysis. *Comput. Geotech.* **181**, 107131 (2025)
76. Zhu, Z., Hao, J., Huang, J., Huang, B.: BC-PINN: an adaptive physics informed neural network based on biased multiobjective coevolutionary algorithm. *Neural Comput. Appl.* **35**, 21093–21113 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.